

Mohsin Iban Hossain
AIUB, Algorithm Notes

FINAL TERM

Table of Contents		
Sl. No	Topic	Pages
1	Dynamic Programming	3-6
2	Longest Common Subsequence (LCS)	7-13
3	Matrix Chain Multiplication (MCM)	14-24
4	0/1 Knapsack Problem	25-34
5	Graphs	35-47
6	Breadth First Search (BFS)	48-59
7	Depth First Search (DFS)	60-66
8	Strongly Connected Components (SCC)	67-72
9	Topological Sort	73-84
10	Prim's Algorithm	85-93
11	Kruskal's Algorithm	94-99
12	Dijkstra's Algorithm	100-108
13	Bellman-Ford's Algorithm	109-113
14	All Pair Shortest Path	114-121

DYNAMIC PROGRAMMING

Dynamic Programming

➔ Dynamic Programming (DP) is a way to solve problems more efficiently by breaking them into smaller parts, solving each part once, and reusing the results. It works best for problems that have **overlapping subproblems** (where the same smaller problems are solved multiple times) and an **optimal substructure** (where the overall solution can be built from the solutions of its smaller parts).

How It Works

➔ When a problem can be divided into smaller subproblems, and those subproblems are solved repeatedly, dynamic programming saves their solutions for later use. This avoids redundant calculations and improves the performance of your program.

➤ Example: Fibonacci Sequence

⇒ The Fibonacci sequence is a series where each number is the sum of the two before it. For example:

0, 1, 1, 2, 3, 5...

Let's find the Fibonacci sequence up to the 5th term using DP:

1. Start with the first two numbers: 0 and 1.
2. Add them to get the next number.
3. Continue adding the last two numbers to get the sequence:
 - $0 + 1 = 1$
 - $1 + 1 = 2$
 - $1 + 2 = 3$

So, the Fibonacci sequence up to the 5th term is: **0, 1, 1, 2, 3**.

Dynamic programming makes this process faster by storing the results of smaller calculations and reusing them, instead of recalculating repeatedly.

- **Algorithm:**

Let n be the number of terms.

1. If $n = 0$, return 0.
2. If $n = 1$, return 1.
3. Else, return the sum of two preceding numbers.

⇒ We are calculating the Fibonacci sequence up to the 5th term.

⇒ **First Term:** $F(0) = 0$.

⇒ **Second Term:** $F(1) = 1$.

⇒ **Third Term:** $F(2) = F(1) + F(0) = 1 + 0 = 1$.

⇒ **Fourth Term:** $F(3) = F(2) + F(1) = 1 + 1 = 2$.

⇒ **Fifth Term:** $F(4) = F(3) + F(2) = 2 + 1 = 3$.

Final Sequence: 0, 1, 1, 2, 3.

This uses previously calculated results, which is the essence of the dynamic programming approach.

Recursion vs Dynamic Programming

- **Dynamic Programming:**
 - Often used with recursive algorithms.
 - Helps optimize problems by **solving overlapping subproblems** and **storing their results**.
 - Example: Fibonacci sequence (overlapping subproblems exist).
- **Recursion without Dynamic Programming:**
 - Used for problems without overlapping subproblems.
 - Solves problems by dividing them into independent parts.
 - Example: Merge Sort (subproblems are independent and do not overlap).

In summary, dynamic programming is **useful when overlapping subproblems exist**, while plain **recursion works for divide-and-conquer problems** without overlapping subproblems.

Greedy Algorithms vs Dynamic Programming

- **Similarity:**
Both are optimization techniques used to solve complex problems.
- **Greedy Algorithms:**
 - Make locally optimal choices (greedy decisions) at each step.
 - Aim to find the global optimum but do not guarantee it.
 - May lead to suboptimal solutions if the problem does not have the "greedy-choice property."
 - Example: Finding the minimum number of coins for change using denominations.
- **Dynamic Programming:**
 - Solve subproblems optimally and combine their results to find the global optimum.
 - Always guarantees the best solution (if implemented correctly).
 - Requires problems with overlapping subproblems and optimal substructure.
 - Example: Solving the knapsack problem or longest common subsequence.

Key Difference: Greedy algorithms make decisions on the go, while dynamic programming makes informed choices based on all subproblem solutions.

Different Types of Dynamic Programming Algorithms

1. Longest Common Subsequence. (LCS)
2. Matrix Chain Multiplication. (MCM)
3. 0/1 Knapsack with Multiple Constraints.
4. Fibonacci Sequence.

LONGEST COMMON SUBSEQUENCE (LCS)

Longest Common Subsequence (LCS)

⇒ The **Longest Common Subsequence (LCS)** is the longest sequence that appears in the same relative order in two given sequences, though the elements don't have to be consecutive.

✂ Key Points

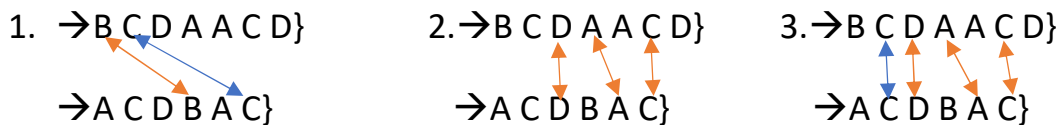
- If S_1 and S_2 are two sequences, Z is a **common subsequence** of S_1 and S_2 if:
 1. Z is a subsequence of both S_1 and S_2 .
 2. The indices of elements in Z follow the same relative order in both S_1 and S_2 .
- A **subsequence** is created by deleting some or no elements from a sequence without changing the order of the remaining elements.

✂ Example

Given Sequences:

- $S_1 = \{B, C, D, A, A, C, D\}$,
- $S_2 = \{A, C, D, B, A, C\}$

Common Subsequences:



Examples of common subsequences include:

1. $\{B, C\}$
2. $\{D, A, C\}$
3. $\{C, D, A, C\}$
4. $\{A, A, C\}$
5. $\{C, D\}$

Longest Common Subsequence:

The longest common subsequence is: $\{C, D, A, C\}$

Using Dynamic Programming to find the LCS

☞ Example 1:

>> Given 2 string of characters. Find the **length of the longest common subsequence (LCS)**. Extract **longest common subsequence** from the table. Describe your answer with simulation.

X=

A	C	A	D	B
---	---	---	---	---

Y=

C	B	D	A
---	---	---	---

Step 1: Calculate the length of X and Y.

→ Length[X]=5 and length[Y]=4

Step 2: Draw a matrix of name L in which (Length+1 = 5+1 = **6 columns**) for X and (Length+1 = 4+1 = **5 rows**) for Y.

Step 3: initialize **first row =0** and **first column =0**.

		C	B	D	A
	0	0	0	0	0
A	0				
C	0				
A	0				
D	0				
B	0				

Step 4: Fill each cell of the table using the following logic.

- I. **If characters match:** Add 1 to the diagonal value and point to it.
- II. **If characters don't match:** Take the maximum value from the previous row or column and point to that cell.

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0				
A	0				
D	0				
B	0				

Step 4.2: Repeat this until the Matrix is filled.

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

Step 5: The **value in the last row and last column** of the table represents the **length of the Longest Common Subsequence (LCS) = 2**.

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

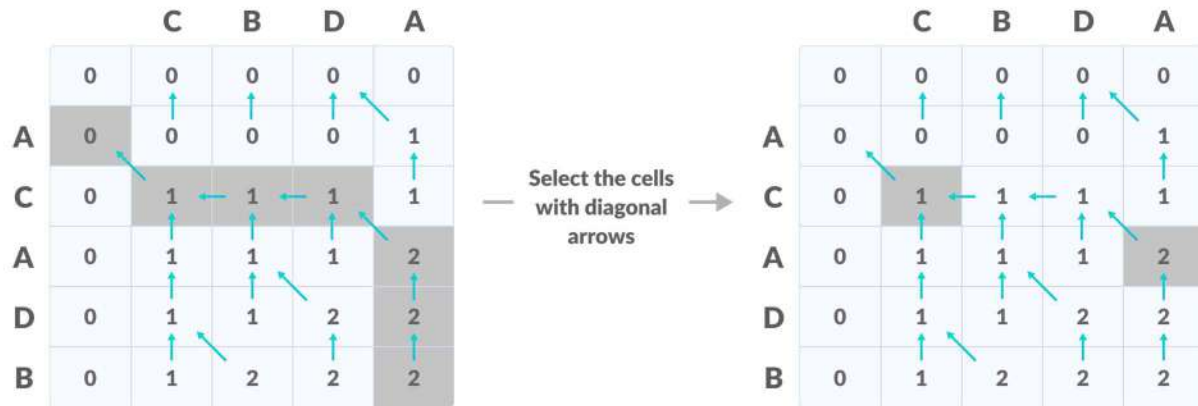
Step 6:

⇒ To find the Longest Common Subsequence (LCS), start from the **last element** of the table (bottom-right corner).

⇒ Follow **the direction of the arrows**:

- If the arrow points **diagonally**, it means the characters match. Add those characters to the LCS.
- If the arrow points **up or left**, move in the direction of the larger value.

• The **characters corresponding to the diagonal arrows** form the **Longest Common Subsequence**.



Step 7: Thus, the longest common subsequence is

C A

✂ Example 2:

>> Given 2 string of characters, **"A= XYZZY"** AND **"B= XXYZXYZ"**. Find the **length of the longest common subsequence (LCS)**. Extract longest common subsequence from the table. Describe your answer with simulation.

Step 1: Calculate the length of A and B.

→ Length[A]=5 and length[B]=7

Step 2: Draw a matrix of name L in which (Length+1 = 7+1 = **8 columns**) for B and

(Length+1 = 5+1 = **6 rows**) for A.

Step 3: initialize **first row =0** and **first column =0**.

B \ A		X	X	Y	Z	X	Y	Z
	0	0	0	0	0	0	0	0
X	0							
Y	0							
Z	0							
Z	0							
Y	0							

Step 4: Fill each cell of the table using the following logic.

- I. **If characters match:** Add 1 to the diagonal value and point to it.
- II. **If characters don't match:** Take the maximum value from the previous row or column and point to that cell.

B \ A		X	X	Y	Z	X	Y	Z
	0	0	0	0	0	0	0	0
X	0	1↖	1↖	1←	1←	1↖	1←	1←
Y	0							
Z	0							
Z	0							
Y	0							

Step 4.2: Repeat this until the Matrix is filled.

B \ A		X	X	Y	Z	X	Y	Z
	0	0	0	0	0	0	0	0
X	0	1↖	1↖	1←	1←	1↖	1←	1←
Y	0	1↑	1↑	2↖	2←	2←	2↖	2←
Z	0	1↑	1↑	2↑	3↖	3←	3←	3↖
Z	0	1↑	1↑	2↑	3↖	3↑	3↑	4↖
Y	0	1↑	1↑	2↖	3↑	3↑	4↖	4↑

Step 5: The **value in the last row and last column** of the table represents the **length of the Longest Common Subsequence (LCS) = 4**.

B \ A		X	X	Y	Z	X	Y	Z
	0	0	0	0	0	0	0	0
X	0	1↖	1↖	1←	1←	1↖	1←	1←
Y	0	1↑	1↑	2↖	2←	2←	2↖	2←
Z	0	1↑	1↑	2↑	3↖	3←	3←	3↖
Z	0	1↑	1↑	2↑	3↖	3↑	3↑	4↖
Y	0	1↑	1↑	2↖	3↑	3↑	4↖	4↑

Step 6:

⇒ To find the Longest Common Subsequence (LCS), start from the **last element** of the table (bottom-right corner).

⇒ Follow the **direction of the arrows**:

- If the arrow points **diagonally**, it means the characters match. Add those characters to the LCS.
- If the arrow points **up or left**, move in the direction of the larger value.

• The **characters corresponding to the diagonal arrows** form the **Longest Common Subsequence**.

B \ A		X	X	Y	Z	X	Y	Z
	0	0	0	0	0	0	0	0
X	0	1↖	1↖	1←	1←	1↖	1←	1←
Y	0	1↑	1↑	2↖	2←	2←	2↖	2←
Z	0	1↑	1↑	2↑	3↖	3←	3←	3↖
Z	0	1↑	1↑	2↑	3↖	3↑	3↑	4↖
Y	0	1↑	1↑	2↖	3↑	3↑	4↖	4↑

Select the cells with diagonal arrows →

B \ A		X	X	Y	Z	X	Y	Z
	0	0	0	0	0	0	0	0
X	0	1↖	1↖	1←	1←	1↖	1←	1←
Y	0	1↑	1↑	2↖	2←	2←	2↖	2←
Z	0	1↑	1↑	2↑	3↖	3←	3←	3↖
Z	0	1↑	1↑	2↑	3↖	3↑	3↑	4↖
Y	0	1↑	1↑	2↖	3↑	3↑	4↖	4↑

Step 7: Thus, the longest common subsequence is:

X Y Z Z

Algorithm:

```

i.    m ← length[A]
ii.   n ← length[B]
iii.  For i ← 0 to m
iv.    Do L[i, 0] ← 0
v.    For j ← 1 to n
vi.     Do L[0, j] ← 0
vii.  For i ← 1 to m
viii.  For j ← 1 to n
ix.    If Ai = Bj
x.      L[i, j] ← L[i-1, j-1] + 1
xi.     b[i, j] ← "↖"
xii.   Else if L[i-1, j] ≥ L[i, j-1]
xiii.    L[i, j] ← L[i-1, j]
xiv.    b[i, j] ← "↑"
xv.    Else
xvi.     L[i, j] ← L[i, j-1]
xvii.    b[i, j] ← "←"
xviii. Return b and L.

```

Time Complexity:

$$\rightarrow O(m \cdot n)$$

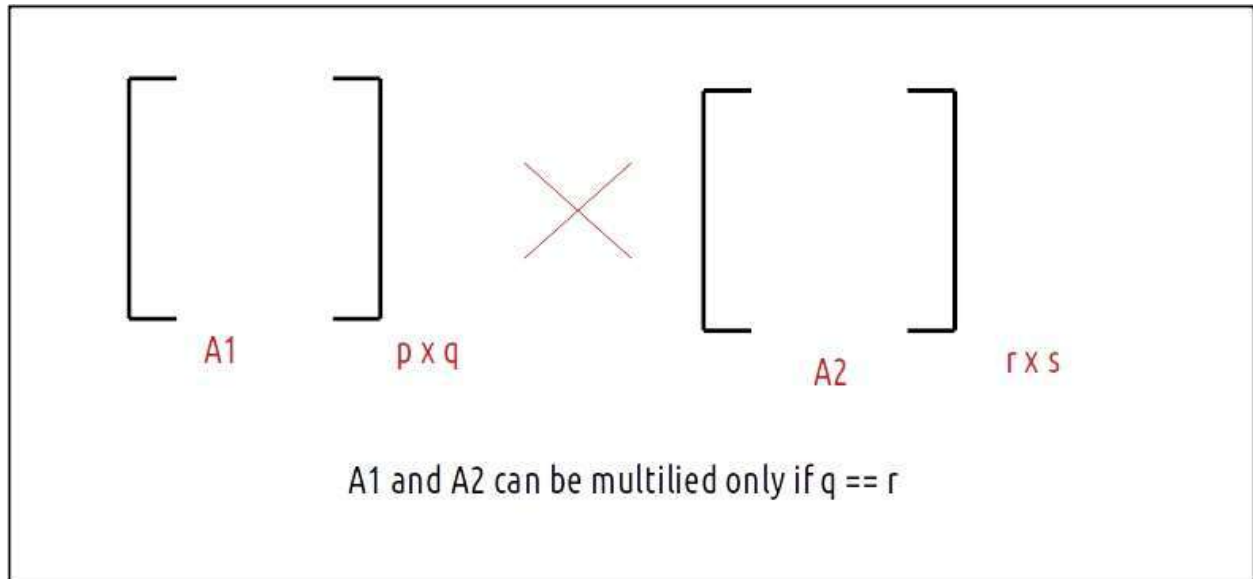
where m & n are the lengths of the two input sequences.

MATRIX CHAIN MULTIPLICATION (MCM)

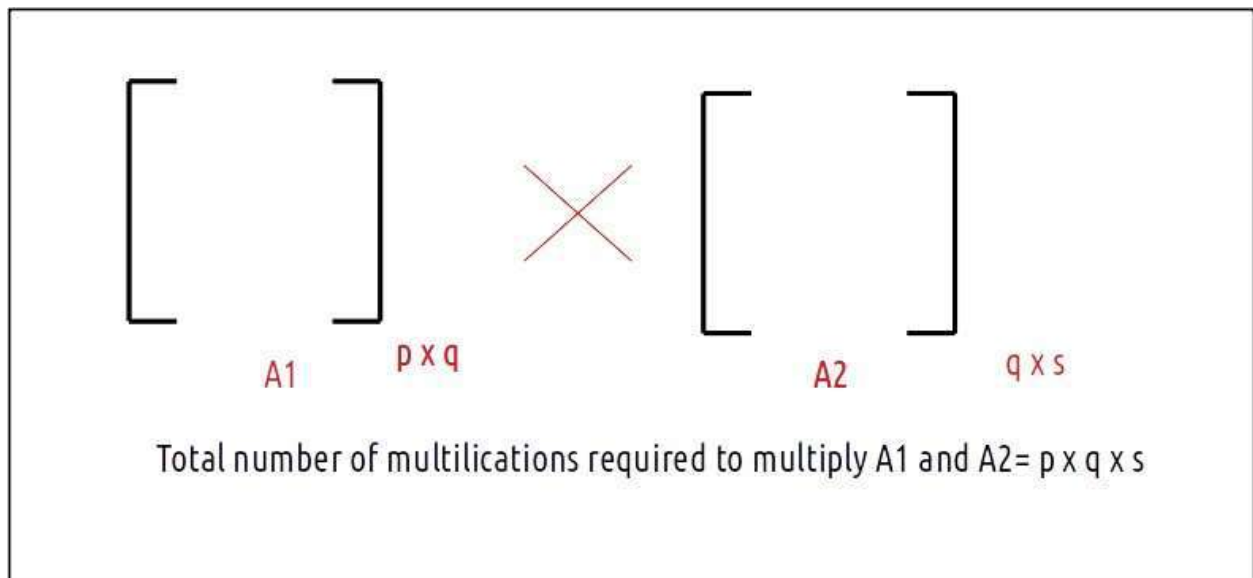
Matrix Chain Multiplication (MCM)

⇒ Let us first understand the problem of matrix chain multiplication. In order to understand the problem, we need to first understand the rules of matrix multiplication:

Step 1:



Step 2:



Introduction

⇒ Matrix Chain Multiplication aims to find the **most efficient order to multiply a sequence** of matrices, **minimizing the total scalar multiplications required**, **without performing the actual multiplications**.

✂ Example 1:

⇒ Let,

Matrix	Dimension
A	2×10
B	10×50
C	50×20

Compute ABC:

⇒ Consider computing A(BC):

- Number of multiplications for (BC) = $10 \times 50 \times 20 = 10000$,
- Number of multiplications for A(BC) = $2 \times 10 \times 20 = 400$,
- Total multiplications = $10000 + 400 = 10400$.

.

⇒ Consider computing (AB)C:

- Number of multiplications for (AB) = $2 \times 10 \times 50 = 1000$,
- Number of multiplications for (AB)C = $2 \times 50 \times 20 = 2000$,
- Total multiplications = $1000 + 2000 = 3000$,

Thus, the (AB)C is the winner because the goal to multiply the fewest number of multiplications necessary to compute the product.

Dynamic Programming Approach

🔗 Example 2:

>> Find the minimum number of operations to multiply 6 matrices, where the dimensions of the matrices are given below.

Matrix	Dimension
T1	30 X 35
T2	35 X 15
T3	15 X 5
T4	5 X 10
T5	10 X 20
T6	20 X 25

⇒ Solution:

Step 1: Identify the Dimensions:

	0	1	2	3	4	5	6
d	30	35	15	5	10	20	25

Step 2: Draw a matrix labeled M and another labeled S.

M	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

S	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

Step 3: Initialize $M[i][i] = 0$;

M	1	2	3	4	5	6
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

Step 4: Now, we determine the number of multiplications necessary for 2 matrices.

Len=2, M[i,j]=[1,2] to [5,6], k = 1 to 5

- $M[1,2] = T1.T2 = M[1,1] + M[2,2] + d_0 * d_1 * d_2 = 0 + 0 + 30 * 35 * 15 = 15750;$
- $M[2,3] = T2.T3 = M[2,2] + M[3,3] + d_1 * d_2 * d_3 = 0 + 0 + 35 * 15 * 5 = 2625;$
- $M[3,4] = T3.T4 = M[3,3] + M[4,4] + d_2 * d_3 * d_4 = 0 + 0 + 15 * 5 * 10 = 750;$
- $M[4,5] = T4.T5 = M[4,4] + M[5,5] + d_3 * d_4 * d_5 = 0 + 0 + 5 * 10 * 20 = 1000;$
- $M[5,6] = T5.T6 = M[5,5] + M[6,6] + d_4 * d_5 * d_6 = 0 + 0 + 10 * 20 * 25 = 5000;$

M	1	2	3	4	5	6
1	0	15750				
2		0	2625			
3			0	750		
4				0	1000	
5					0	5000
6						0

S	1	2	3	4	5	6
1		1				
2			2			
3				3		
4					4	
5						5
6						

Step 4: Now, we determine the number of multiplications necessary for 3 matrices.

Len=3, $M[i,j]=[1,3]$ to $[4,6]$, $k = [1,2]$ to $[4,5]$

➤ $M[1,3] = M[1,1] + M[2,3] + d_0 * d_1 * d_3 = 0 + 2625 + 30 * 35 * 5 = 7875; (\text{min}) (T1.(T2.T3))$

$= M[1,2] + M[3,3] + d_0 * d_2 * d_3 = 15750 + 0 + 30 * 15 * 5 = 18000; ((T1. T2). T3)$

➤ $M[2,4] = M[2,2] + M[3,4] + d_1 * d_2 * d_4 = 0 + 750 + 35 * 15 * 10 = 6000; (T2. (T3.T4))$

$= M[2,3] + M[4,4] + d_1 * d_3 * d_4 = 2625 + 0 + 35 * 5 * 10 = 4375; (\text{min}) ((T2. T3). T4)$

➤ $M[3,5] = M[3,3] + M[4,5] + d_2 * d_3 * d_5 = 0 + 1000 + 15 * 5 * 20 = 2500; (\text{min}) (T3. (T4.T5))$

$= M[3,4] + M[5,5] + d_2 * d_4 * d_5 = 750 + 0 + 15 * 10 * 20 = 3750; ((T3. T4). T5)$

➤ $M[4,6] = M[4,4] + M[5,6] + d_3 * d_4 * d_6 = 0 + 5000 + 5 * 10 * 25 = 6250; (T4. (T5.T6))$

$= M[4,5] + M[6,6] + d_3 * d_5 * d_6 = 1000 + 0 + 5 * 20 * 25 = 3500; (\text{min}) ((T4. T5). T6)$

M	1	2	3	4	5	6
1	0	15750	7875			
2		0	2625	4375		
3			0	750	2500	
4				0	1000	3500
5					0	5000
6						0

S	1	2	3	4	5	6
1		1	1			
2			2	3		
3				3	3	
4					4	5
5						5
6						

Step 5: Now, we determine the number of multiplications necessary for 4 matrices.

Len=4, $M[i,j]=[1,4]$ to $[3,6]$, $k = [1,2,3]$ to $[3,4,5]$

➤ $M[1,4] = M[1,1]+M[2,4]+d_0*d_1*d_4 = 0+ 4375+30*35*10 =14875; (T1.(T2.T3.T4))$

$= M[1,2]+M[3,4]+d_0*d_2*d_4 = 15750+750+30*15*10 =21000; ((T1.T2).(T3.T4))$

$= M[1,3]+M[4,4]+d_0*d_3*d_4 = 7875+ 0+30*5*10 =9375; (\text{min}) ((T1.T2.T3).T4)$

➤ $M[2,5] = M[2,2]+M[3,5]+d_1*d_2*d_5 = 0+2500+35*15*20 = 13000; (T2.(T3.T4.T5))$

$= M[2,3]+M[4,5]+d_1*d_3*d_5 = 2625+1000+35*5*20 = 7125; (\text{min}) ((T2.T3).(T4.T5))$

$= M[2,4]+M[4,5]+d_1*d_4*d_5 = 4375+1000+35*10*20 = 12375; ((T2.T3.T4).T5)$

➤ $M[3,6] = M[3,3]+M[4,6]+d_2*d_3*d_6 = 0+3500+15* 5*25 = 5375; (\text{min}) (T3.(T4.T5.T6))$

$= M[3,4]+M[5,6]+d_2*d_4*d_6 = 750+5000+15* 10*25 = 9500; ((T3.T4).(T4.T5))$

$= M[3,5]+M[6,6]+d_2*d_5*d_6 = 2500+0+15* 20*25 = 10000; ((T3.T4.T5).T6)$

M	1	2	3	4	5	6
1	0	15750	7875	9375		
2		0	2625	4375	7125	
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

S	1	2	3	4	5	6
1		1	1	3		
2			2	3	3	
3				3	3	3
4					4	5
5						5
6						

Step 6: Now, we determine the number of multiplications necessary for 5 matrices.

Len=5, [i,j]=[1,5] to [2,6], k = [1,2,3,4] to [2,3,4,5]

➤ $M[1,5] = M[1,1] + M[2,5] + d_0 * d_1 * d_5 = 0 + 7125 + 30 * 35 * 20 = 28125; (T1.(T2.T3.T4.T5))$
 $= M[1,2] + M[3,5] + d_0 * d_2 * d_5 = 15750 + 2500 + 30 * 15 * 20 = 27250; ((T1.T2).(T3.T4.T5))$
 $= M[1,3] + M[4,5] + d_0 * d_3 * d_5 = 7875 + 1000 + 30 * 5 * 20 = 11875; (\text{min}) ((T1.T2.T3).(T3.T4))$
 $= M[1,4] + M[5,5] + d_0 * d_4 * d_5 = 9375 + 0 + 30 * 10 * 20 = 15375; ((T1.T2.T3.T4).T5)$
➤ $M[2,6] = M[2,2] + M[3,6] + d_1 * d_2 * d_6 = 0 + 5375 + 35 * 15 * 25 = 18500; (T2.(T3.T4.T5.T6))$
 $= M[2,3] + M[4,6] + d_1 * d_3 * d_6 = 2625 + 3500 + 35 * 5 * 25 = 10500; (\text{min}) ((T2.T3).(T4.T5.T6))$
 $= M[2,4] + M[5,6] + d_1 * d_4 * d_6 = 4375 + 5000 + 35 * 10 * 25 = 18125; ((T2.T3.T4).(T5.T6))$
 $= M[2,5] + M[6,6] + d_1 * d_5 * d_6 = 0 + 5375 + 35 * 20 * 25 = 22875; ((T2.T3.T4.T5).T6)$

M	1	2	3	4	5	6
1	0	15750	7875	9375	11875	
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

S	1	2	3	4	5	6
1		1	1	3	3	
2			2	3	3	3
3				3	3	3
4					4	5
5						5
6						

Step 7: Now, we determine the number of multiplications necessary for 6 matrices.

Len=6, [i,j]=[1,6], k = [1,2,3,4,5]

$M[1,6] = M[1,1] + M[2,6] + d_0 * d_1 * d_6 = 0 + 10500 + 30 * 35 * 25 = 36750$; (T1.(T2.T3.T4.T5.T6))

$= M[1,2] + M[3,6] + d_0 * d_2 * d_6 = 15750 + 5375 + 30 * 15 * 25 = 32375$; ((T1.T2).(T3.T4.T5.T6))

$= M[1,3] + M[4,6] + d_0 * d_3 * d_6 = 7875 + 3500 + 30 * 5 * 25 = 15125$; (min) ((T1.T2.T3).(T4.T5.T6))

$= M[1,4] + M[5,6] + d_0 * d_4 * d_6 = 9375 + 5000 + 30 * 10 * 25 = 21875$; ((T1.T2.T3.T4).(T5.T6))

$= M[1,5] + M[6,6] + d_0 * d_5 * d_6 = 11875 + 0 + 30 * 20 * 25 = 26875$; ((T1.T2.T3.T4.T5).T6)

M	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

S	1	2	3	4	5	6
1		1	1	3	3	3
2			2	3	3	3
3				3	3	3
4					4	5
5						5
6						

Step 8: Constructing Optimal Solution:

To compute the optimal multiplication order for $T_1 \dots T_6$ using $S[\text{matrix}]$:

1. Start with the overall split:

$$T_{1..6} = (T_{1..3} \cdot T_{4..6}) \quad (\text{since } S[1,6]=3)$$

2. Break down each subproblem further:

$$T_{1..3} = (T_{1..1} \cdot T_{2..3}), \quad T_{4..6} = (T_{4..5} \cdot T_{6..6}) \quad (\text{since } S[1,3]=1 \text{ and } S[4,6]=5)$$

3. Substitute the splits:

$$T_{1..6} = ((T_1 \cdot (T_2 \cdot T_3)) \cdot ((T_4 \cdot T_5) \cdot T_6)).$$

$$\therefore \text{Optimal Solution} = ((T_1) \cdot (T_2 \cdot T_3)) \cdot ((T_4 \cdot T_5) \cdot (T_6))$$

Algorithm:

```
Matrix-Chain-Order( d )  
01 n = length[d]-1  
02 for i = 1 to n do  
03     M[i,i] = 0  
04 for len = 2 to n do  
05     for i = 1 to n-len+1 do  
06         j = i+len-1  
07         M[i,j] = ∞  
08         for k = i to j-1 do  
09             q = M[i,k]+M[k+1,j]+di-1dkdj  
10             if q < M[i,j] then  
11                 M[i,j] = q  
12                 s[i,j] = k  
13 return M and s
```

Time Complexity:

$$\rightarrow O(n^3)$$

“0/1 KNAPSACK PROBLEM”

0/1 Knapsack Problem

⇒ In 0/1 Knapsack Problem,

- As the name suggests, items are indivisible here.
- We cannot take the fraction of any item.
- We have to either take an item completely or leave it completely.

0-1 Knapsack Algorithm:

```
for w = 0 to W
  B[0,w] = 0
for i = 1 to n
  B[i,0] = 0
  for i = 1 to n
    for w = 0 to W
      if  $w_i \leq w$  // item i can be part of the solution
        if  $b_i + B[i-1, w-w_i] > B[i-1, w]$ 
           $B[i, w] = b_i + B[i-1, w-w_i]$ 
        else
           $B[i, w] = B[i-1, w]$ 
      else  $B[i, w] = B[i-1, w]$  //  $w_i > w$ 
```

//w = Knapsack weight capacity
//W = knapsack Max weight
//B =Max Benefit value

Time Complexity:

Consider-

- Knapsack weight capacity = w
- Number of items each having some weight and value = n

∴ Time complexity = $\theta(nw)$

PRACTICE PROBLEM BASED ON 0/1 KNAPSACK PROBLEM

✂ Example 1:

- ⇒ For the given set of items and **knapsack capacity = 5 kg**, find the optimal solution for the 0/1 knapsack problem making use of dynamic programming approach.

Item	Weight	Value
1	2	3
2	3	4
3	4	5
4	5	6

OR

- ⇒ Find the optimal solution for the 0/1 knapsack problem making use of dynamic programming approach. Consider-

$$n = 4$$

$$w = 5 \text{ kg}$$

$$(w_1, w_2, w_3, w_4) = (2, 3, 4, 5)$$

$$(b_1, b_2, b_3, b_4) = (3, 4, 5, 6)$$

OR

- ⇒ A thief enters a house for robbing it. He can carry a maximal weight of 5 kg into his bag. There are 4 items in the house with the following weights and values. What items should thief take if he either takes the item completely or leaves it completely?

Item	Weight (kg)	Value (\$)
Mirror	2	3
Silver nugget	3	4
Painting	4	5
Vase	5	6

→ Solution: -

Step (1)

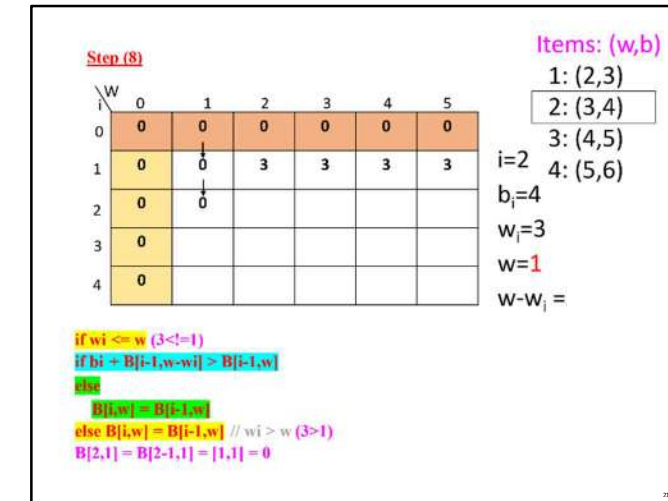
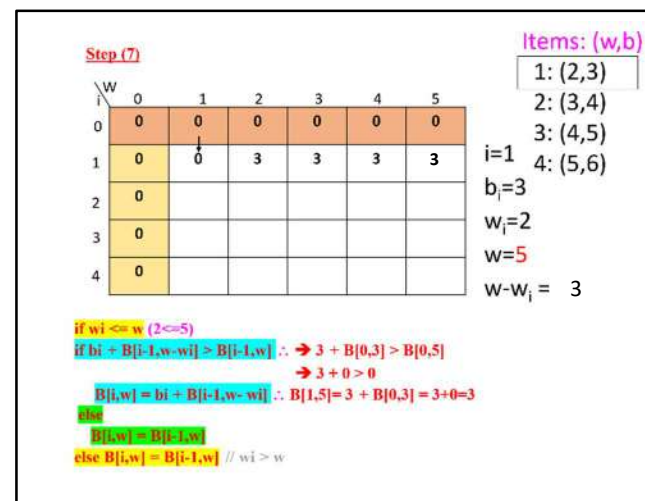
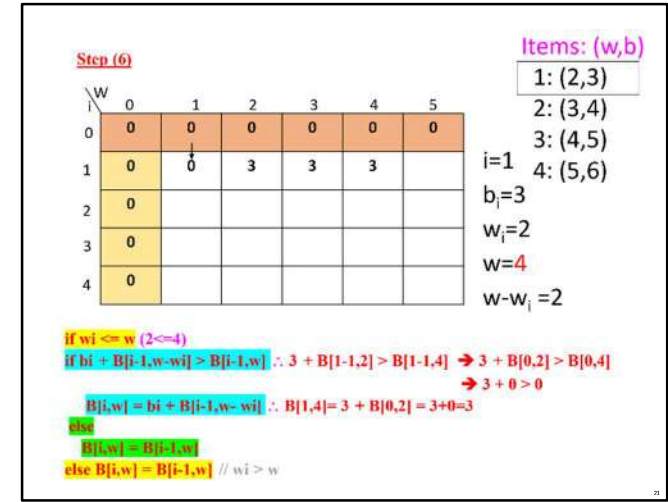
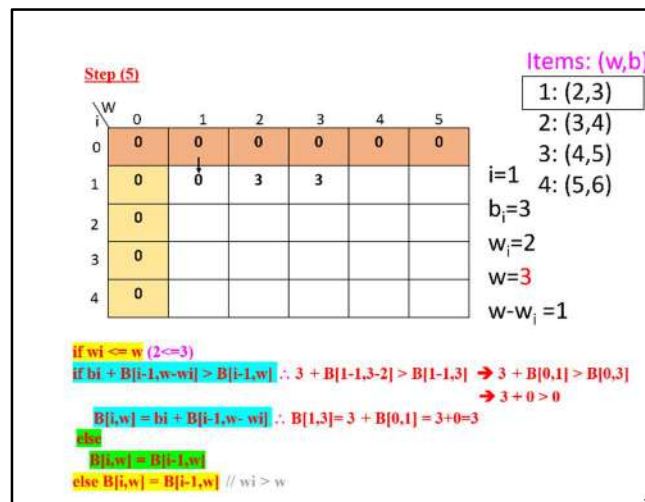
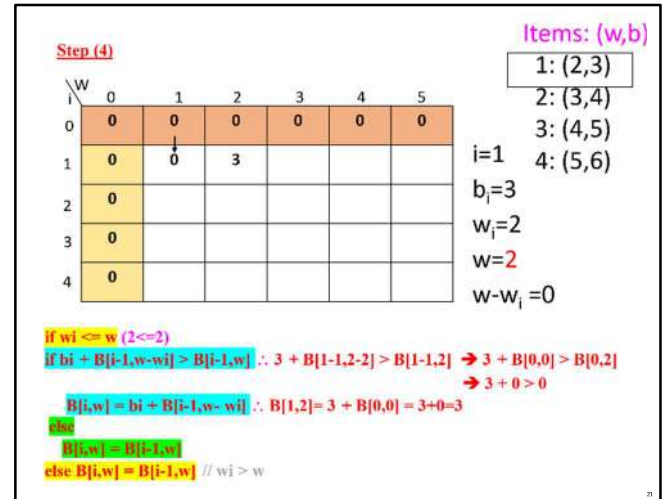
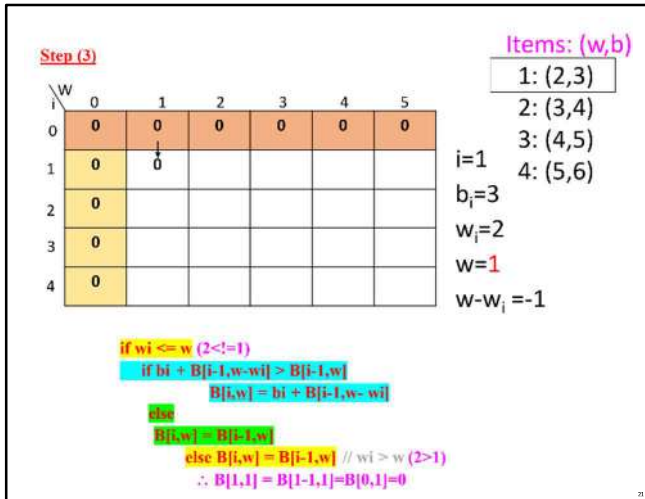
i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1						
2						
3						
4						

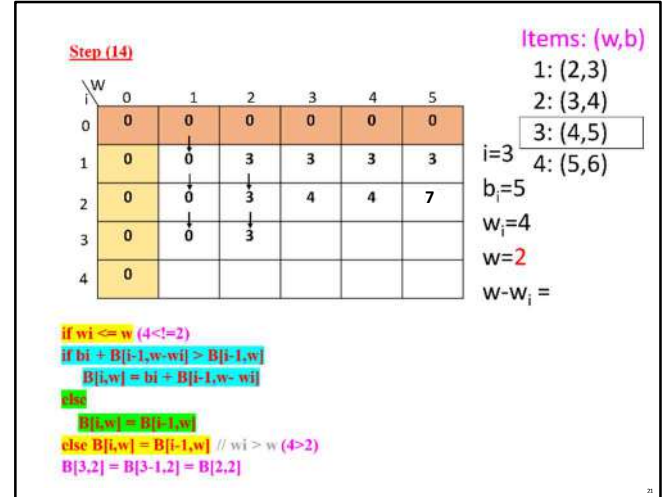
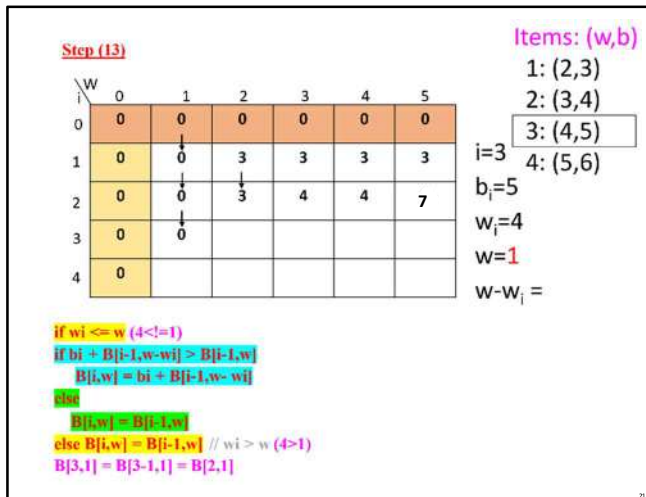
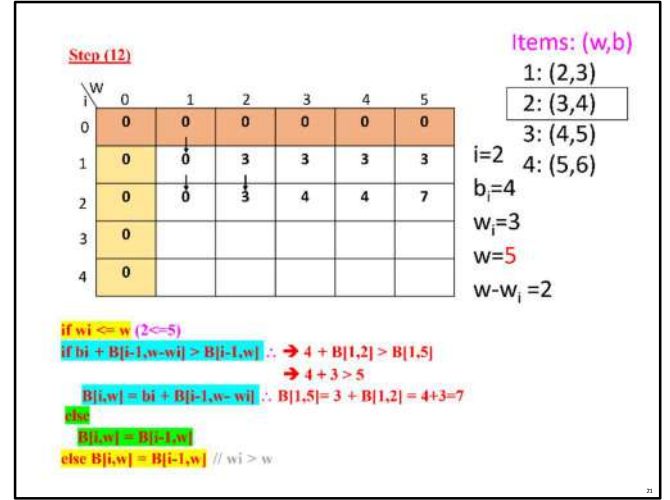
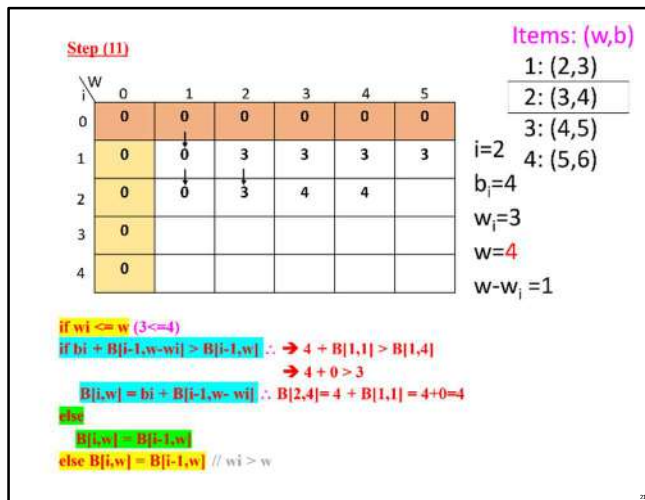
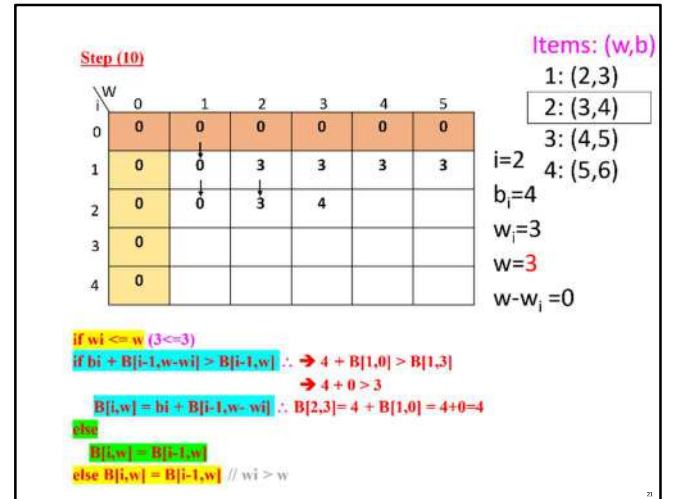
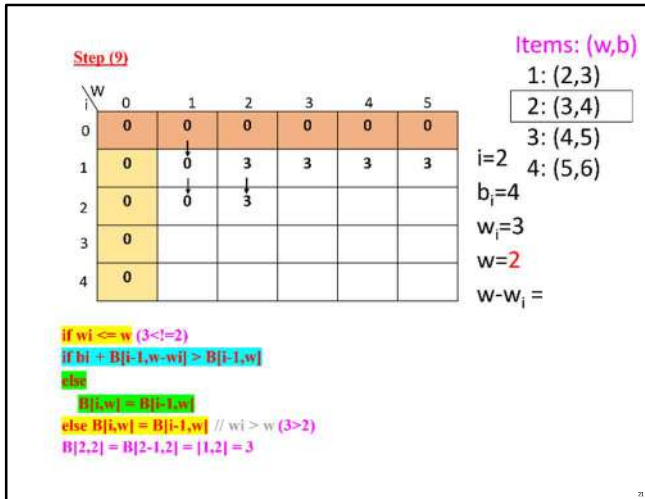
for $w = 0$ to W
 $B[0, w] = 0$

Step (2)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

for $i = 1$ to n
 $B[i, 0] = 0$





Step (15)

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4		
4	0					

Items: (w,b)
 1: (2,3)
 2: (3,4)
 3: (4,5)
 4: (5,6)

$i=3$
 $b_i=5$
 $w_i=4$
 $w=3$
 $w-w_i=$

if $w_i \leq w$ ($4 \leq 3$)
 if $b_i + B[i-1, w-w_i] > B[i-1, w]$
 $B[i, w] = b_i + B[i-1, w-w_i]$
 else
 $B[i, w] = B[i-1, w]$
 else $B[i, w] = B[i-1, w]$ // $w_i > w$ ($4 > 3$)
 $B[3, 3] = B[3-1, 3] = B[2, 3]$

Step (16)

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	
4	0					

Items: (w,b)
 1: (2,3)
 2: (3,4)
 3: (4,5)
 4: (5,6)

$i=3$
 $b_i=5$
 $w_i=4$
 $w=4$
 $w-w_i=0$

if $w_i \leq w$ ($4 \leq 4$)
 if $b_i + B[i-1, w-w_i] > B[i-1, w]$ $\therefore 5 + B[2, 0] > B[2, 4]$
 $\rightarrow 5 + 0 > 4$
 $B[i, w] = b_i + B[i-1, w-w_i]$ $\therefore B[3, 4] = 5 + B[2, 0] = 5 + 0 = 5$
 else
 $B[i, w] = B[i-1, w]$
 else $B[i, w] = B[i-1, w]$ // $w_i > w$

Step (17)

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0					

Items: (w,b)
 1: (2,3)
 2: (3,4)
 3: (4,5)
 4: (5,6)

$i=3$
 $b_i=5$
 $w_i=4$
 $w=5$
 $w-w_i=1$

if $w_i \leq w$ ($5 \leq 5$)
 if $b_i + B[i-1, w-w_i] > B[i-1, w]$ $\therefore 5 + B[2, 1] > B[2, 5]$
 $\rightarrow 5 + 0 > 7$
 $B[i, w] = b_i + B[i-1, w-w_i]$
 else
 $B[i, w] = B[i-1, w]$ $\therefore B[3, 5] = B[2, 5] = 7$
 else $B[i, w] = B[i-1, w]$ // $w_i > w$

Step (18)

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0				

Items: (w,b)
 1: (2,3)
 2: (3,4)
 3: (4,5)
 4: (5,6)

$i=4$
 $b_i=6$
 $w_i=5$
 $w=1$
 $w-w_i=$

if $w_i \leq w$ ($5 \leq 1$)
 if $b_i + B[i-1, w-w_i] > B[i-1, w]$
 $B[i, w] = b_i + B[i-1, w-w_i]$
 else
 $B[i, w] = B[i-1, w]$
 else $B[i, w] = B[i-1, w]$ // $w_i > w$ ($5 > 1$)
 $B[4, 1] = B[4-1, 1] = B[3, 1] = 0$

Step (19)

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3			

Items: (w,b)
 1: (2,3)
 2: (3,4)
 3: (4,5)
 4: (5,6)

$i=4$
 $b_i=6$
 $w_i=5$
 $w=2$
 $w-w_i=$

if $w_i \leq w$ ($5 \leq 2$)
 if $b_i + B[i-1, w-w_i] > B[i-1, w]$
 $B[i, w] = b_i + B[i-1, w-w_i]$
 else
 $B[i, w] = B[i-1, w]$
 else $B[i, w] = B[i-1, w]$ // $w_i > w$ ($5 > 2$)
 $B[4, 2] = B[4-1, 2] = B[3, 2] = 3$

Step (20)

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4		

Items: (w,b)
 1: (2,3)
 2: (3,4)
 3: (4,5)
 4: (5,6)

$i=4$
 $b_i=6$
 $w_i=5$
 $w=3$
 $w-w_i=$

if $w_i \leq w$ ($5 \leq 3$)
 if $b_i + B[i-1, w-w_i] > B[i-1, w]$
 $B[i, w] = b_i + B[i-1, w-w_i]$
 else
 $B[i, w] = B[i-1, w]$
 else $B[i, w] = B[i-1, w]$ // $w_i > w$ ($5 > 3$)
 $B[4, 3] = B[4-1, 3] = B[3, 3] = 4$

Step (21)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	

Items: (w,b)
 1: (2,3)
 2: (3,4)
 3: (4,5)
 4: (5,6)

$i=4$
 $b_i=6$
 $w_i=5$
 $w=4$
 $w-w_i=$

if $w_i \leq w$ ($5 \leq 4$)
 if $b_i + B[i-1, w-w_i] > B[i-1, w]$
 $B[i, w] = b_i + B[i-1, w-w_i]$
 else
 $B[i, w] = B[i-1, w]$
 else $B[i, w] = B[i-1, w]$ // $w_i > w$ ($5 > 4$)
 $B[4, 4] = B[4-1, 4] = B[3, 4] = 5$

Step (22)

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Items: (w,b)
 1: (2,3)
 2: (3,4)
 3: (4,5)
 4: (5,6)

$i=4$
 $b_i=6$
 $w_i=5$
 $w=5$
 $w-w_i=0$

if $w_i \leq w$ ($5 \leq 5$)
 if $b_i + B[i-1, w-w_i] > B[i-1, w]$ $\therefore 6 + B[3, 0] > B[3, 5]$
 $\rightarrow 6 + 0 > 7$
 $B[i, w] = b_i + B[i-1, w-w_i]$
 else
 $B[i, w] = B[i-1, w]$ $\therefore B[4, 5] = B[3, 5] = 7$
 else $B[i, w] = B[i-1, w]$ // $w_i > w$

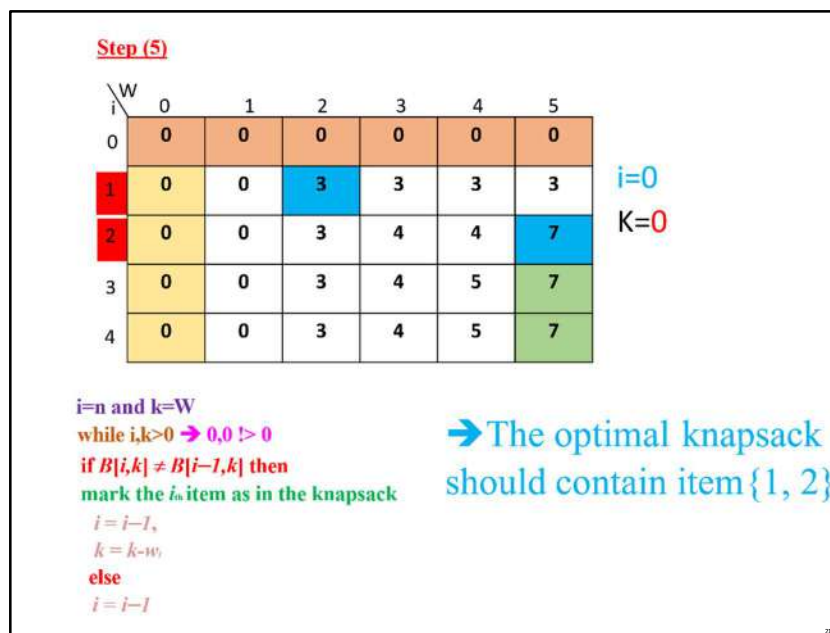
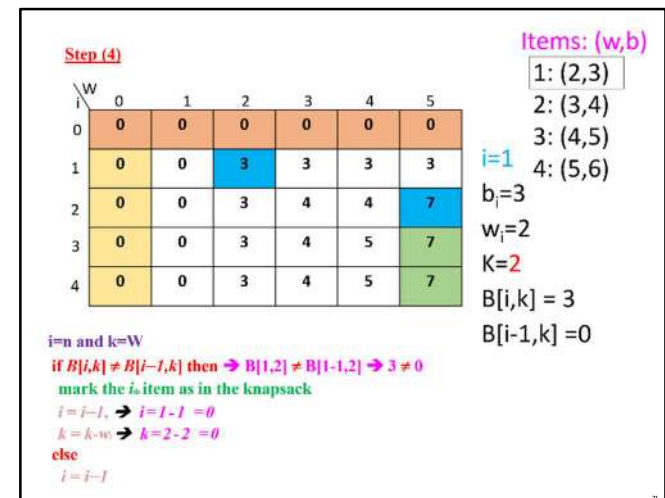
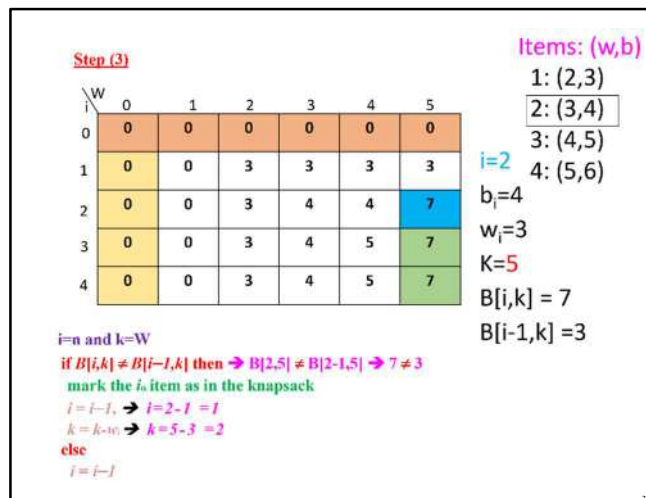
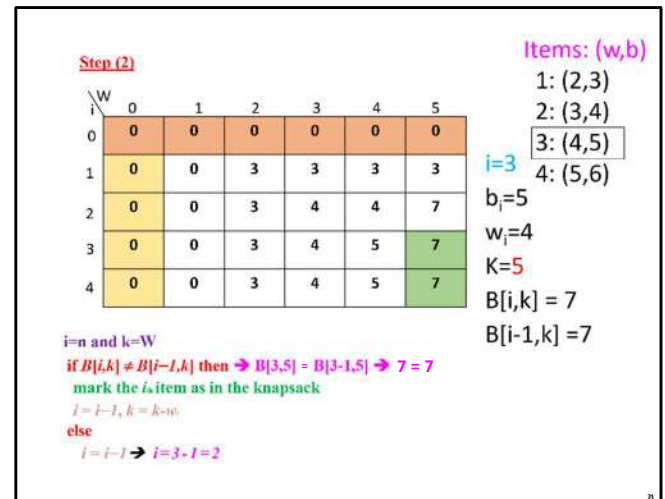
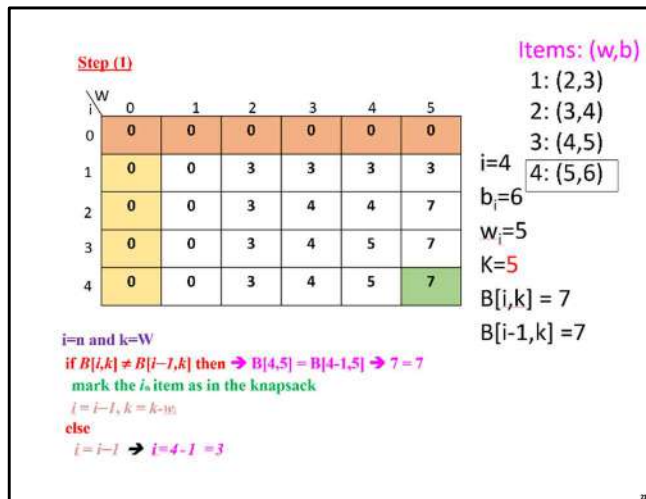
Comments

- ◆ This algorithm only finds the **max possible value that can be carried in the knapsack**
- ◆ To know the **items** that make this maximum value, an addition to this algorithm is necessary.

Finding the Items

How to find actual Knapsack Items

- ◆ All of the information we need is in the table.
- ◆ $B[n, W]$ = maximal value of items that can be placed in the Knapsack.
- ◆ Let $i=n$ and $k=W$
 while $i, k > 0$
 if $B[i, k] \neq B[i-1, k]$ then
 mark the i^{th} item as in the knapsack
 $i = i-1, k = k-w_i$
 else
 $i = i-1$ // Assume the i^{th} item is not in the knapsack



✂ Example 2:

⇒ For the given set of items and **knapsack capacity = 10 kg**, find the optimal solution for the 0/1 knapsack problem making use of dynamic programming approach.

Item	Weight	Value
1	2	3
2	3	4
3	4	5
4	5	6
5	6	7

Solution: -

i \ W											
		1	2	3	4	5	6	7	8	9	10
i		0	0	0	0	0	0	0	0	0	0
	1	0	0	3	3	3	3	3	3	3	3
	2	0	0	3	4	4	7	7	7	7	7
	3	0	0	3	4	5	7	8	9	9	12
	4	0	0	3	4	5	7	8	9	10	12
5		0	0	3	4	5	7	8	9	10	12

Max Benefit =13
&
Item Taken = 1,2,4

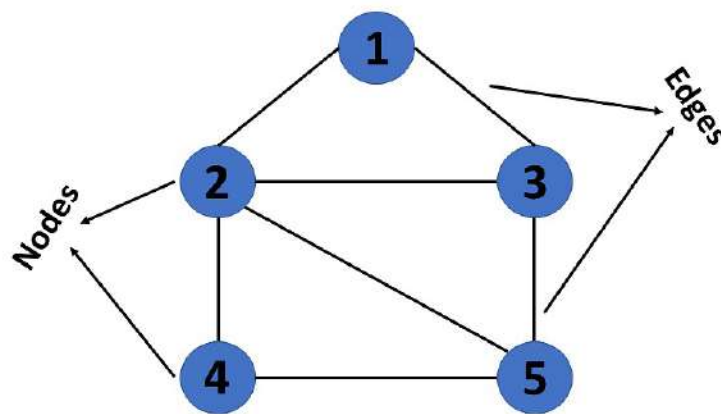
GRAPHS

Graphs

Graph: A graph $G = (V, E)$ is composed of:

V: set of vertices (Nodes).

E: set of edges connecting the vertices in V.



Here,

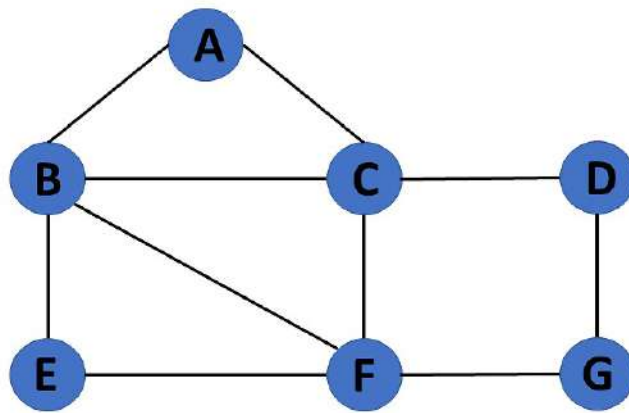
vertices $V = \{1, 2, 3, 4, 5\}$

edges $E = \{(1,2), (1,3), (2,3), (2,4), (2,5), (3,5), (4,5)\}$.

Types of Graphs

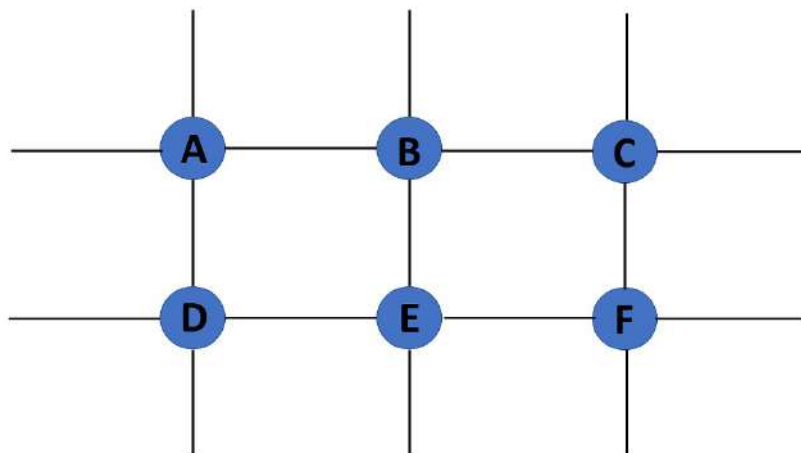
1. Finite Graph:

⇒ A graph $G = (V, E)$ is called a **finite graph** if the number of vertices (V) and edges (E) in the graph is limited.



2. Infinite Graph:

⇒ The graph $G = (V, E)$ is called an **infinite graph** if the number of vertices and edges in the graph is interminable.



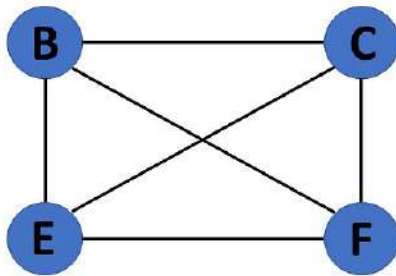
3. Trivial Graph:

⇒ A graph $G = (V, E)$ is **trivial** if it contains only a **single** vertex and **no** edges.



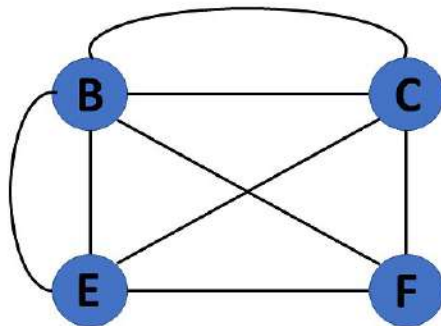
4. Simple Graph:

⇒ A graph $G = (V, E)$ is a **simple graph** if **each pair of vertices has only one edge**.



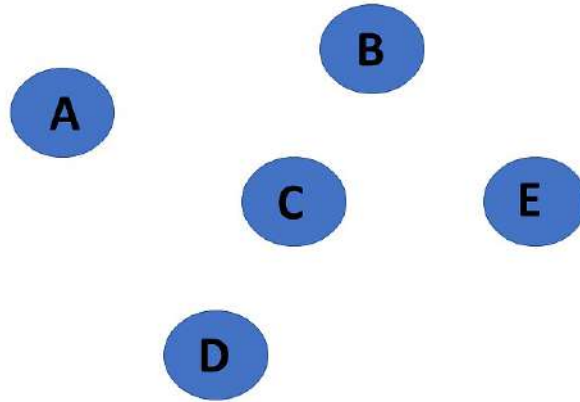
5. Multi Graph:

⇒ A graph $G = (V, E)$ is a **multigraph** if **there are multiple edges between a pair of vertices, with no self-loops**.



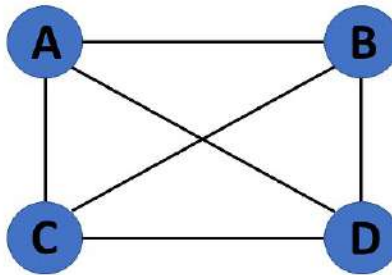
6. Null Graph:

⇒ A graph $G = (V, E)$ is a **null graph** if it has **several vertices** but **no edges**.



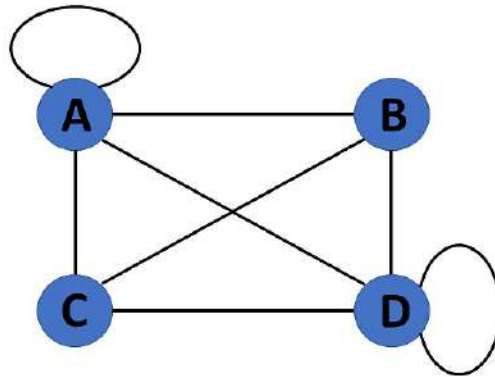
7. Complete Graph:

⇒ A graph $G = (V, E)$ is a **complete graph** if it is a simple graph where **every pair of vertices is connected by an edge**, and **each vertex has a degree of $n-1$** , where n is the number of vertices.



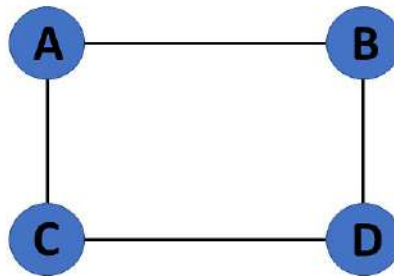
8. Pseudo Graph:

⇒ If a graph $G = (V, E)$ **contains a self-loop** besides other edges, it is a pseudograph.



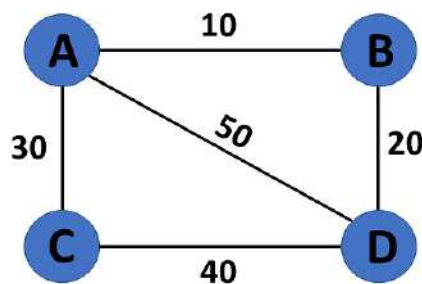
9. Regular Graph:

⇒ A graph $G = (V, E)$ is a **regular graph** if **it is a simple graph where all vertices have the same degree.**



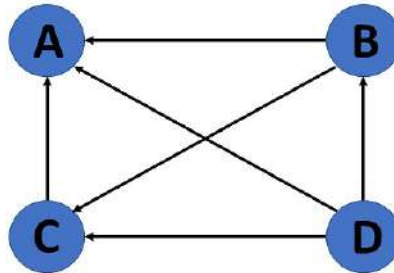
10. Weighted Graph:

⇒ A graph $G = (V, E)$ is a **weighted graph** if **each edge has a value or weight representing the cost of traversal.**



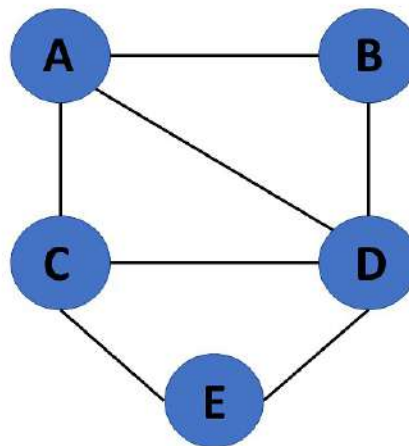
11. Directed Graph:

⇒ A directed graph (digraph) is a graph where edges have a **specific direction**, connecting an ordered pair of nodes.



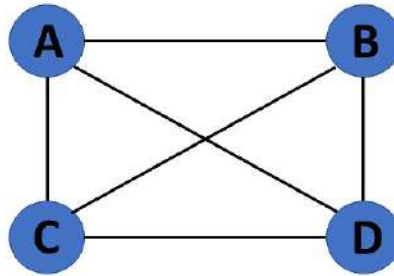
12. Undirected Graph:

⇒ An **undirected graph** consists of **nodes connected by edges without direction**, where the order of connected vertices is irrelevant.



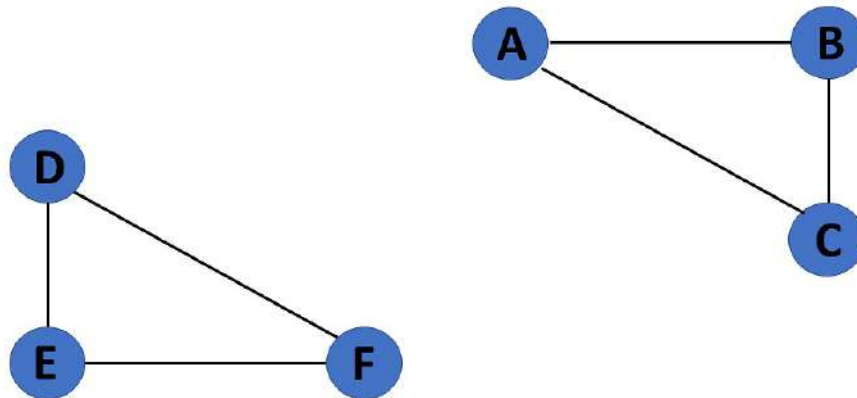
13. Connected Graph:

⇒ A graph is connected **if there is a path between any pair of vertices.**



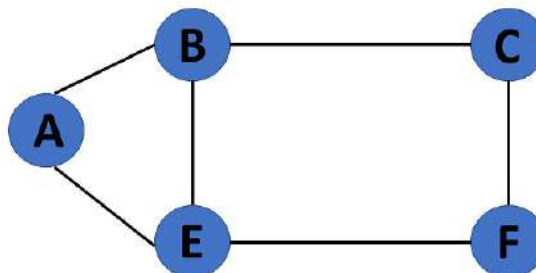
14. Disconnected Graph:

⇒ A **disconnected graph** is **a graph where no edges link some or all of the vertices**, also referred to as a null graph in this context.



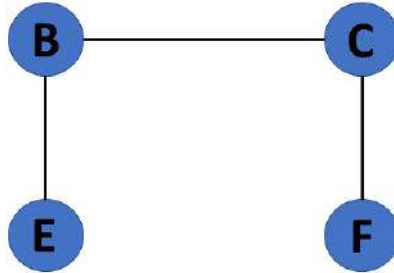
15. Cyclic Graph:

⇒ A graph is cyclic if it **contains at least one cycle.**



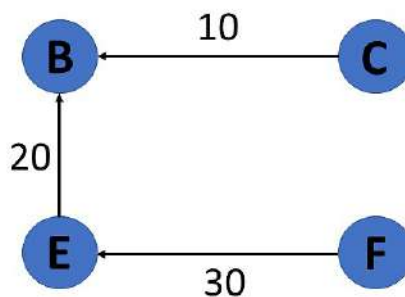
16. Acyclic Graph:

⇒ A graph is **acyclic** if it **contains no cycle**.



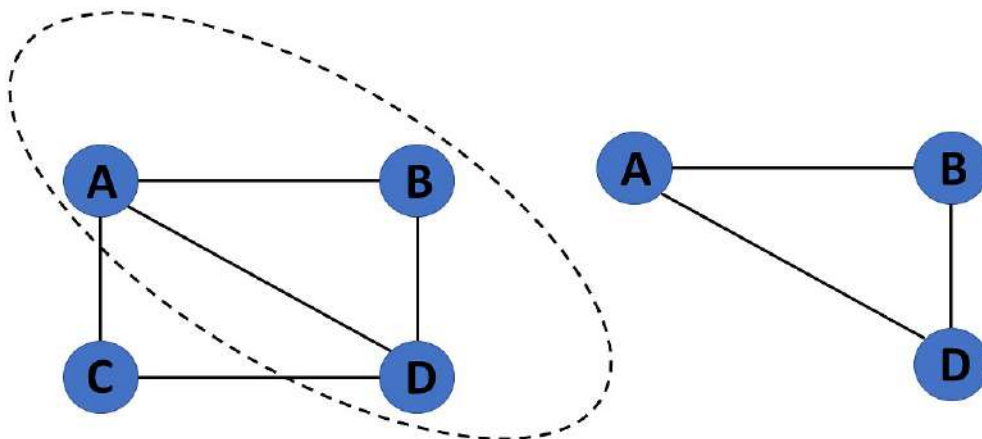
17. Directed Acyclic Graph:

⇒ A **directed acyclic graph (DAG)** is a **graph with directed edges and no cycles**, where the edges represent ordered pairs of vertices and can store data.



18. Sub-Graph:

⇒ A **subgraph** is a graph whose **vertices and edges are subsets of another graph**.



Representation of Graphs

➔ **Graphs** in data structures represent relationships between objects, consisting of **vertices** (or nodes) connected by **edges**. The vertices represent entities in a network.

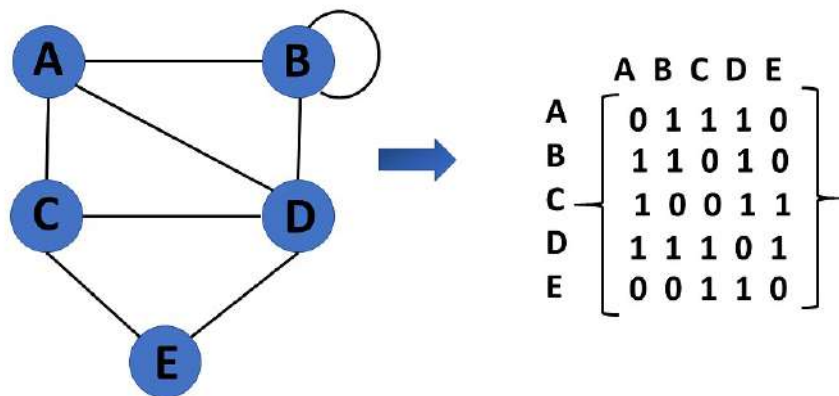
The two most common graph representations are:

- **Adjacency Matrix:** A 2D array used to represent a graph, where each element indicates whether there is an edge between two vertices.
- **Adjacency List:** A list where each vertex has a corresponding list of neighboring vertices, representing edges.

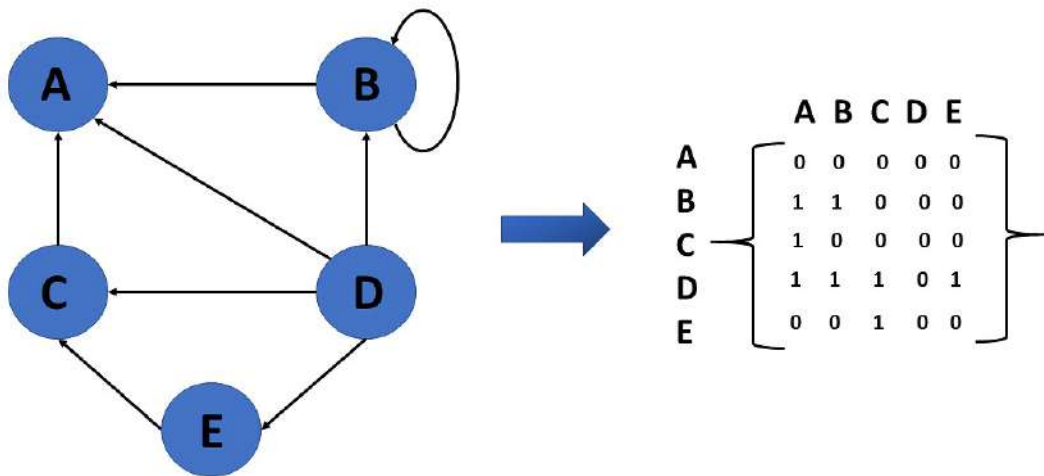
➔ Adjacency Matrix:

- An **adjacency matrix** is a sequential representation used to show which nodes are connected in a graph.
 - An $M \times M$ **matrix G** is created for this representation, where M is the number of vertices.
 - If there is an **edge between vertex a** and **vertex b**, the corresponding element $g_{i,j}$ in **the matrix is set to 1** (or the **weight of the edge in the case of a weighted graph**); otherwise, it is set to 0.

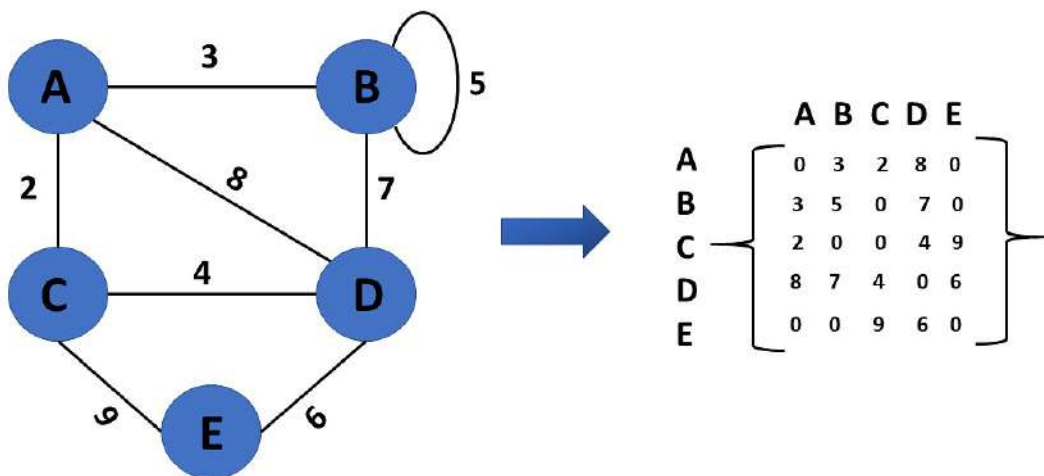
🔗 Undirected Graph Representation:



↻ Directed Graph Representation:



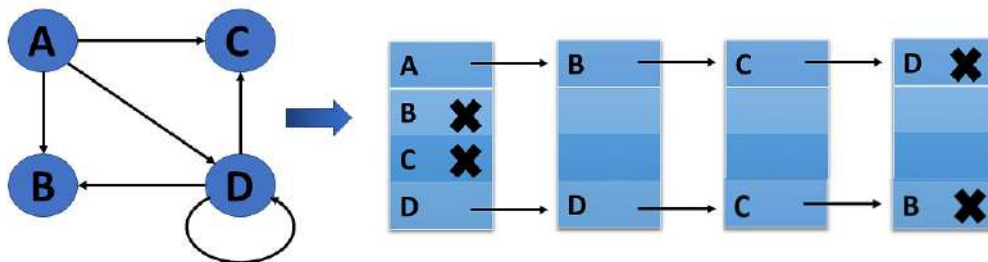
↻ Weighted Undirected Graph Representation:



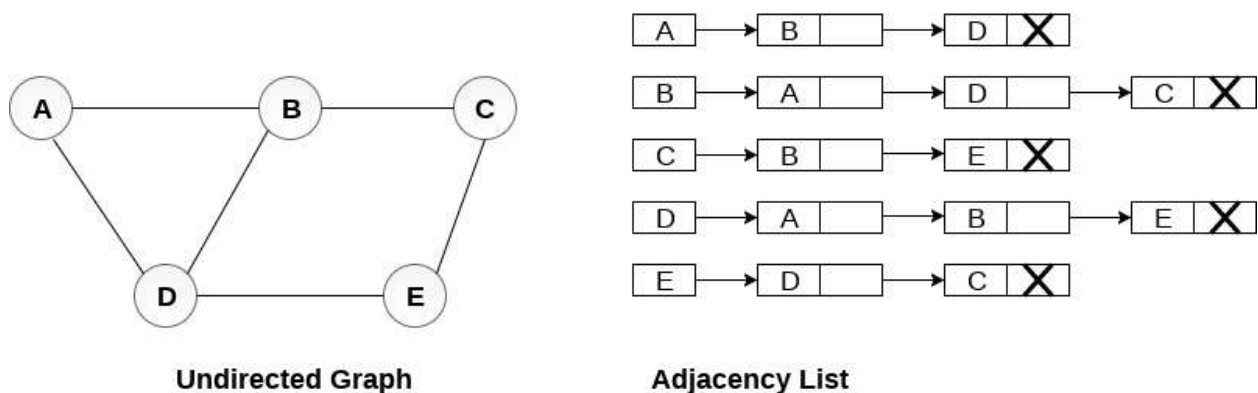
→ Adjacency List:

- An **adjacency list** is a **linked representation** where each vertex in the graph has a list of its **neighboring vertices**.
 - The graph is represented by an array of vertices, indexed by vertex number.
 - Each array member corresponds to a vertex and points to a **singly linked list** of that vertex's neighbors.

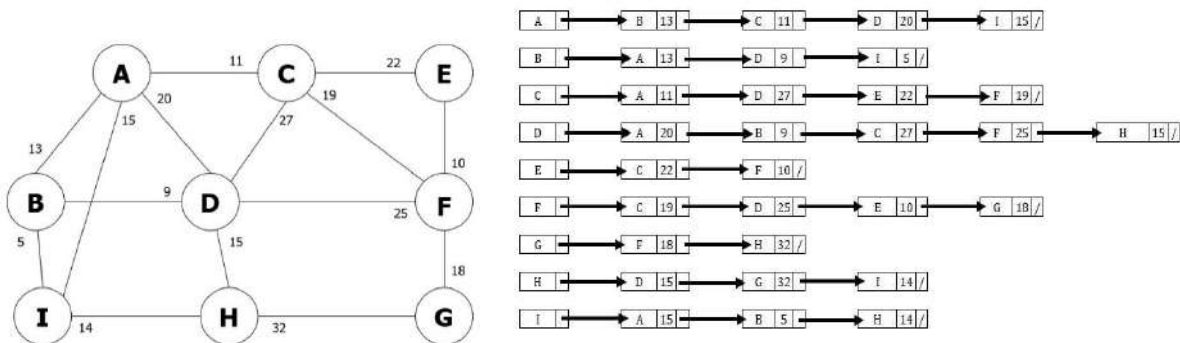
↻ Directed Graph Representation:



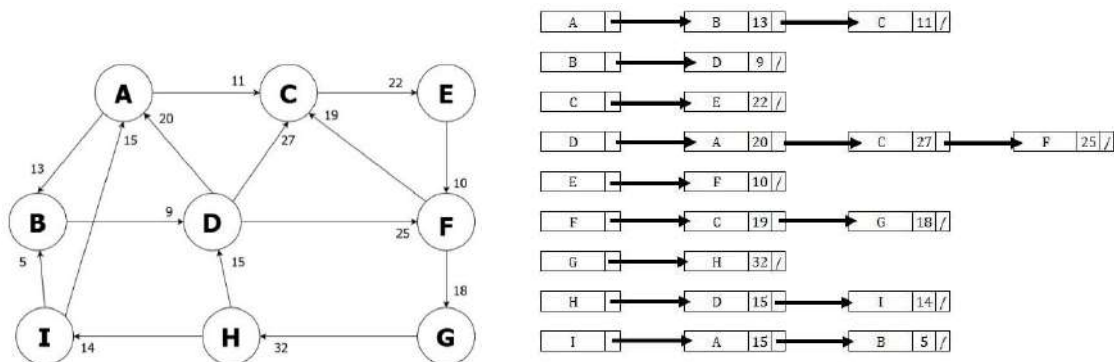
↻ Undirected Graph Representation:



⚡ Weighted UnDirected Graph Representation:



⚡ Weighted Directed Graph Representation:



Graph Traversal Algorithm

⇒ There are two techniques to implement a graph traversal algorithm:

- **Breadth-first search (BFS)**
- **Depth-first search (DFS)**

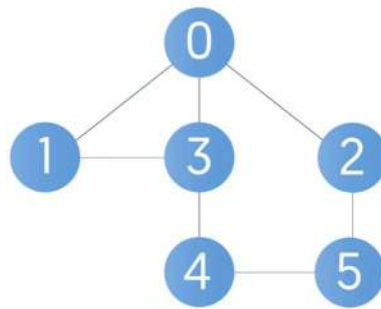
BREADTH-FIRST SEARCH (BFS)

Breadth-first search (BFS)

→ BFS Algorithm:

- **Step 1:** Create an **empty queue** to store the nodes to be visited and a Boolean vector to track the visited nodes. **Initially, no node is visited.**
- **Step 2:** Choose a starting node, **mark it visited**, and **add it to the queue**.
- **Step 3:** **Extract the front item** of the queue, add it to the BFS order, and traverse through the list of that node's adjacent nodes.
- **Step 4:** If a node in that **list has not been already visited**, mark it visited, and **add it to the back of the queue**.
- **Step 5:** **Keep repeating steps 3 and 4 until the queue is empty.**

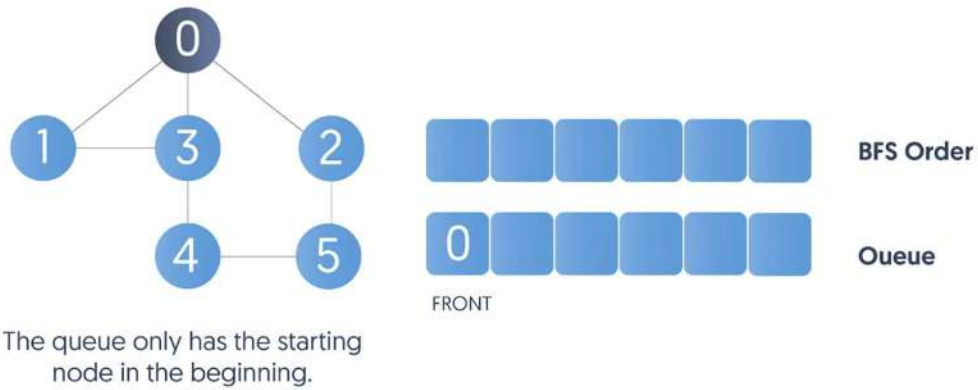
🔗 Example 1:



Graph with six nodes

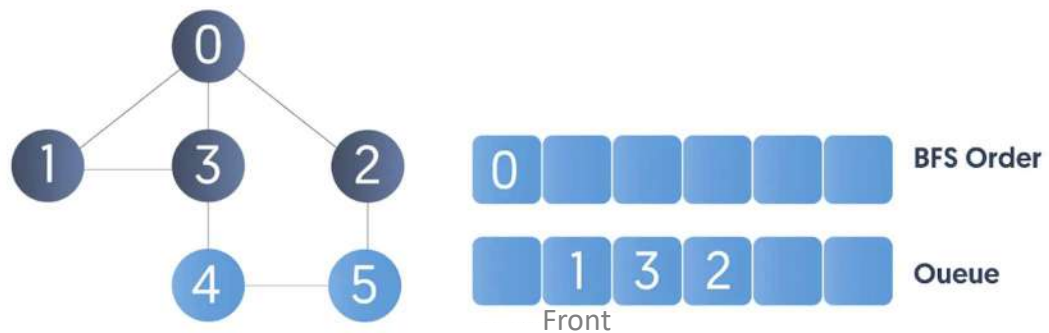
→ We first choose a node to start BFS and put it in the queue after marking it "visited." **In this case, we have chosen 0.**

Step 1:

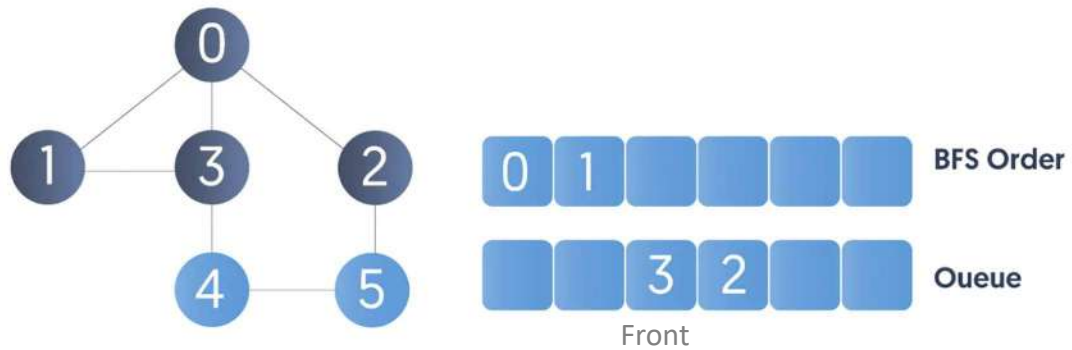


Step 2:

➔ Pop the front element of the queue (0), traverse its unvisited neighbors, mark them as visited, and add them to the queue. Add 0 to the "BFS Order."

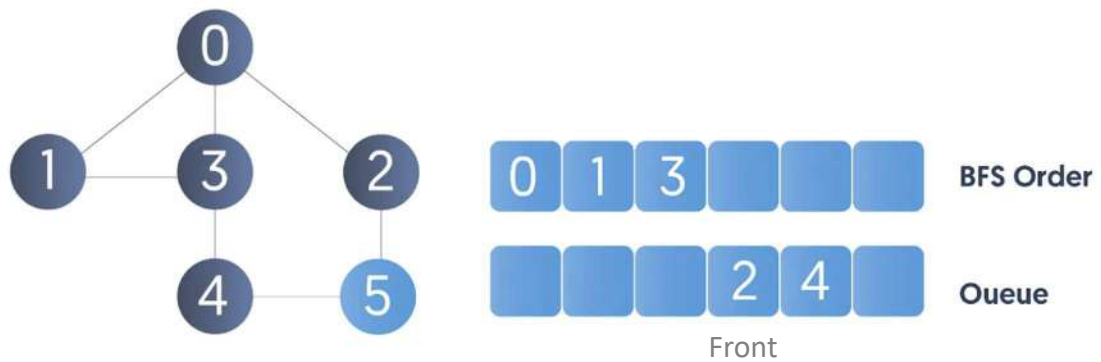


Step 3:



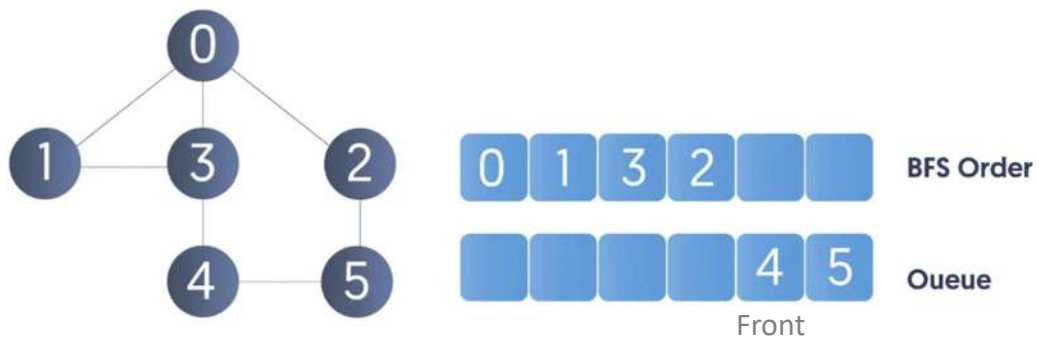
We visit 1 and but it doesn't have any unvisited neighbor

Step 4:



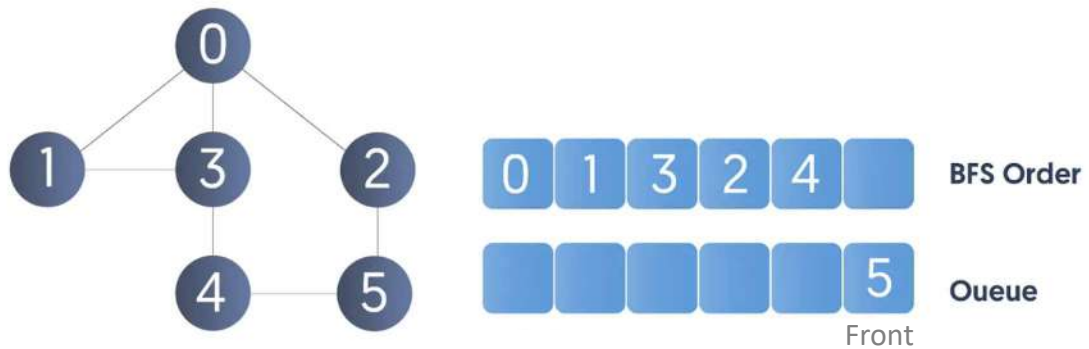
We visit 3 and add its only unvisited neighbour 4 to the queue.

Step 5:



We visit 2 and add its only neighbour 5 to the queue

Step 6:



We visit 4 but it doesn't have any unvisited neighbor.

Step 7:

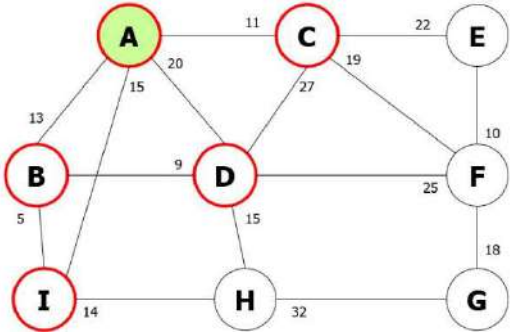
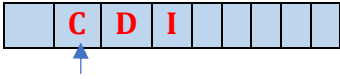
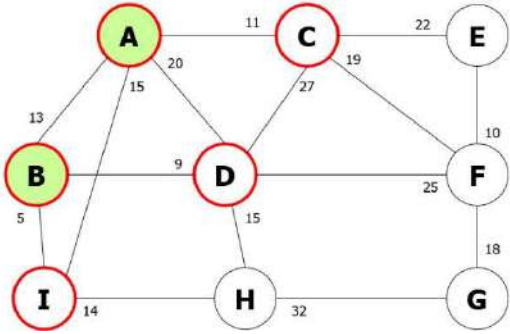
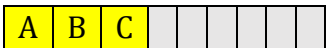
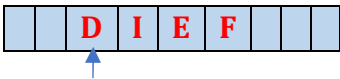
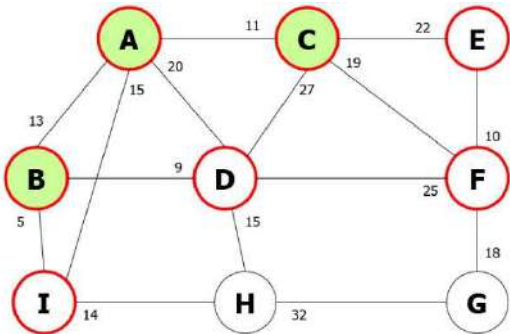
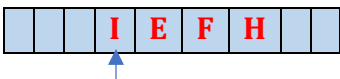
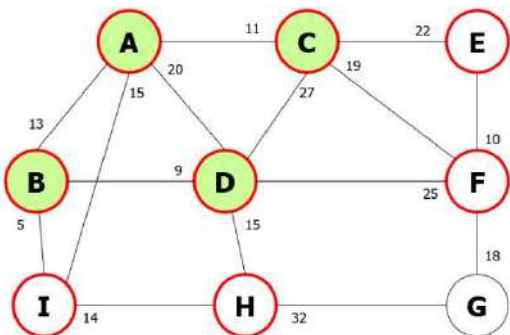


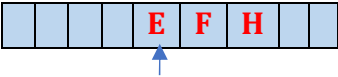
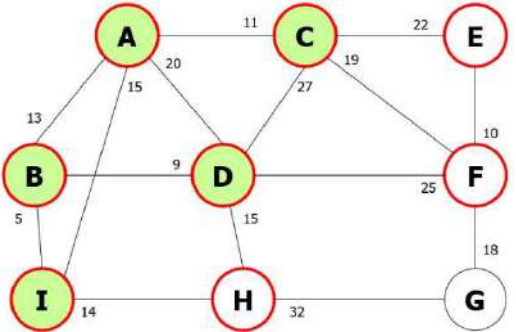
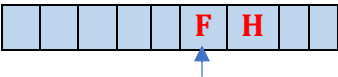
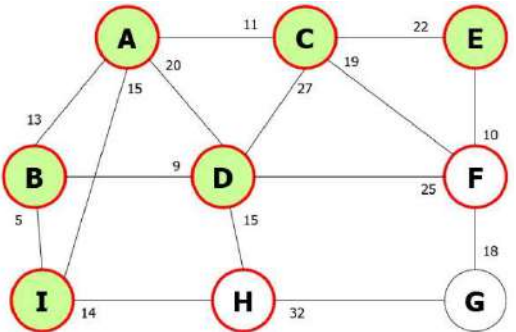
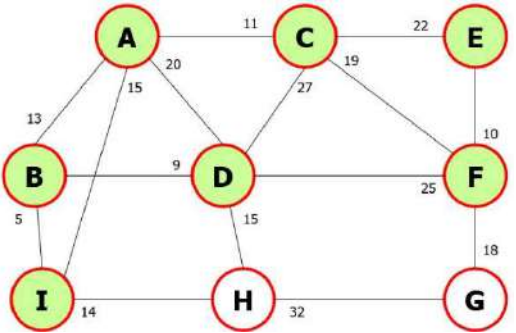
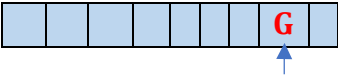
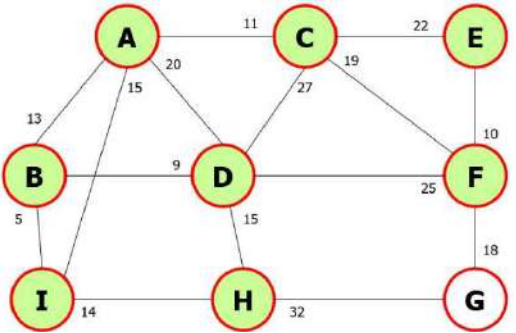
We visit 5 and now all nodes have been visited and the queue is empty.

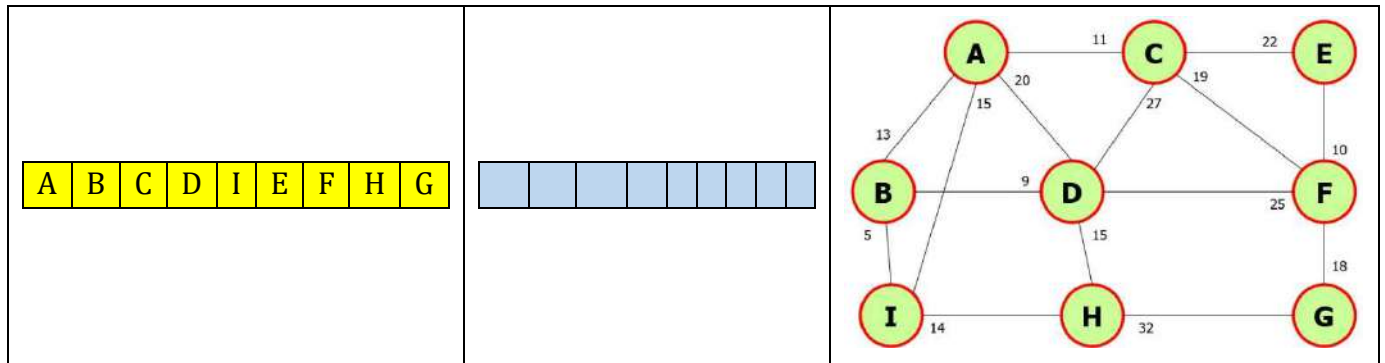
Time Complexity

- Best-case time complexity: $O(V)$, when the graph is very sparse
- Worst-case time complexity: $O(V^2)$, when the graph is very dense.

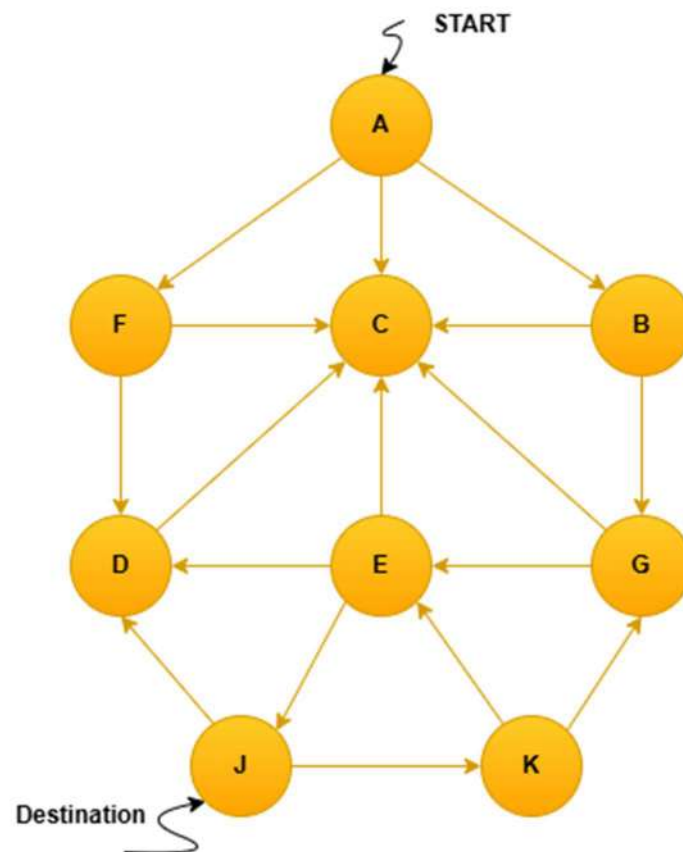
✂ Example 2:

Vertex Visited	Vertex in Queue	Graph
		
		
		
		

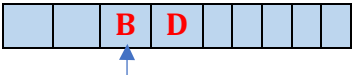


Path Finding Using BFS

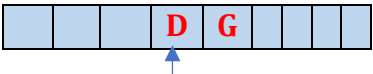
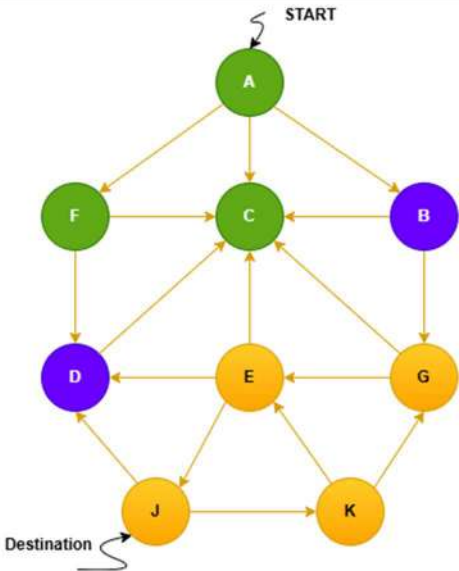


⇒ Find the path Using BFS

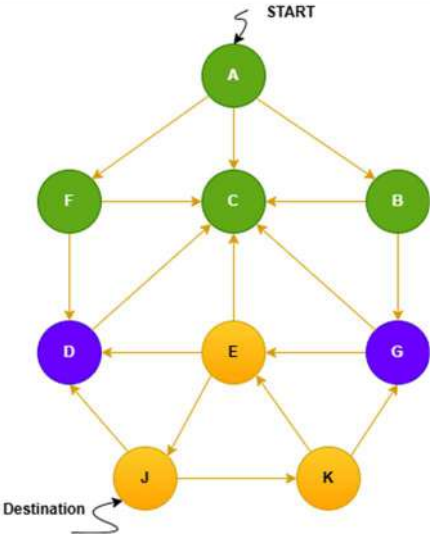
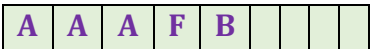
Vertex Visited	Vertex in Queue	Graph
A	F C B ORIGIN: A A A	START A F C B D E G J K Destination J
A F	C B D ORIGIN: A A A F	START A F C B D E G J K Destination J



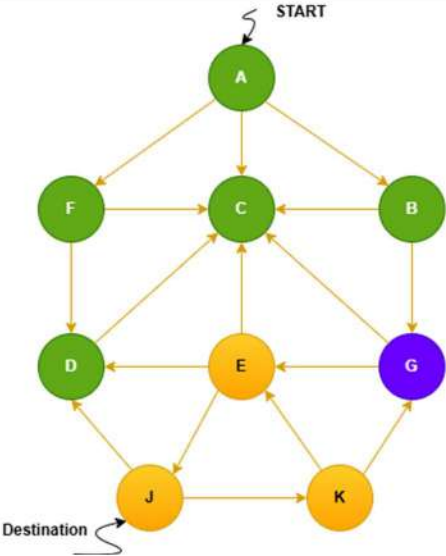
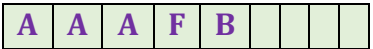
ORIGIN:

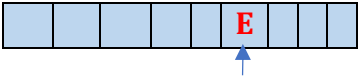


ORIGIN:

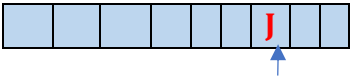
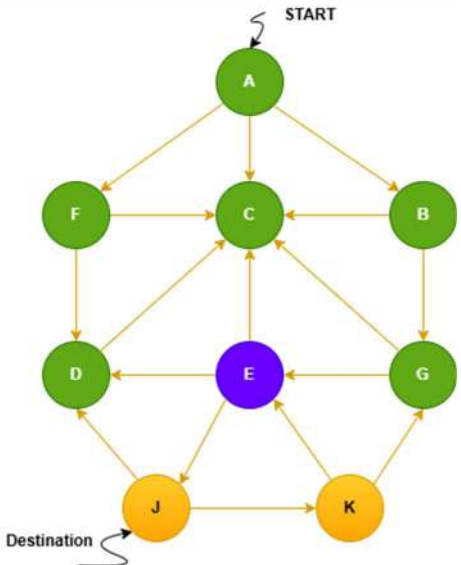
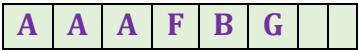


ORIGIN:

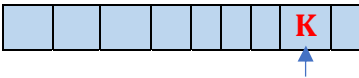
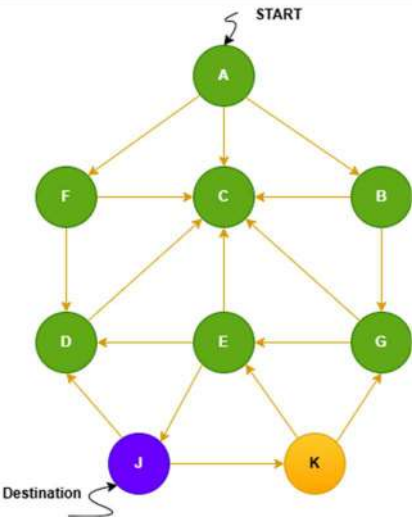




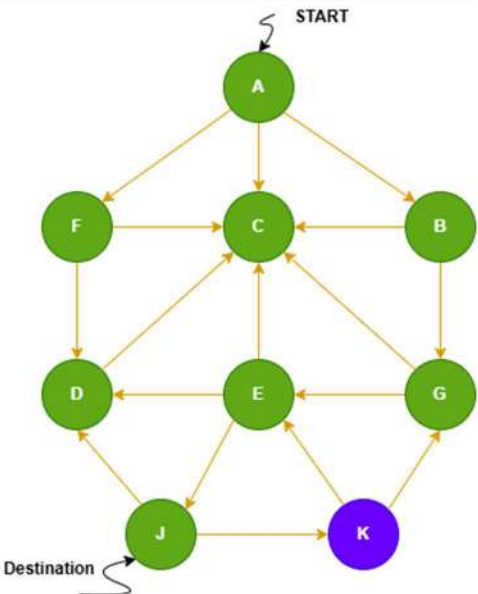
ORIGIN:

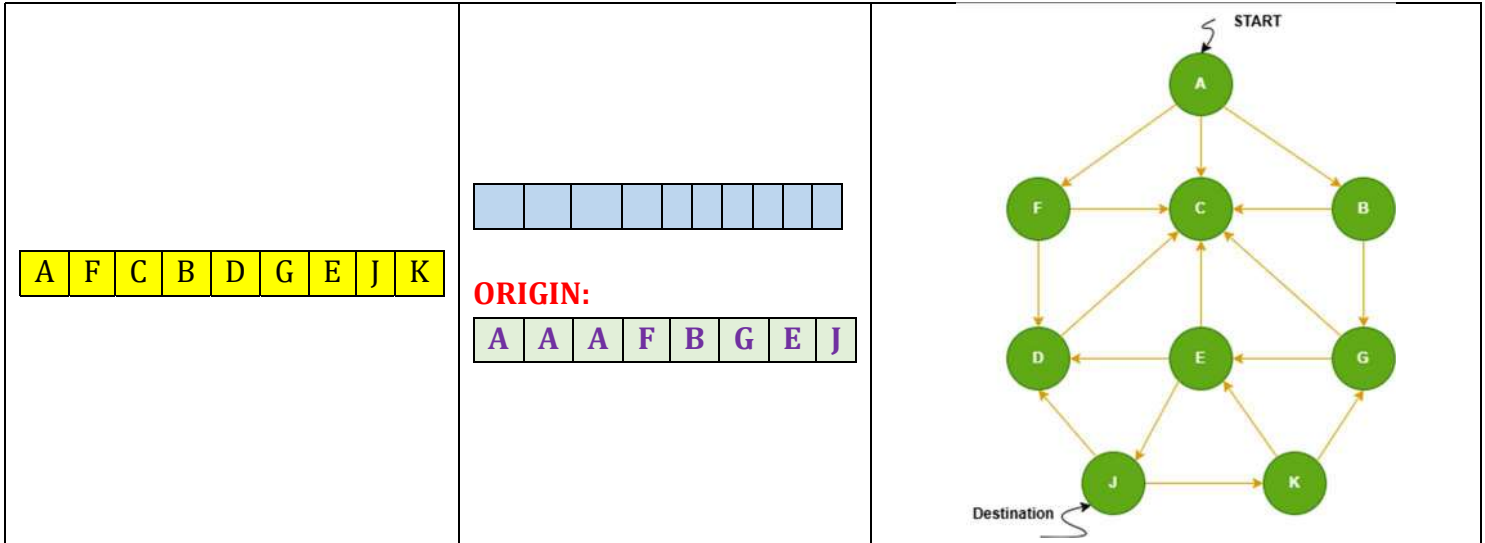


ORIGIN:

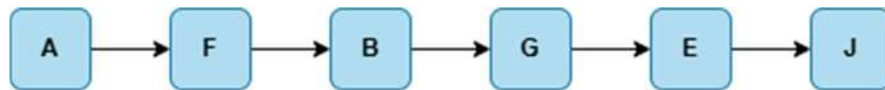


ORIGIN:





⇒ Now the path is:



⇒ Now "**Node F** should be removed from the path as it does not provide a **direct route** to the **destination node J**."

⇒ After removing F, the new path could be:



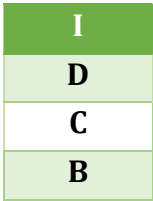
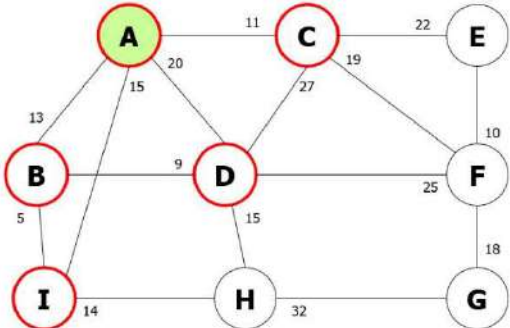
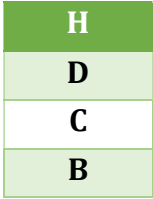
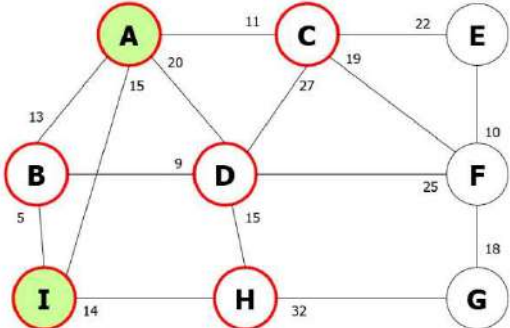
DEPTH FIRST SEARCH (DFS)

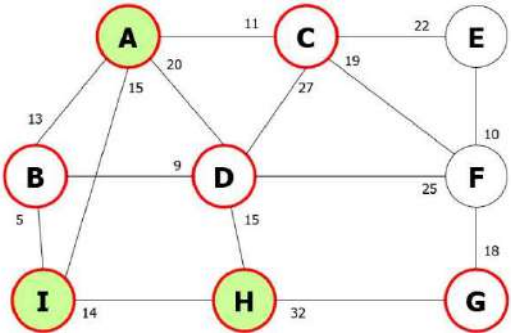
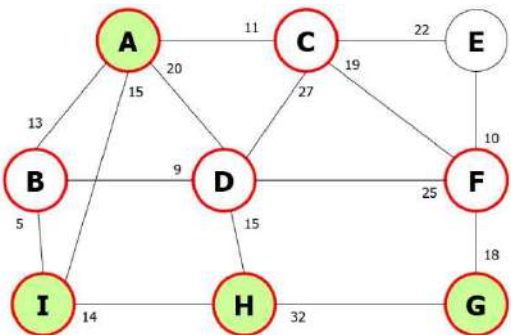
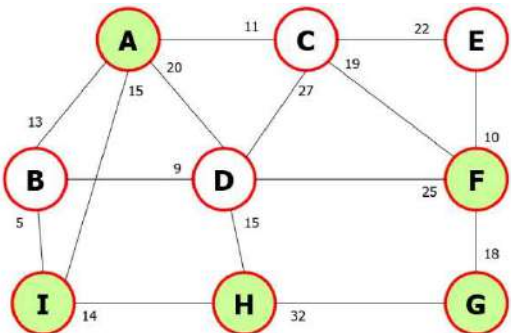
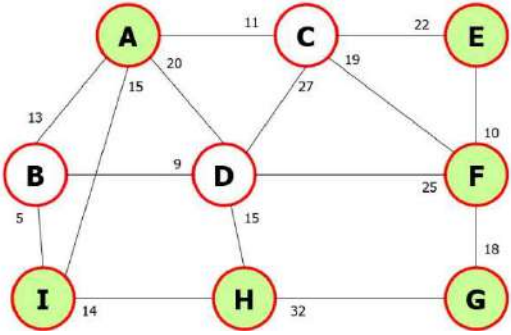
Depth First Search (DFS)

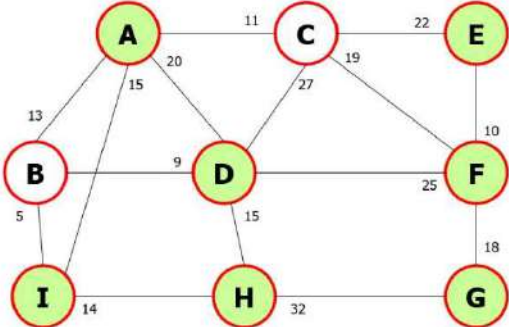
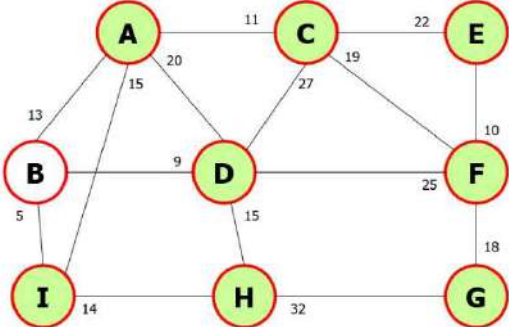
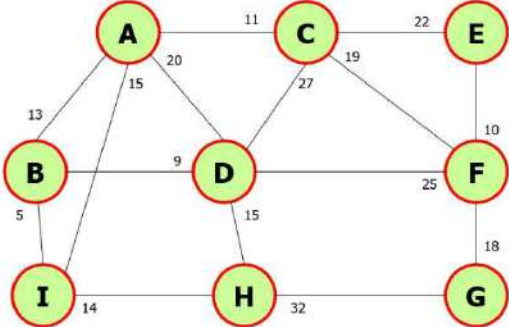
→ DFS Algorithm:

- **Step 1:** Create an empty Boolean array representing the nodes' state (visited or unvisited). **Initially, all nodes are unvisited**, so the entire array is initialized with false.
- **Step 2:** Choose a starting node. Then initiate a recursive function with the starting node as a parameter.
- **Step 3:** Inside the recursive function, mark the node from the **parameter as visited**. After that, loop through the neighbors and call the same recursive function with all unvisited neighbors as parameters.

✂ Example 1:

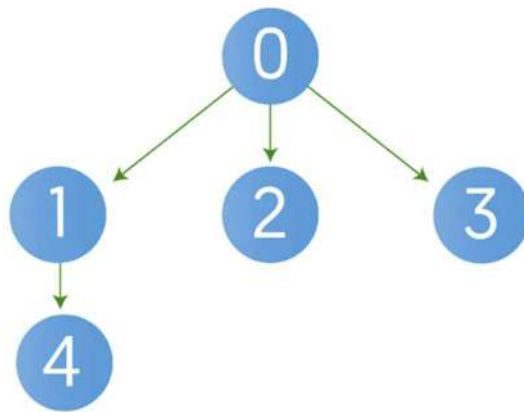
Vertex Visited	Vertex in Stack	Graph
		
		

A I H	G D C B	
A I H G	F D C B	
A I H G F	E D C B	
A I H G F E	D C B	

A I H G F E D	C B	
A I H G F E D C	B	
A I H G F E D C B		

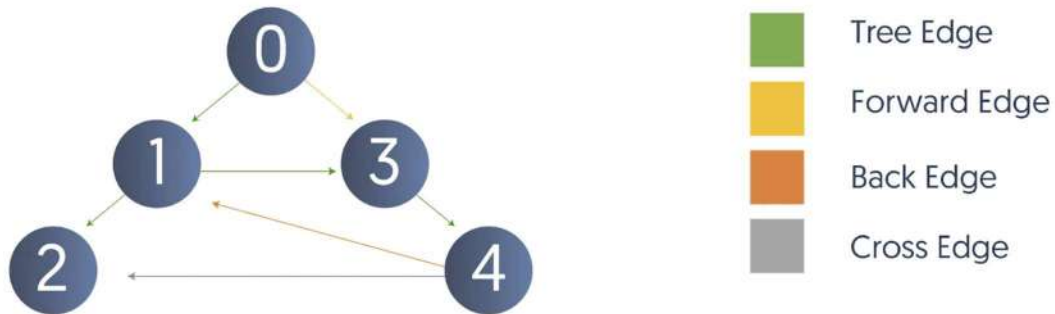
DFS Tree of a Graph

- ⇒ The **DFS tree** is formed by keeping only the edges used in the DFS traversal, with all other edges removed. The green edges represent the DFS path.



Different Types of Edges in DFS

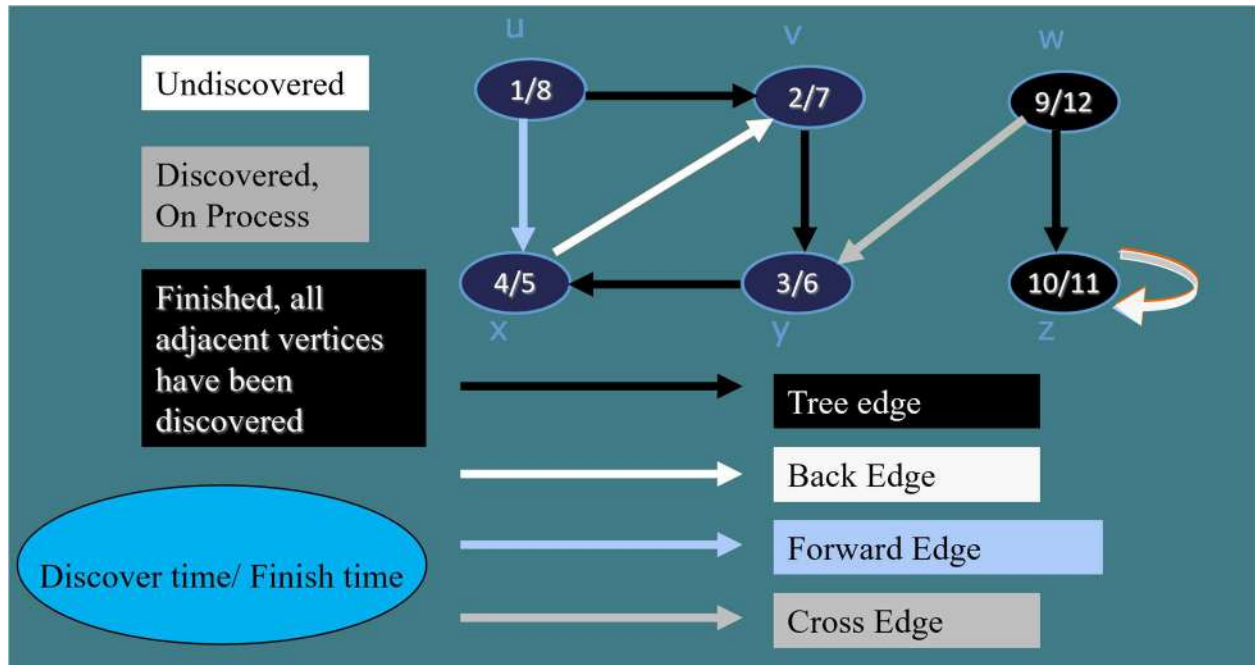
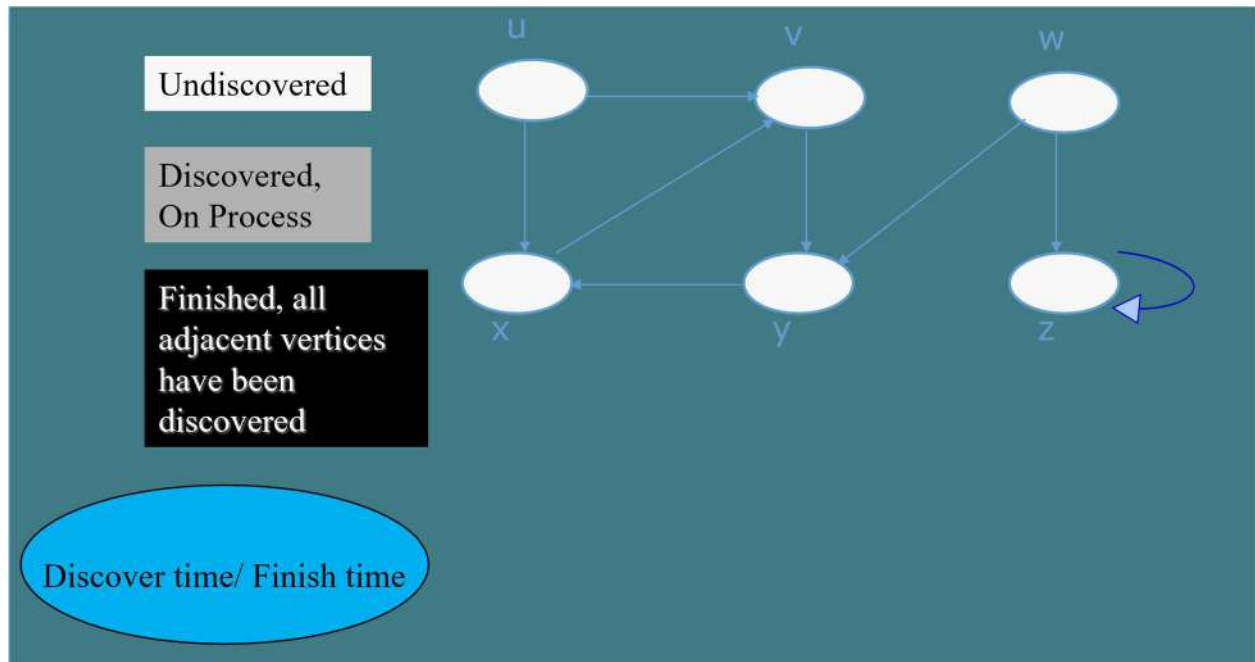
- **Tree edge:** An edge that is present in the DFS tree after applying DFS on the graph.
- **Forward edge:** An edge that is not present in the DFS tree after applying DFS, and it connects a node “u” to one of its successors.
- **Back edge:** An edge that is not present in the DFS tree after applying DFS, and it connects a node “u” to one of its ancestors. Its presence indicates a cycle in directed graphs.
- **Cross edge:** An edge that is not present in the DFS tree after applying DFS, and it connects a node “u” to another node, where there is no parent-child relationship between the two nodes.



Time Complexity

- **DFS Time Complexity:**
 - **Adjacency List:** $O(V + E)$, where V = No of vertices and E = number of edges, as each node and edge is visited once.
 - **Adjacency Matrix:** $O(V^2)$, as each node is visited, and finding neighbors requires checking all other nodes.
 - **Special Case (Tree):** If the graph is a tree (where $E=V-1$), the complexity becomes $O(V)$ with an adjacency list.

Operations of DFS



**STRONGLY
CONNECTED
COMPONENTS**

Strongly Connected Components (SCC)

➔ A **Strongly Connected Component (SCC)** of a directed graph is a maximal subgraph in which every vertex is reachable from every other vertex in that subgraph. In other words, **for any two vertices u & v in an SCC, there exists a directed path from u to v and from v to u .**

Key Points:

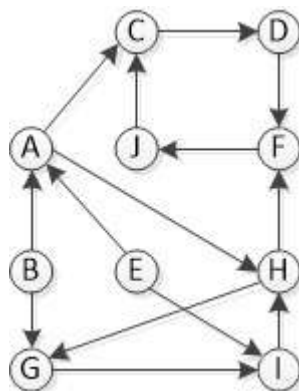
- **Uses two DFS traversals:** One on the original graph and one on the transposed graph.
- **First DFS:** Determines the finishing times of vertices.
- **Graph Transposition:** Reverse all edges of the graph.
- **Second DFS:** Processes vertices in decreasing order of finishing times from the first DFS.
- **Each DFS tree in the transposed graph** represents a **Strongly Connected Component (SCC)**.
- **Time Complexity:** $O(V + E)$.

Algorithm Steps:

- Perform DFS on the original graph and record finishing times for each vertex.
- Compute the transpose of the graph (reverse all edges).
- Perform DFS on the transposed graph, processing vertices in decreasing order of finishing times.
- Each DFS tree in this step forms a strongly connected component (SCC).

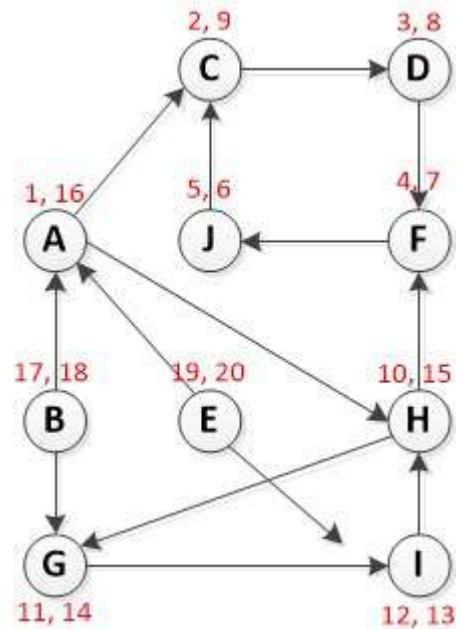
✂ **Example 1:**

✂ >> From the following graph, find out the strongly connected components and classify different ages.

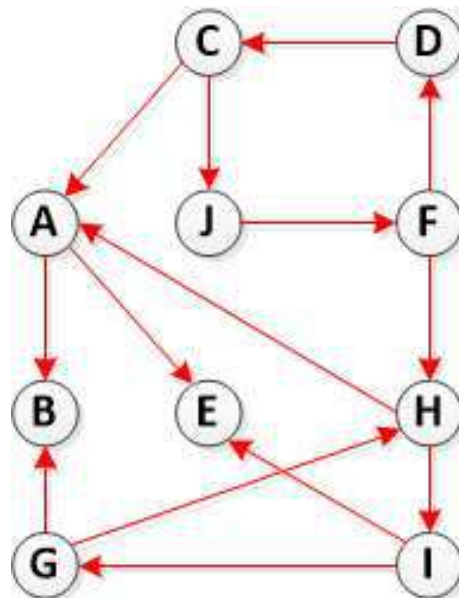


Step 1: Call DFS(G) to compute finishing times $f[u]$ for each vertex u

>>Running DFS starting on vertex A:



Step 2: Compute Transpose(G)



Step 3: Call DFS(Transpose(G)), but in the main loop of DFS, consider the vertices in order of decreasing $f[u]$ (as computed in step 1)

→ So vertices in order of **decreasing post-visit (finishing times)** values:

{E, B, A, H, G, I, C, D, F, J}

- Run DFS on G^T , marking SCCs:
 - DFS(E): {E}
 - DFS(B): {B}
 - DFS(A): {A}
 - DFS(H): {H, I, G}
 - DFS(G): remove from list since it is already visited
 - DFS(I): remove from list since it is already visited
 - DFS(C): {C, J, F, D}
 - DFS(J): remove from list since it is already visited
 - DFS(F): remove from list since it is already visited
 - DFS(D): remove from list since it is already visited

Step 4: Output the vertices of each tree in the depth-first forest of step 3 as a separate strong connected component.

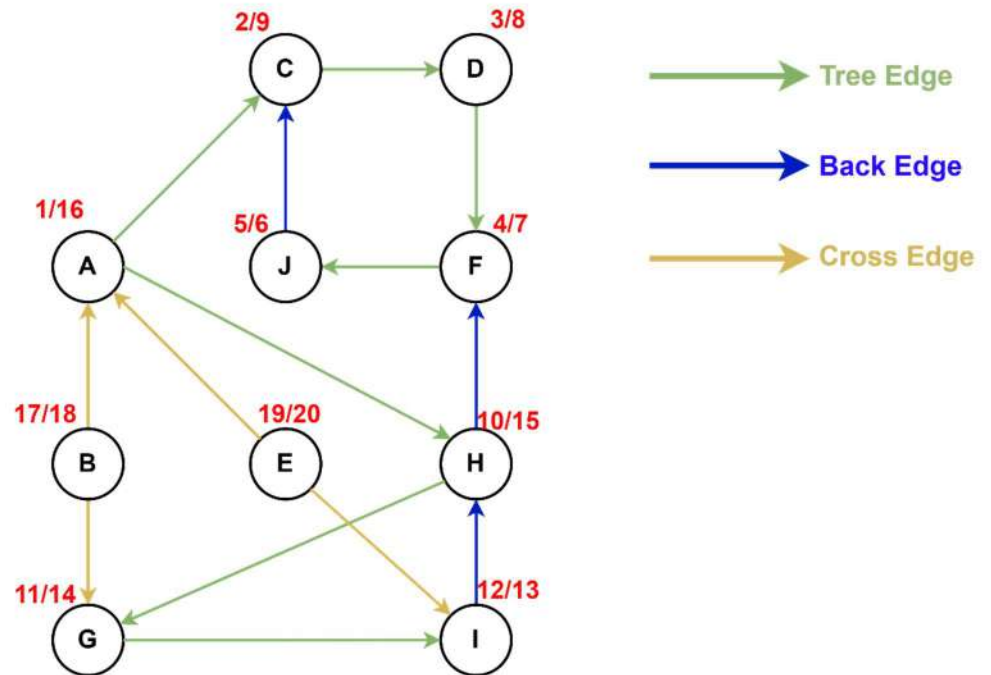
→ So we have five strongly connected components:

{E}, {B}, {A}, {H, I, G}, {C, J, F, D}

→ **Classification of Edges:**

→ **Some Rules:**

- If the numerator of the starting vertex > the numerator of the ending vertex → Back Edge
- If the numerator of the starting vertex < the numerator of the ending vertex → Forward Edge



Classification of Edges:

- Tree Edges** (discovery edges in DFS)
 - A → C (A discovers C)
 - C → D (C discovers D)
 - D → F (D discovers F)
 - F → J (F discovers J)
 - A → H (A discovers H)
 - H → G (H discovers G)
 - G → I (G discovers I)
- Back Edges** (pointing back to an ancestor in DFS)
 - J → C (J points back to C, which is an ancestor → **Back Edge**)
 - I → H (I points back to H, which is an ancestor → **Back Edge**)
 - H → F (H points back to F, which is an ancestor → **Back Edge**)
- Forward Edges** (pointing to an already discovered descendant)
 - None in this case, since all forward edges are also tree edges in this DFS.
- Cross Edges** (connecting different DFS trees)
 - B → A (B starts a separate DFS tree and connects to A → **Cross Edge**)
 - B → G (B is in a different DFS tree and connects to G → **Cross Edge**)
 - E → A (E is in a separate DFS tree, connecting to A → **Cross Edge**)
 - E → I (E is in a different DFS tree, connecting to I → **Cross Edge**)

➔Final Classification:

Edge	Type
$A \rightarrow C$	Tree Edge
$C \rightarrow D$	Tree Edge
$D \rightarrow F$	Tree Edge
$F \rightarrow J$	Tree Edge
$J \rightarrow C$	Back Edge
$A \rightarrow H$	Tree Edge
$H \rightarrow G$	Tree Edge
$G \rightarrow I$	Tree Edge
$I \rightarrow H$	Back Edge
$B \rightarrow A$	Cross Edge
$B \rightarrow G$	Cross Edge
$E \rightarrow A$	Cross Edge
$E \rightarrow I$	Cross Edge
$H \rightarrow F$	Back Edge

TOPOLOGICAL SORT

Topological Sort

➔ **Topological Sorting** is a linear ordering of vertices in a **directed acyclic graph (DAG)** such that for every directed **edge $U \rightarrow V$** , **vertex "U"** appears before **vertex "V"** in the ordering.

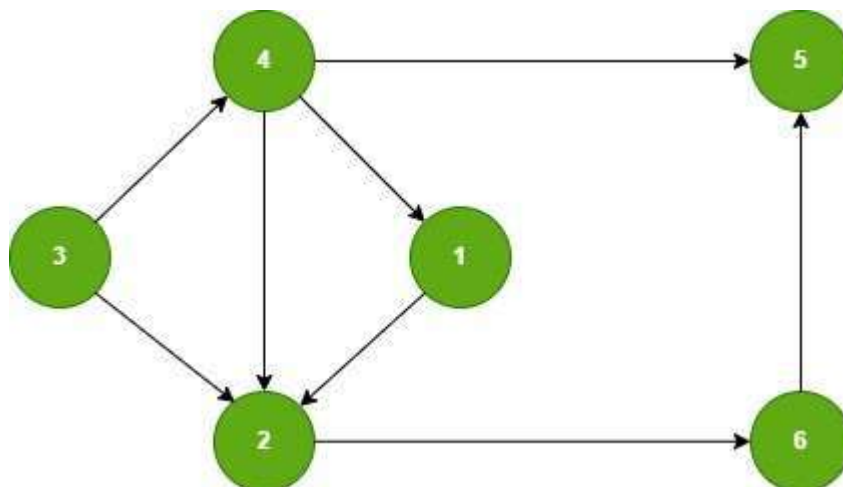
Key Points:

- Possible only for DAGs (Directed Acyclic Graphs).
- Orders tasks based on dependencies.
- Multiple valid orders may exist.

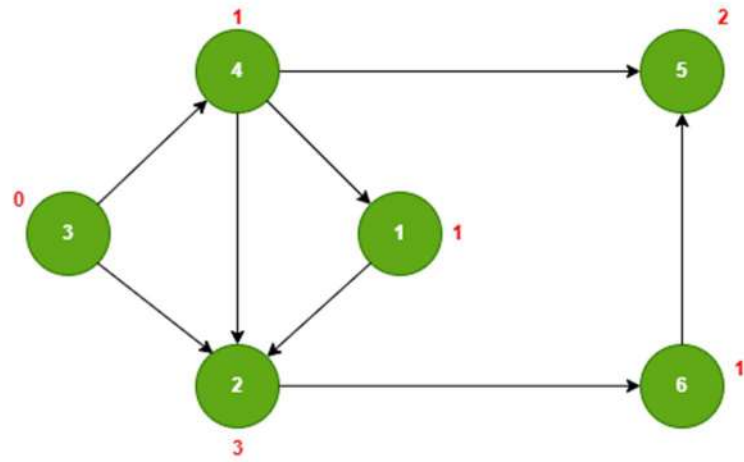
Algorithm Steps:

1. **Calculate In-Degree:** Count incoming edges for each vertex.
2. **Queue Initialization:** Add vertices with in-degree = 0 to a queue.
3. **Process Queue:**
 - Remove a vertex, add it to the result.
 - Decrease in-degree of its neighbors.
 - If any neighbor's in-degree becomes 0, add it to the queue.
4. **Check for Cycles:** If all vertices aren't processed, the graph contains a cycle.

✂ Example 1:

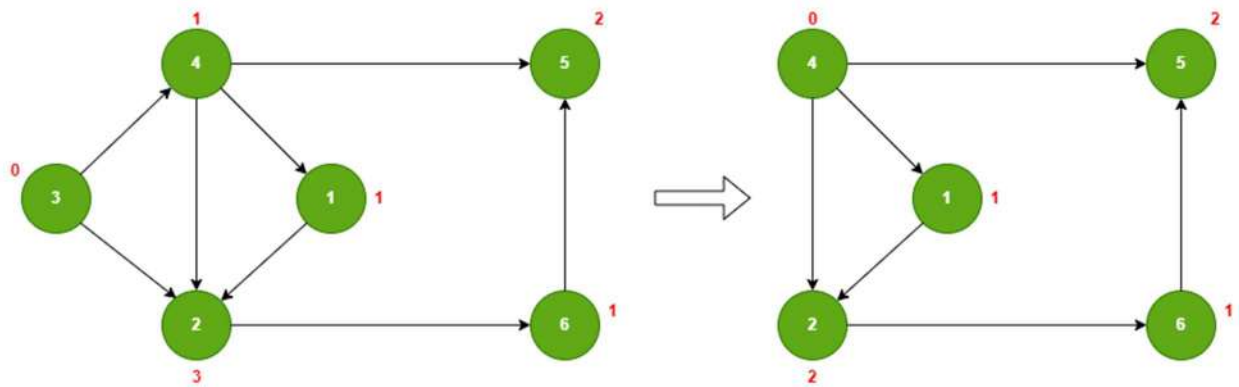


Step 1:



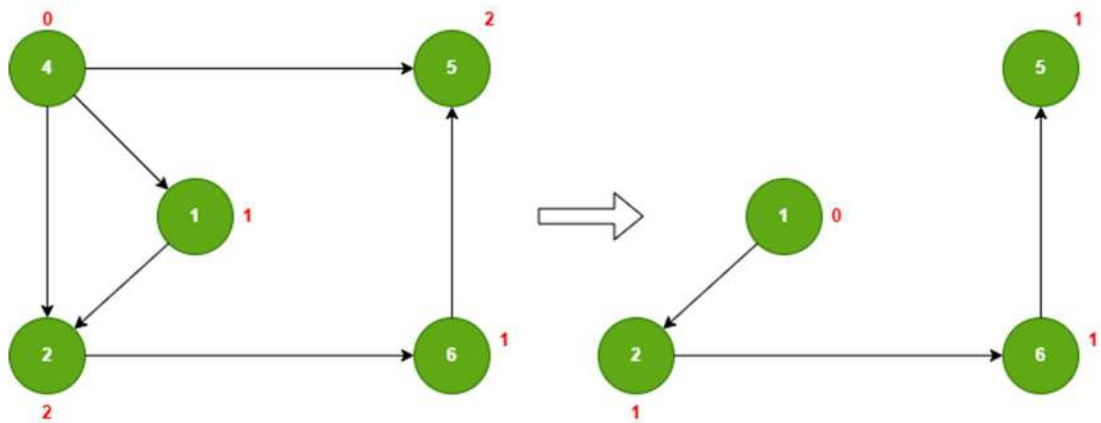
Result : 3

Step 2:



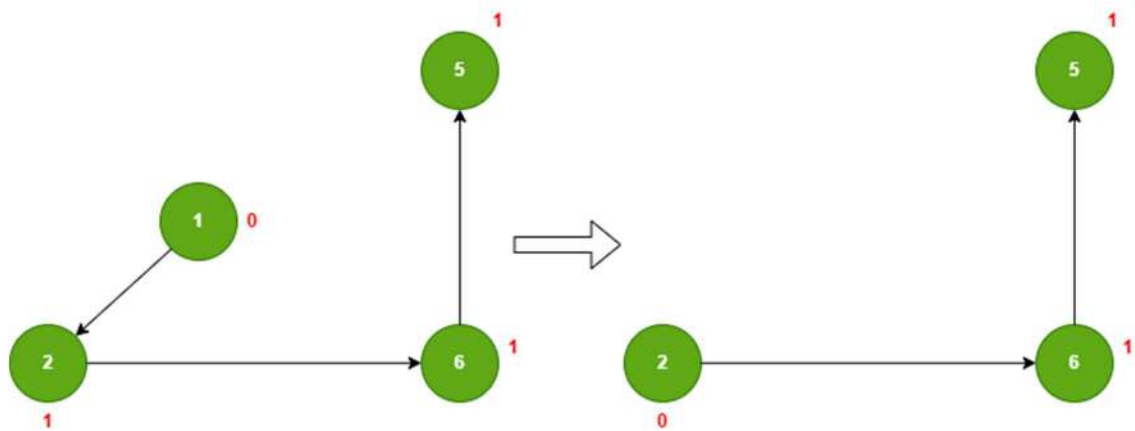
Result: 3 4

Step 3:



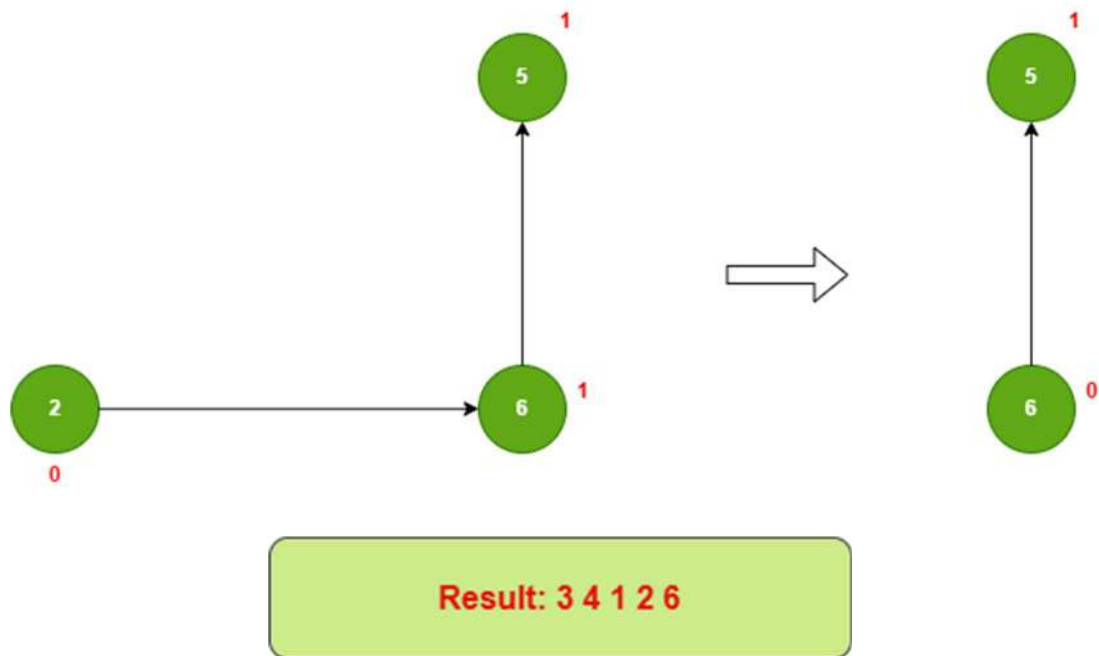
Result: 3 4 1

Step 4:

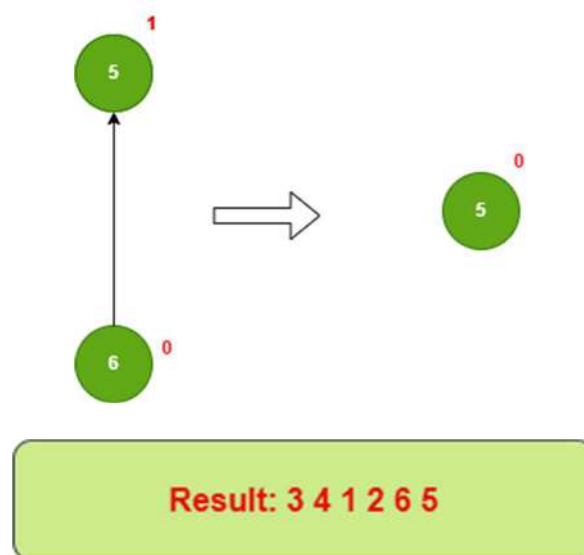


Result: 3 4 1 2

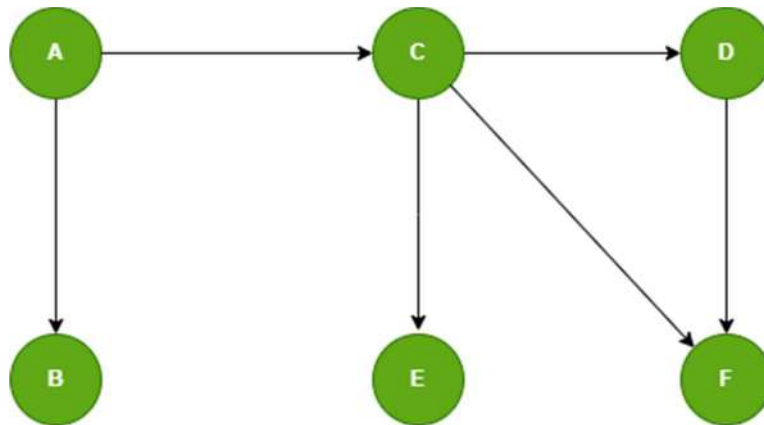
Step 5:



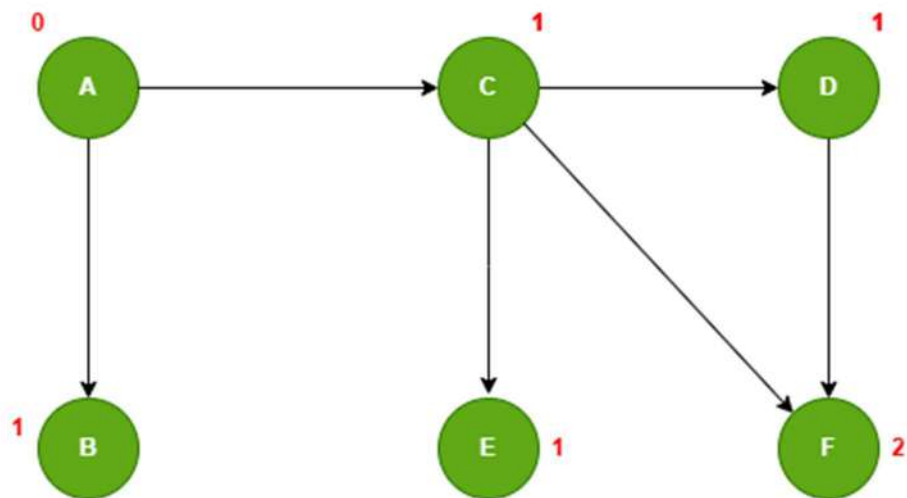
Step 6:



☞ Example 2:

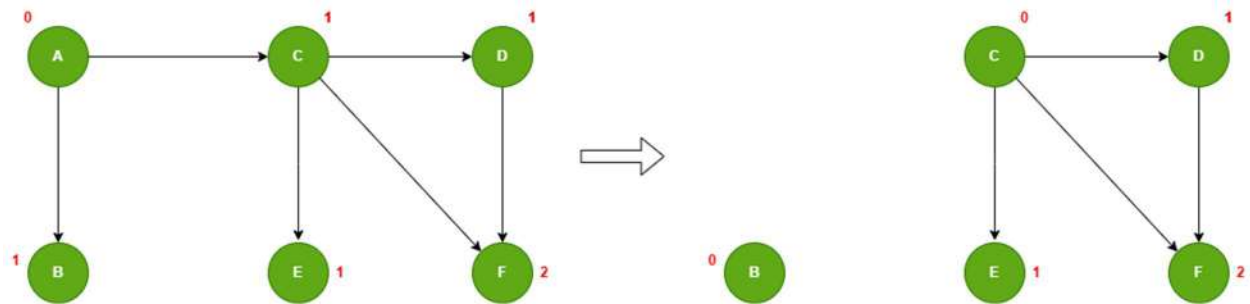


Step 1:



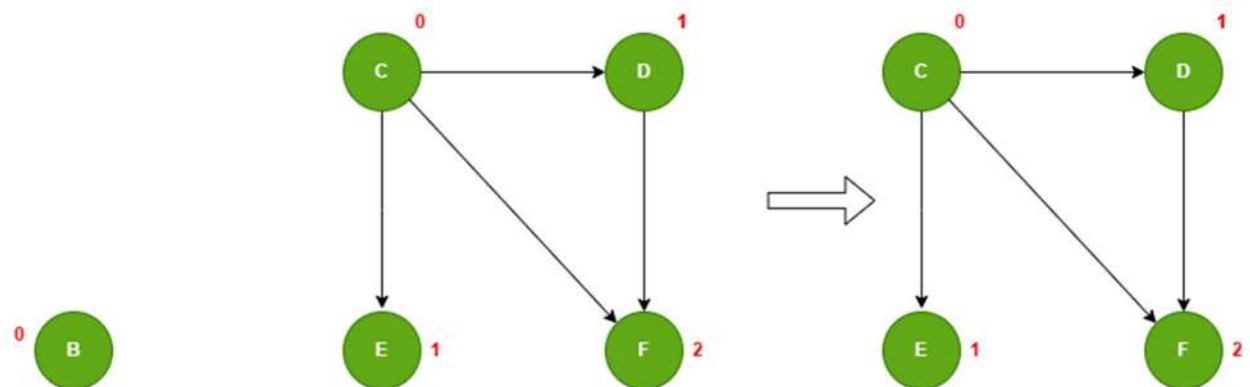
Result: A

Step 2:



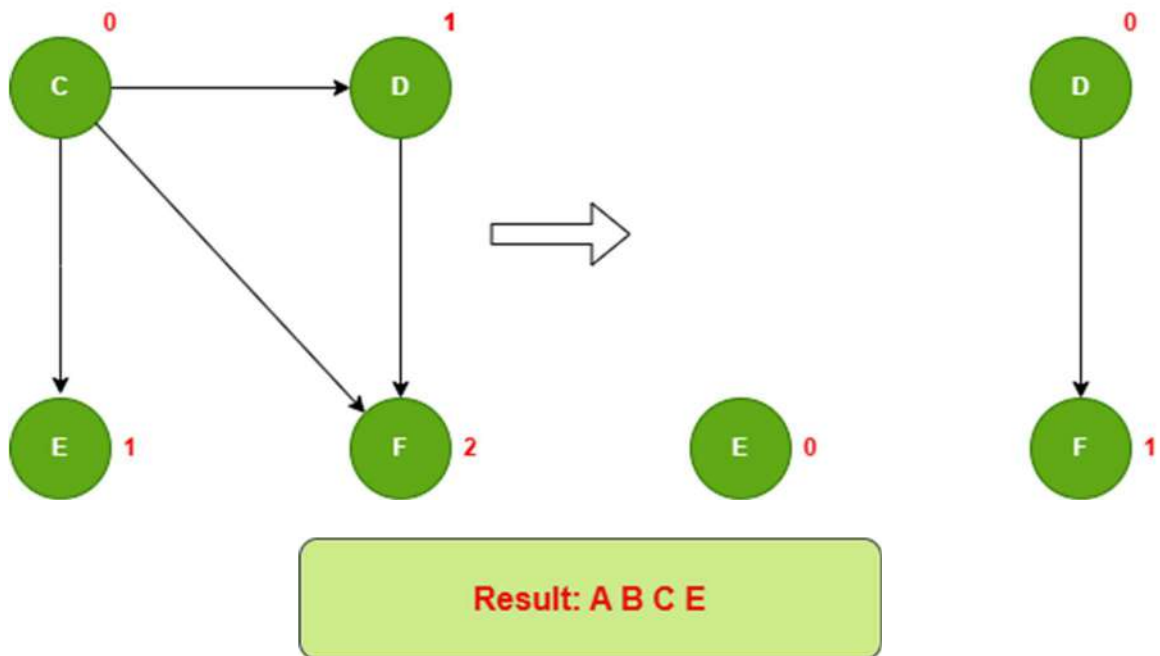
Result: A B

Step 3:

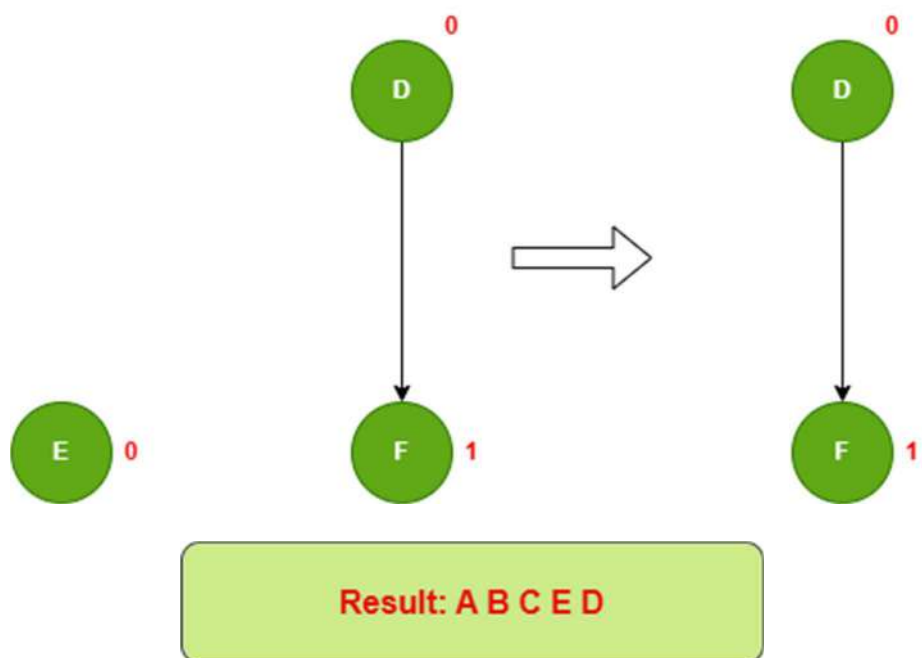


Result: A B C

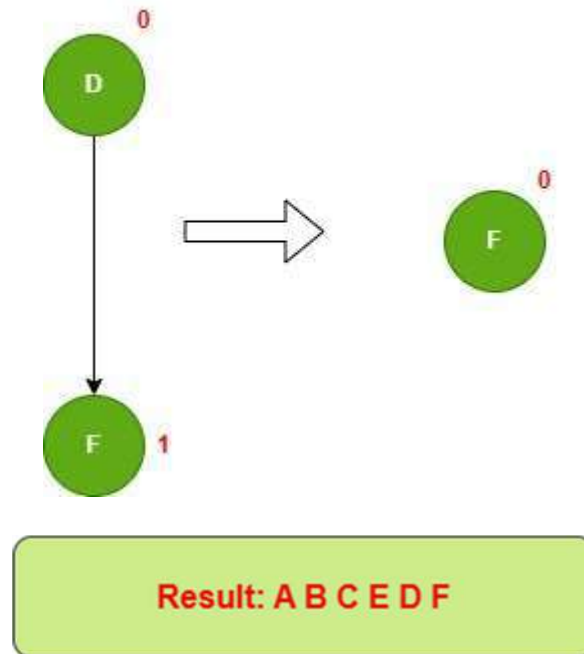
Step 4:



Step 5:



Step 6:



Question:

⇒ A **university** offers **six courses**:

- 1 C1 (Introduction to Programming)
- 2 C2 (Data Structures)
- 3 C3 (Algorithms)
- 4 C4 (Databases)
- 5 C5 (Operating Systems)
- 6 C6 (Computer Networks)

The courses have **prerequisites** as follows:

- C1 → C2 (C1 must be completed before C2)
- C2 → C3
- C3 → C5
- C4 → C5
- C5 → C6

◆ **Your Task is:**

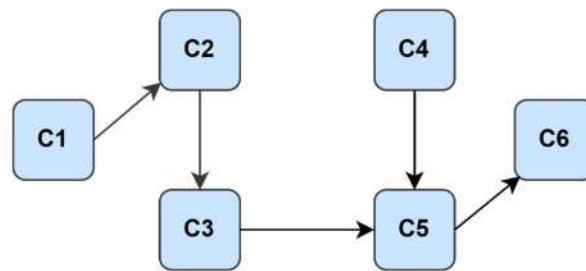
1. Represent this dependency information as a **Directed Acyclic Graph (DAG)**.
2. Find a **valid order** to complete all courses.
3. Implement the solution using **Kahn's Algorithm (BFS-based approach)**.

 Solution:

1 Represent as a DAG:

➔ *Each course is a vertex, and each prerequisite is a directed edge pointing to the dependent course.*

Graph Representation:



2 Applying Topological Sorting using Kahn's Algorithm (BFS)

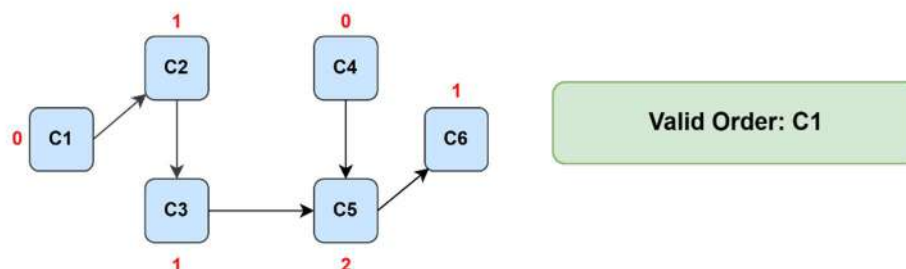
Kahn's Algorithm works by:

1. Calculating the **in-degree** (number of incoming edges) of each node.
2. Adding nodes with **zero in-degree** to a queue.
3. Removing nodes from the queue and decreasing the in-degree of their neighbors.
4. Repeating until all nodes are processed.

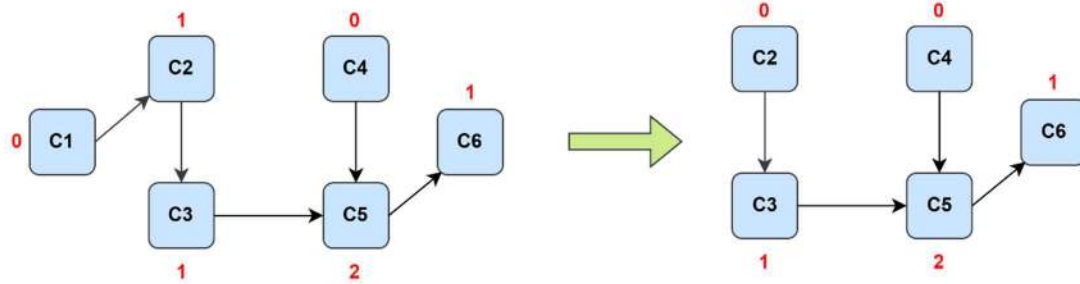
3 Simulations:

Now, let's implement **Kahn's Algorithm (BFS-based Topological Sorting)**.

Step1:

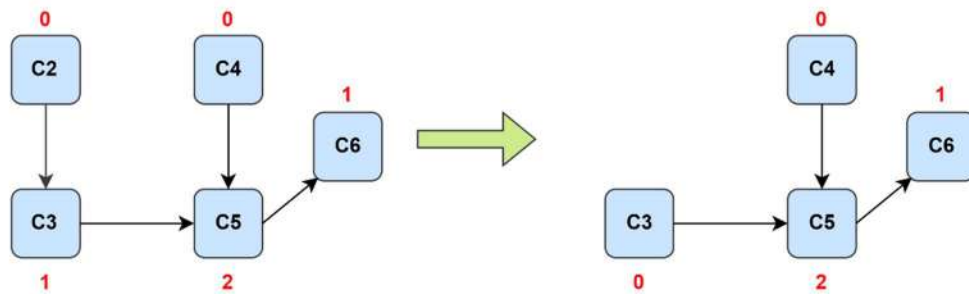


Step2:



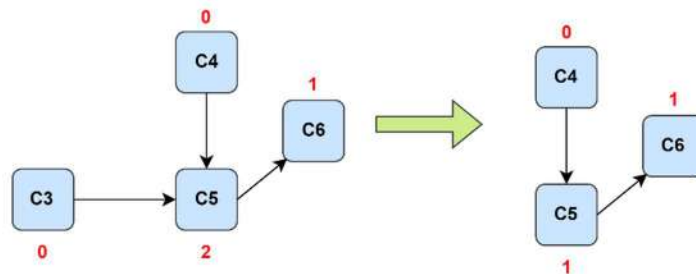
Valid Order: C1 C2

Step3:



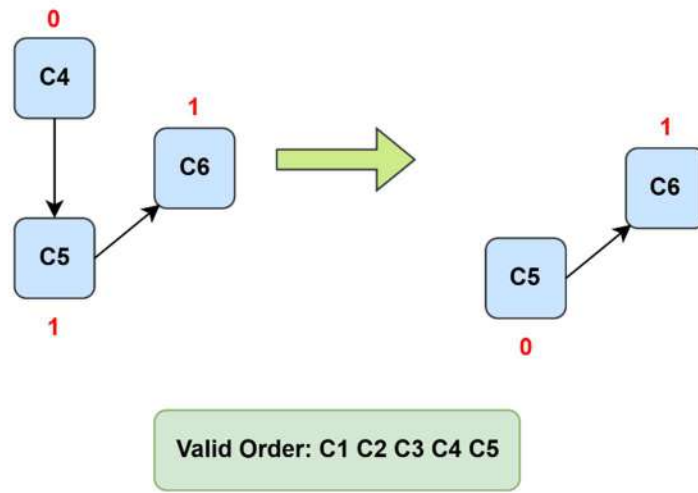
Valid Order: C1 C2 C3

Step4:

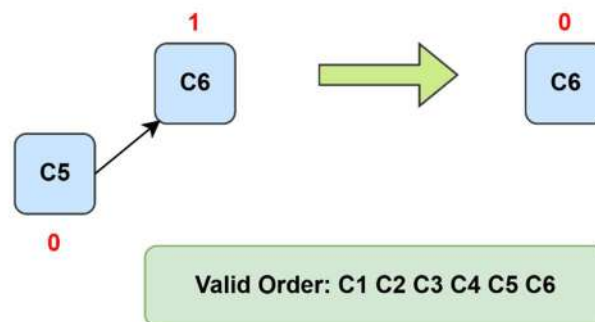


Valid Order: C1 C2 C3 C4

Step5:



Step6:



☑ So, the order **C1, C2, C3, C4, C5, C6** is a valid topological order, respecting all course prerequisites. & The order **C1, C4, C2, C3, C5, C6** is also a valid alternative topological order. It meets all the prerequisites while maintaining the correct sequence of courses.

PRIM'S ALGORITHM

Prim's Algorithm

Conditions for Prim's Algorithm:

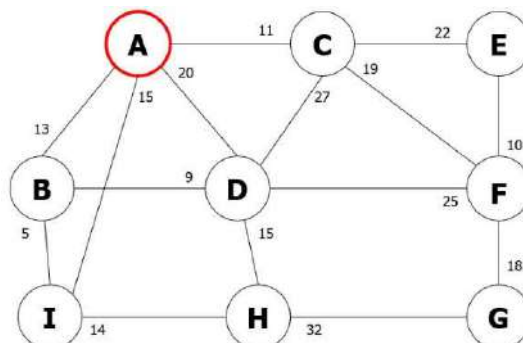
1. **Connected Graph:** Must be connected.
2. **Undirected Graph:** Only works on undirected graphs.
3. **Weighted Graph:** Edges must have weights.
4. **Non-negative Weights:** Edges should have non-negative weights.
5. **Start from Any Vertex:** Can start from any vertex in the graph.
6. **Greedy Approach:** Selects the minimum weight edge to expand the MST.

Prim's Algorithm Steps:

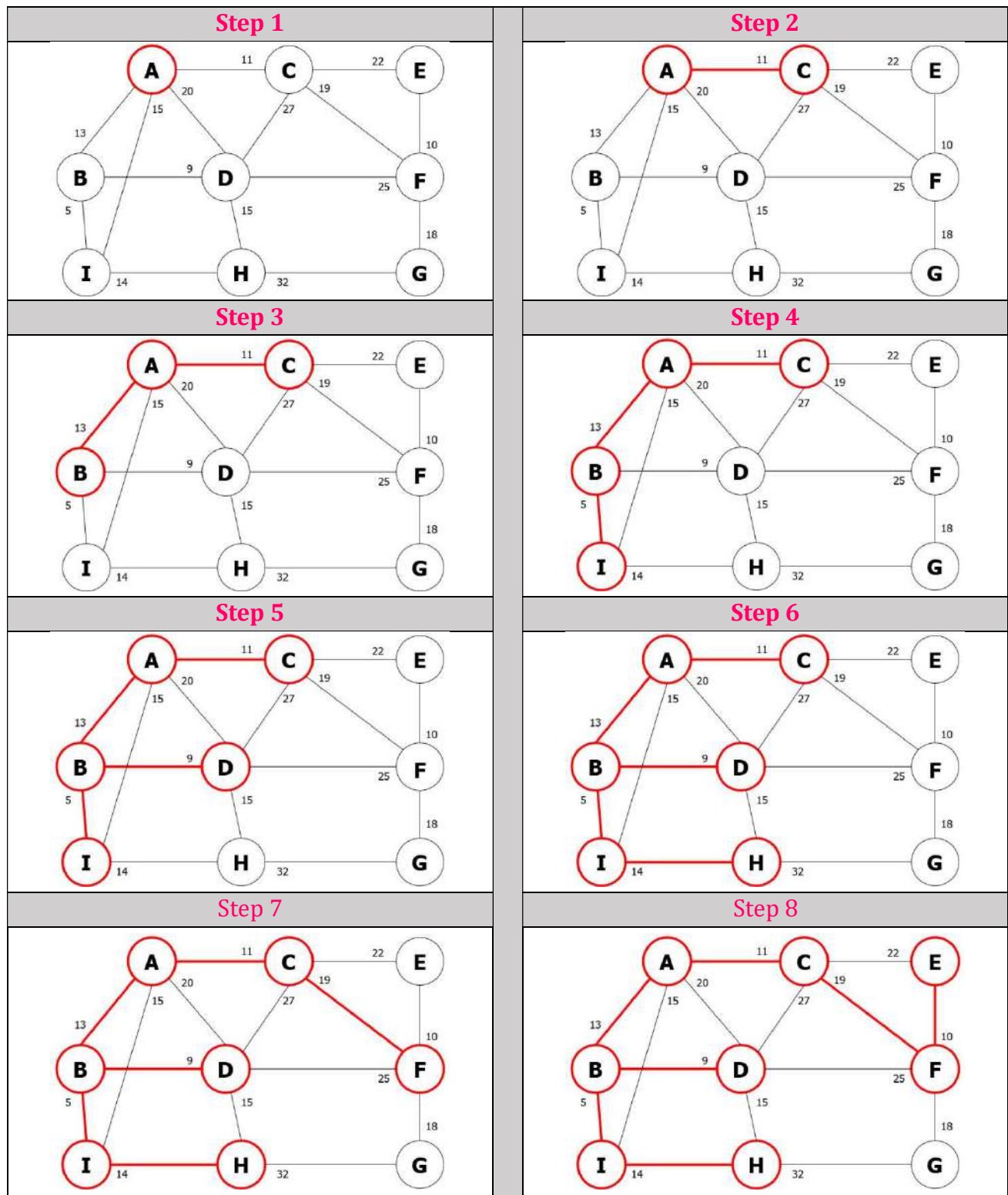
1. Choose an arbitrary starting vertex for the MST.
2. Repeat until all vertices are in the MST:
 - Find the smallest edge connecting a tree vertex to a fringe vertex.
 - Add the edge to the MST if it doesn't form a cycle.
3. Return the MST.

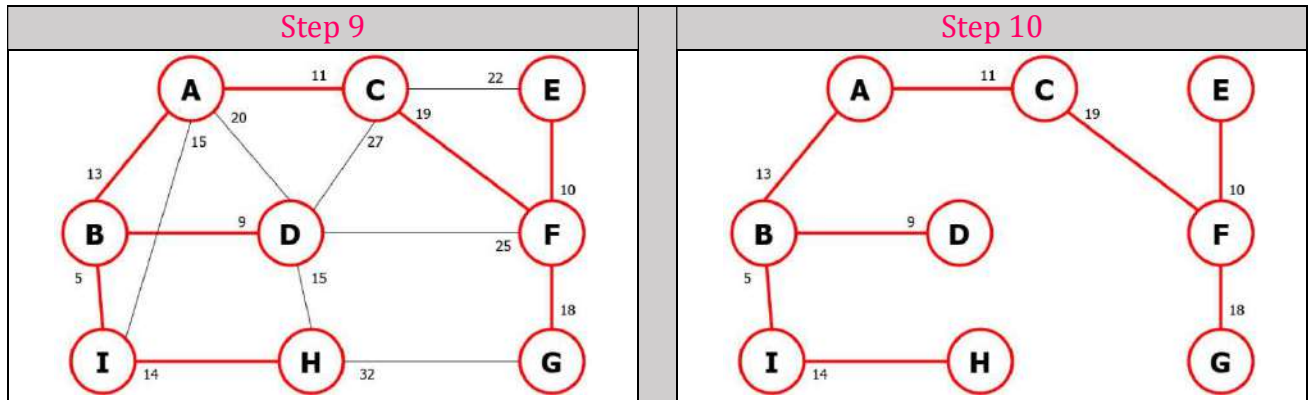
✂ Example 1:

✂ >> Show the detailed simulation steps for the following graph using **Prim's Algorithm** for finding the **Minimum spanning tree (MST)** & also find the **Total Cost**.



→ Simulation of Prim's Algorithm



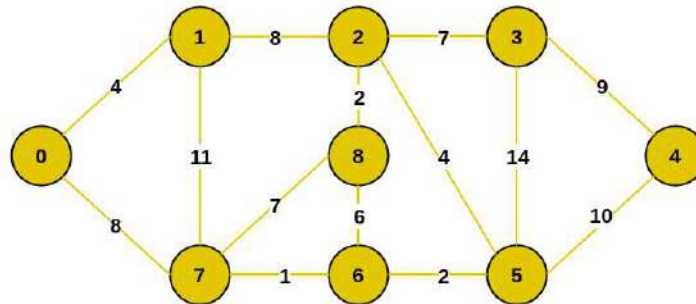


Total Cost:

$$14+5+9+13+11+19+10+18 = 99$$

✂ **Example 2:**

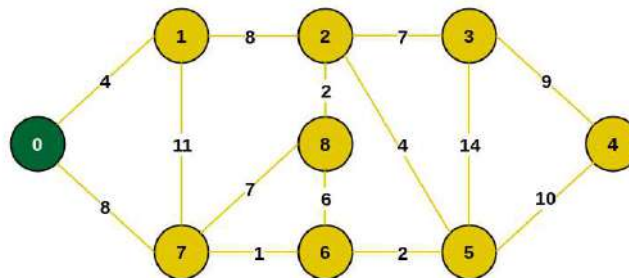
✂ >> Show the detailed simulation steps for the following graph using **Prim's Algorithm** for finding the **Minimum spanning tree (MST)** & also find the **Total Cost**.



Example of a Graph

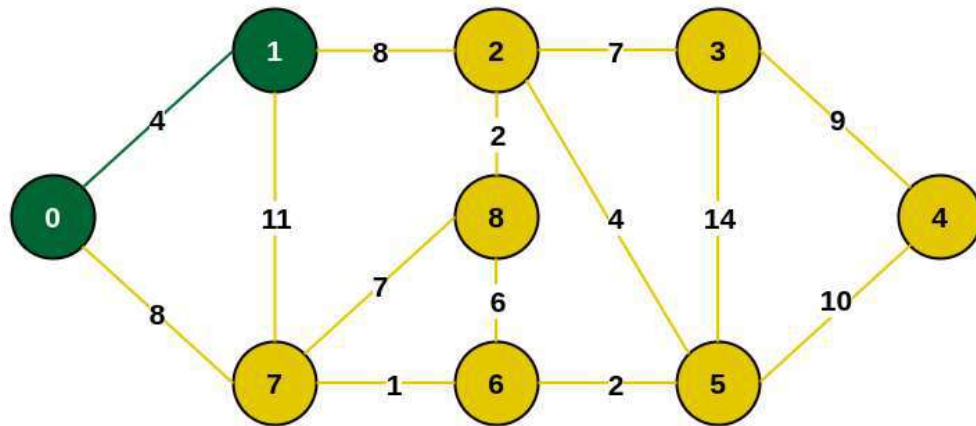
➔ **Simulation of Prim's Algorithm**

Step 1:



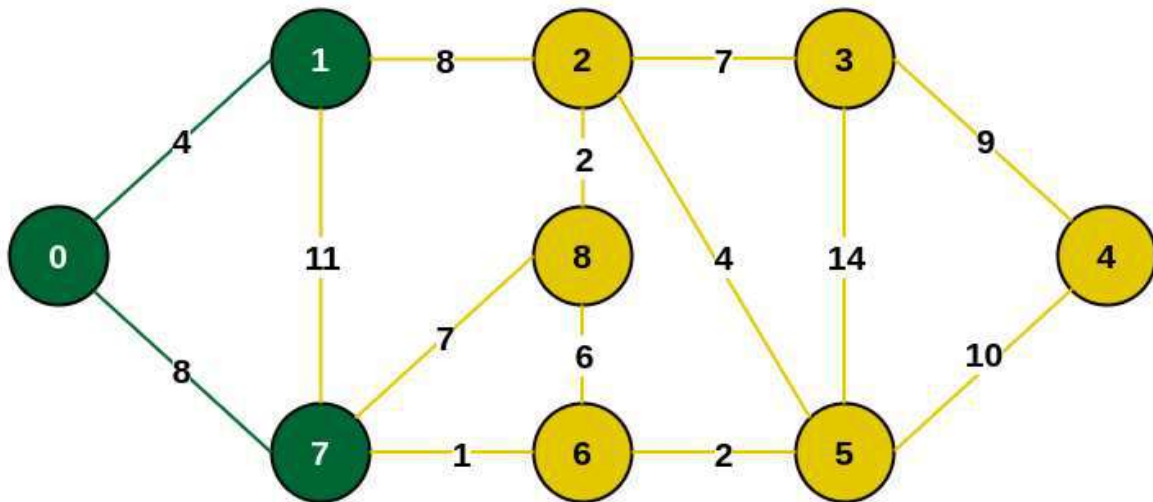
Select an arbitrary starting vertex. Here we have selected 0

Step 2:



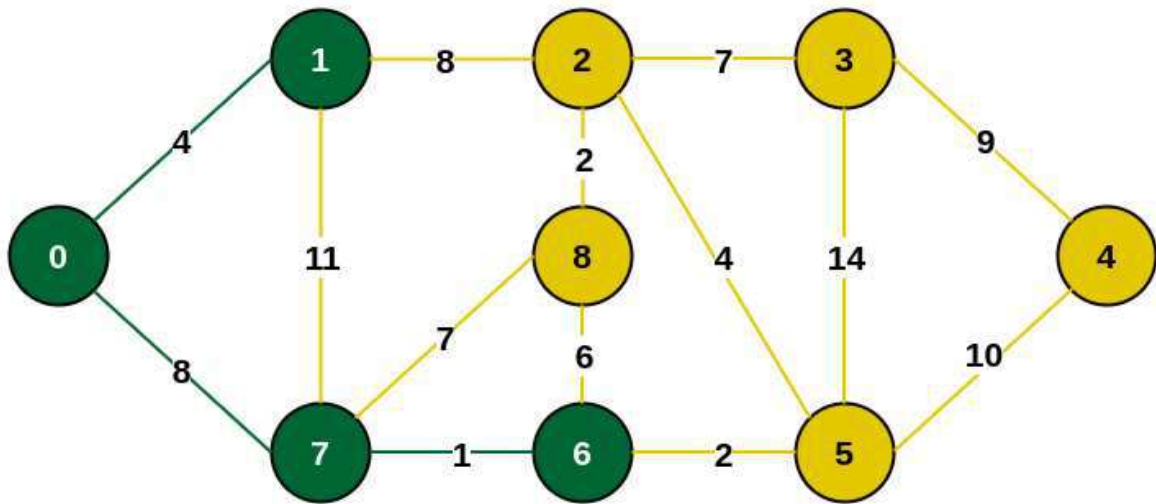
Minimum weighted edge from MST to other vertices is 0-1 with weight 4

Step 3:



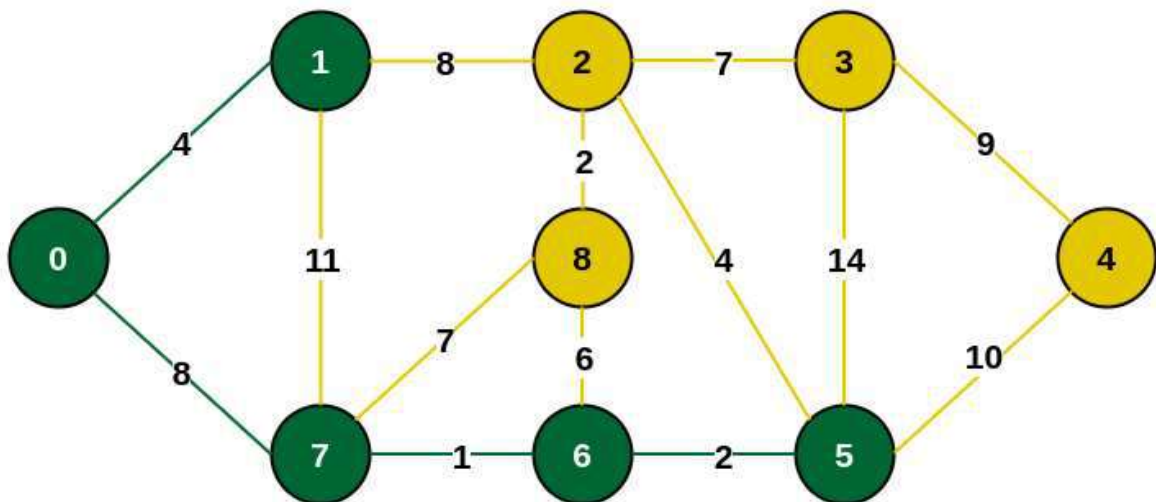
Minimum weighted edge from MST to other vertices is 0-7 with weight 8

Step 4:



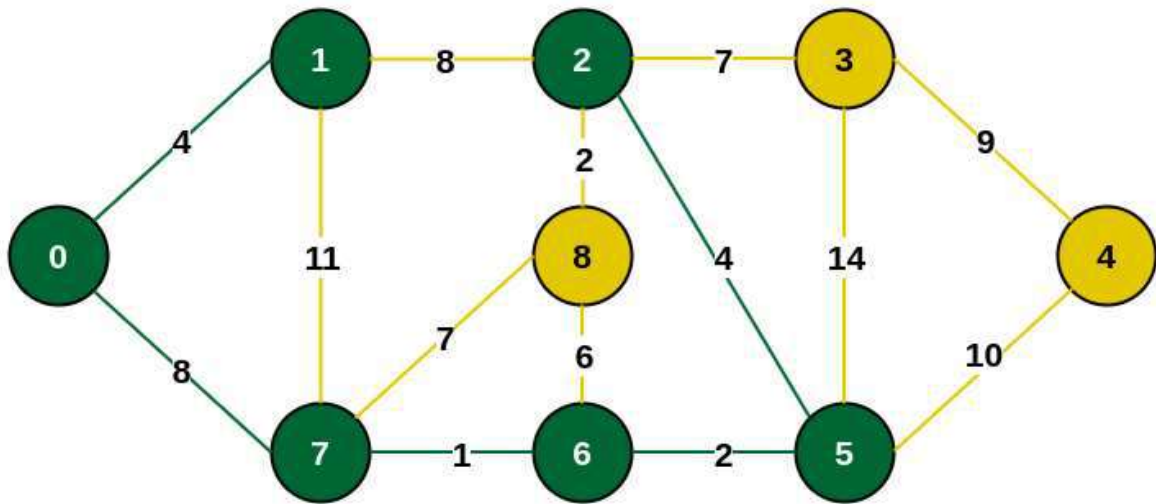
Minimum weighted edge from MST to other vertices is 7-6 with weight 1

Step 5:



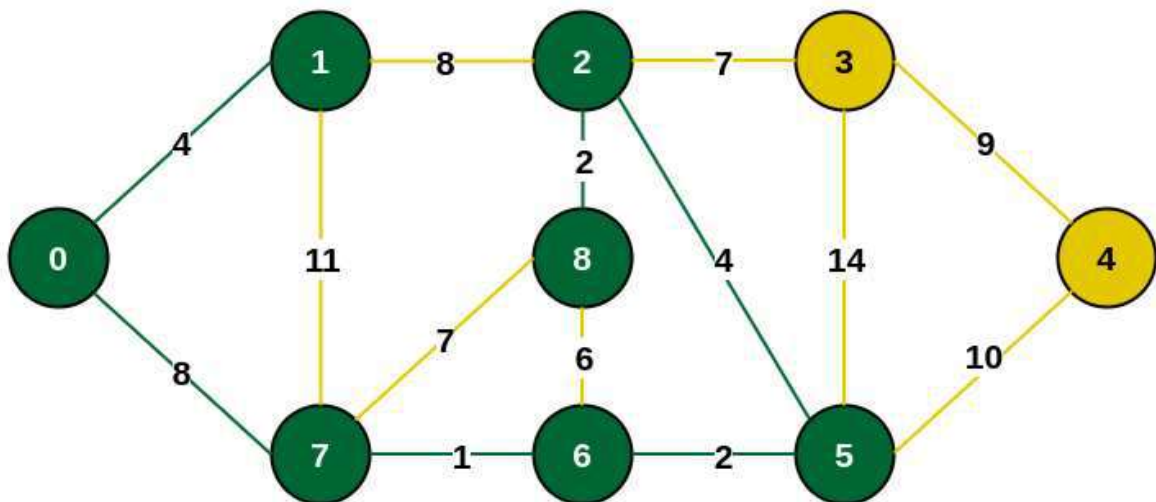
Minimum weighted edge from MST to other vertices is 6-5 with weight 2

Step 6:



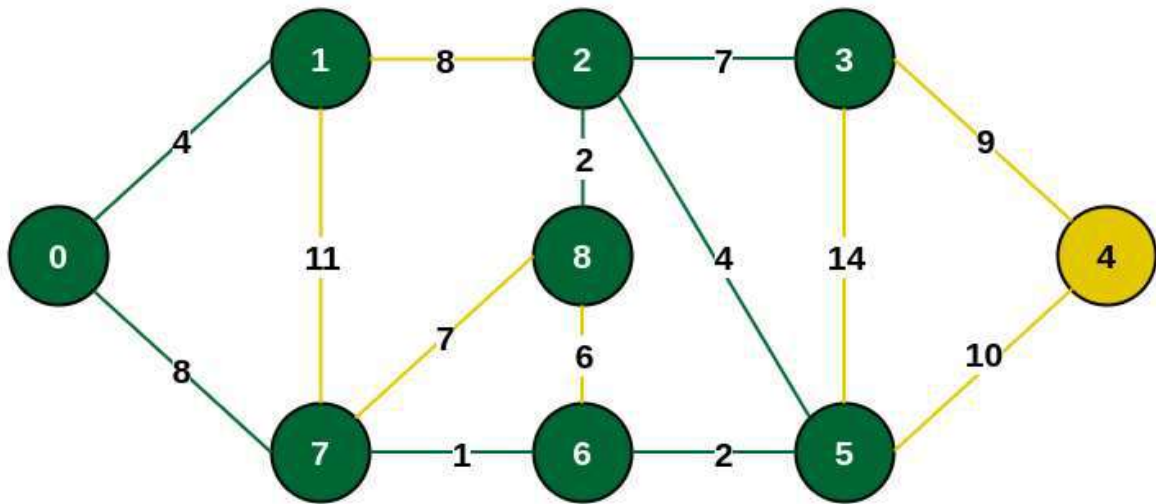
Minimum weighted edge from MST to other vertices is 5-2 with weight 4

Step 7:



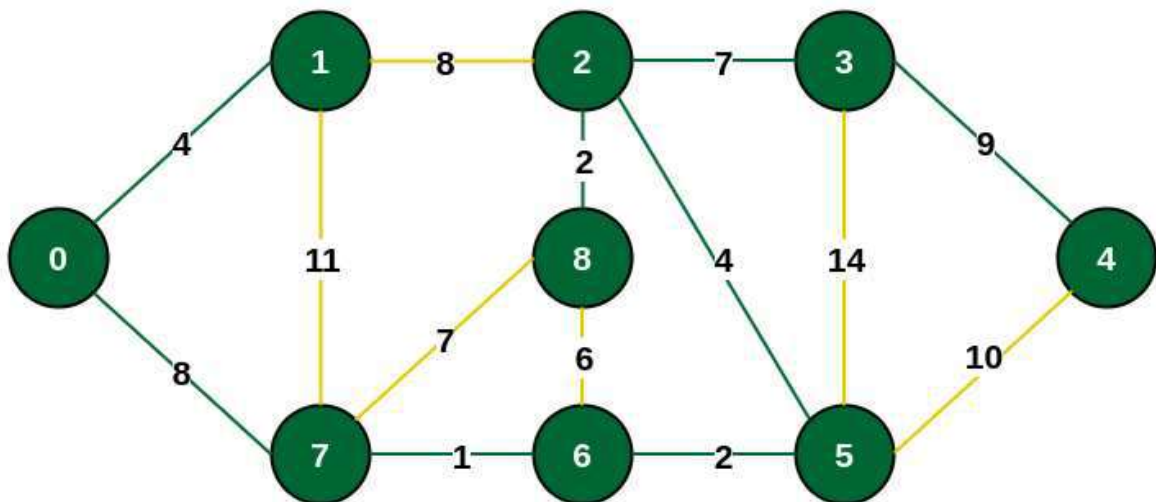
Minimum weighted edge from MST to other vertices is 2-8 with weight 2

Step 8:



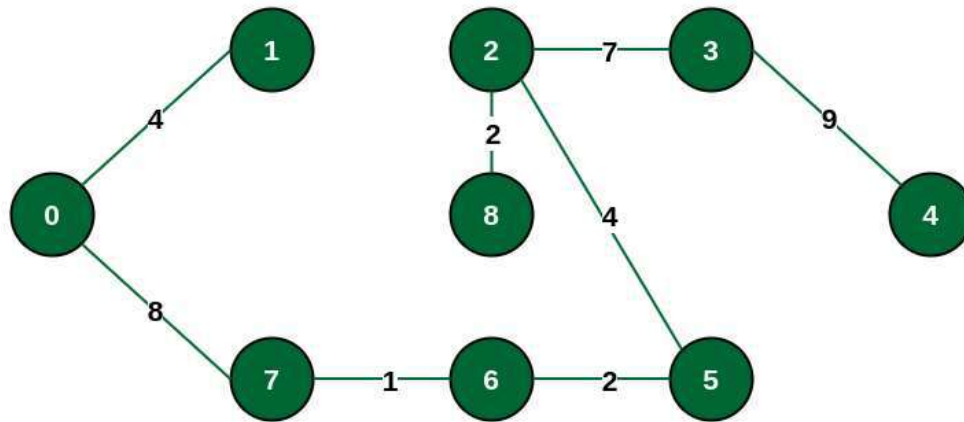
Minimum weighted edge from MST to other vertices is 2-3 with weight 7

Step 9:



Minimum weighted edge from MST to other vertices is 3-4 with weight 9

Step 10:

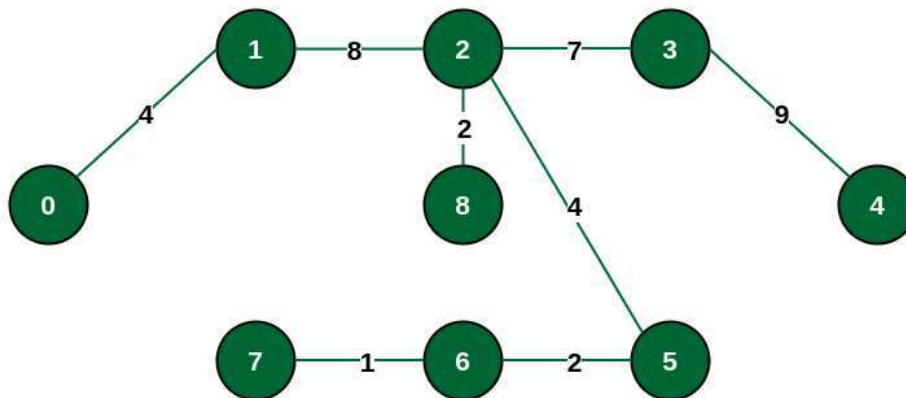


The final structure of MST

Total Cost:

$$4+8+1+2+4+2+7+9 = 37$$

Step 10.2:



Alternative MST structure

Total Cost:

$$4+8+7+2+9+4+2+1 = 37$$

KRUSKAL'S ALGORITHM

Kruskal's Algorithm

Kruskal's Algorithm Conditions:

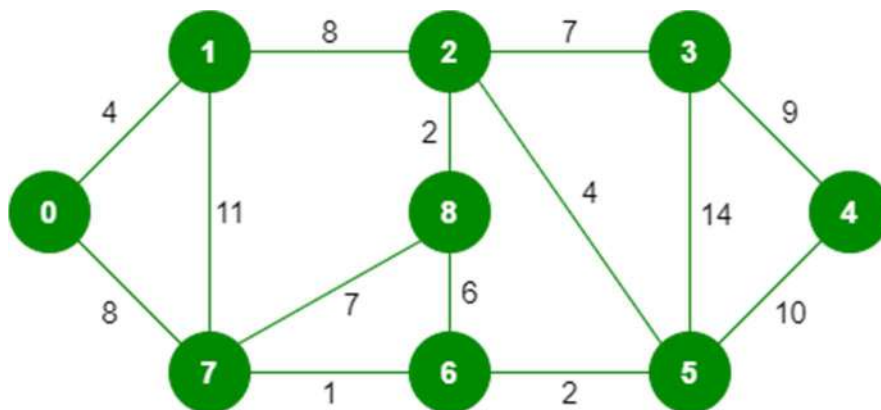
1. **Undirected Graph.**
2. **Weighted Graph** with non-negative weights.
3. **Can be connected or disconnected.**
4. **Greedy Approach:** Select edges in increasing order of weight without forming cycles.

Kruskal's Algorithm Steps:

1. Sort all the edges in non-decreasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
3. Repeat step#2 until there are $(V-1)$ edges in the spanning tree.

✂ Example 1:

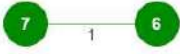
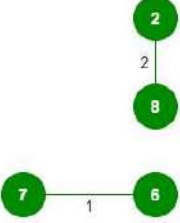
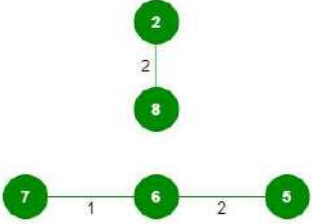
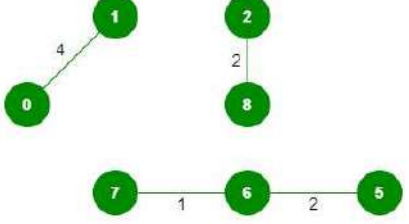
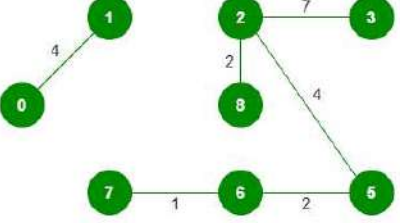
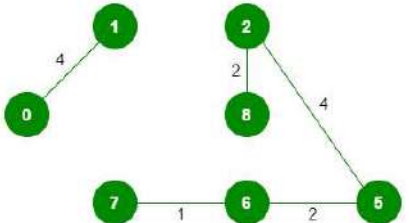
✂ >> Show the detailed simulation steps for the following graph using **Kruskal's Algorithm** for finding the **Minimum spanning tree (MST)** & also find the **Total Cost**.



Weight	Source	Destination
1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3
7	7	8
8	0	7
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

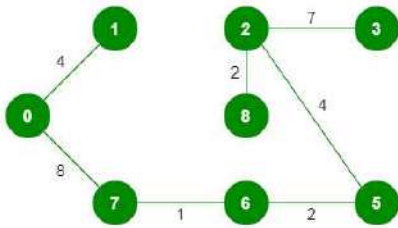
➔ Now pick all edges one by one from the sorted list of edges.

➔ Simulation of Kruskal's Algorithm

Step 1 Add edge 7-6 in the MST  MST using Kruskal's algorithm	Step 2 Add edge 8-2 in the MST  MST using Kruskal's algorithm
Step 3 Add edge 6-5 in the MST  MST using Kruskal's algorithm	Step 4 Add edge 0-1 in the MST  MST using Kruskal's algorithm
Step 6 Add edge 2-3 in the MST as 8-6 can't be added  MST using Kruskal's algorithm	Step 5 Add edge 2-5 in the MST  MST using Kruskal's algorithm

Step 7

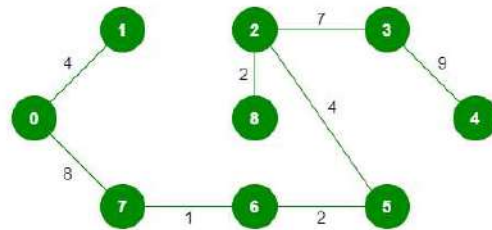
Add edge 0-7 in the MST as 7-8 can't be added



MST using Kruskal's algorithm

Step 8

Add edge 3-4 in the MST. It completes the MST



MST using Kruskal's algorithm

Total Cost:

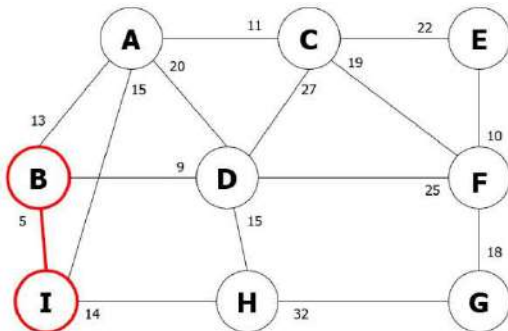
$$4+8+1+2+4+2+7+9 = 37$$

✂ Example 2:

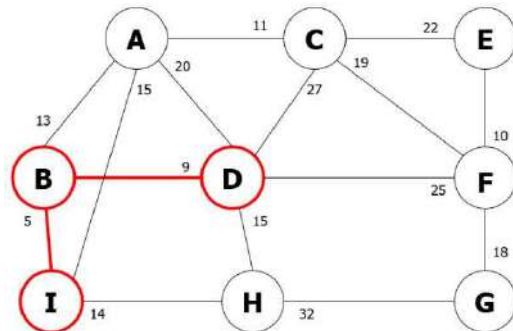
✂ >> Show the detailed simulation steps for the following graph using **Kruskal's Algorithm** for finding the **Minimum spanning tree (MST)** & also find the **Total Cost**.

➔ **Simulation of Kruskal's Algorithm:**

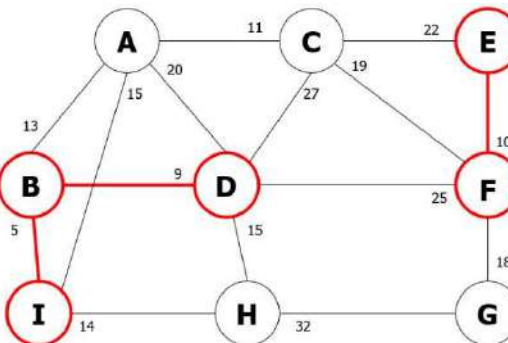
Step 1



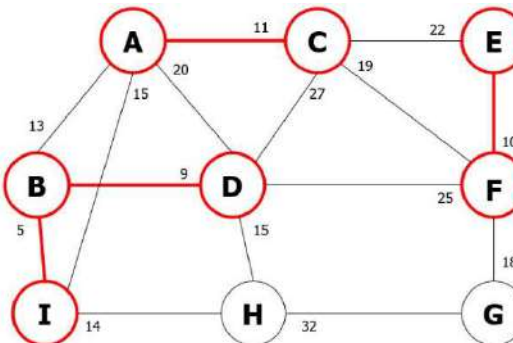
Step 2



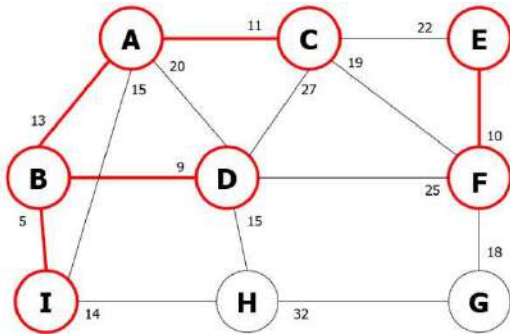
Step 3



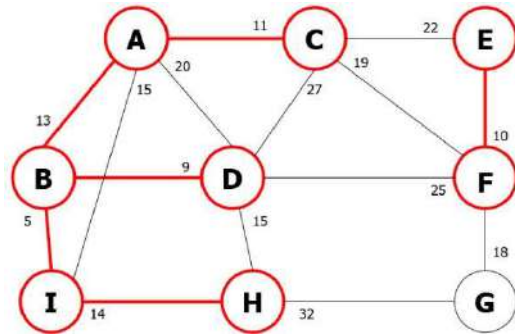
Step 4



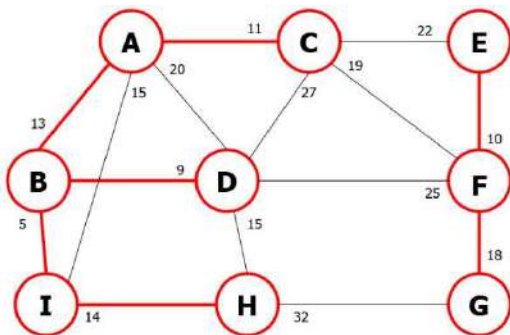
Step 5



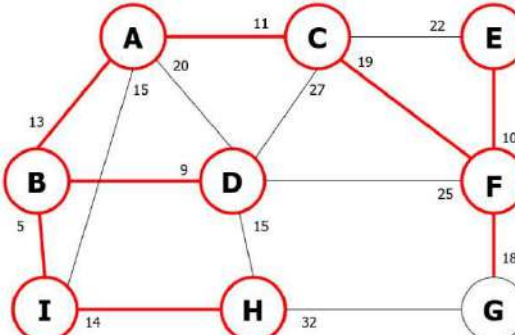
Step 6



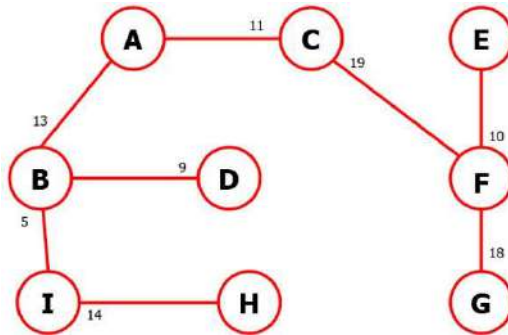
Step 7



Step 8



Step 9



Total Cost:

$$14+5+9+13+11+19+10+18 = 99$$

DIJKSTRA'S ALGORITHM

Dijkstra's Algorithm

Dijkstra's Algorithm Conditions:

1. Directed or undirected graph.
2. Weighted graph with non-negative weights.
3. Finds shortest path from a single source.
4. No negative weight cycles.

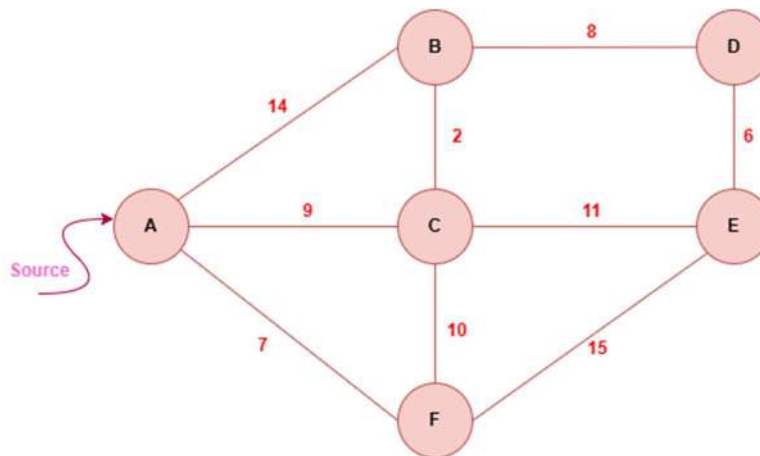
Dijkstra's Algorithm Steps:

1. Initialize distances: source = 0, others = ∞ .
2. Use a priority queue to select the vertex with the smallest distance.
3. Relax neighbors: update distances if shorter paths are found.
4. Repeat until all vertices are processed.

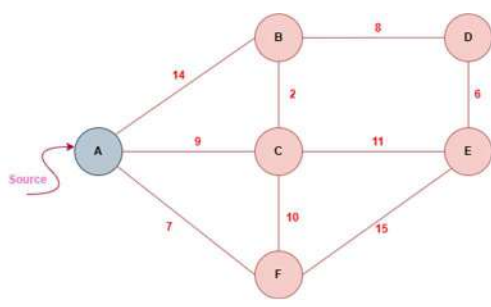
→ Simulation of Dijkstra's Algorithm for Undirected Graph:

✂ Example 1:

✂ >>Apply **Dijkstra's Algorithm** in the following graph and find the single source shortest path on the following graph using "**A**" as the starting/Source node and **find** the path **A** to **D**.

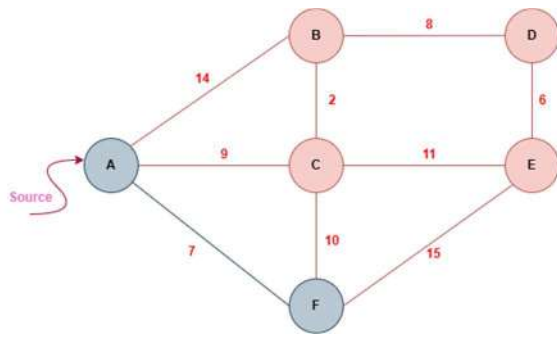


➔Step1:



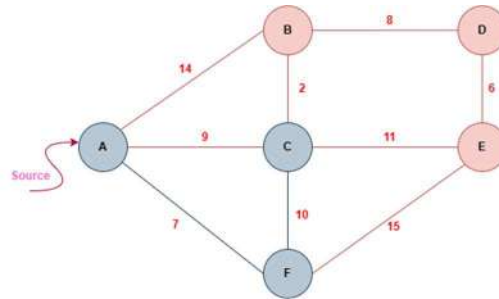
	A	B	C	D	E	F
Initialize	0	∞	∞	∞	∞	∞
A	0	14^{AB}	9^{AC}	∞	∞	7^{AF}

➔Step2:



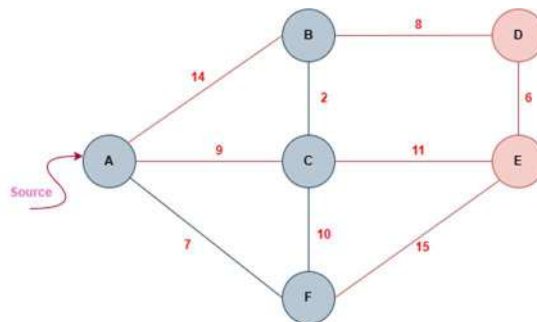
	A	B	C	D	E	F
Initialize	0	∞	∞	∞	∞	∞
A	0	14^{AB}	9^{AC}	∞	∞	7^{AF}
F	0	14^{AB}	9^{AC}	∞	22^{AFE}	7^{AF}
	0					
	0					
	0					
	0					

→ Step3:



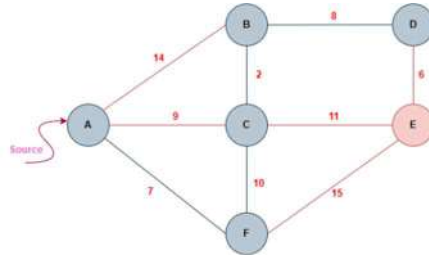
	A	B	C	D	E	F
Initialize	0	∞	∞	∞	∞	∞
A	0	14^{AB}	9^{AC}	∞	∞	7^{AF}
F	0	14^{AB}	9^{AC}	∞	22^{AFE}	7^{AF}
C	0	11^{ACB}	9^{AC}	∞	20^{ACE}	7^{AF}
	0					
	0					
	0					

→ Step4:



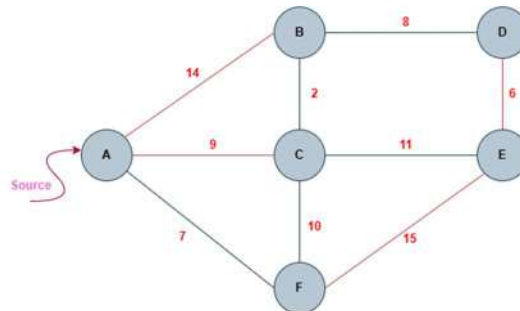
	A	B	C	D	E	F
Initialize	0	∞	∞	∞	∞	∞
A	0	14^{AB}	9^{AC}	∞	∞	7^{AF}
F	0	14^{AB}	9^{AC}	∞	22^{AFE}	7^{AF}
C	0	11^{ACB}	9^{AC}	∞	20^{ACE}	7^{AF}
B	0	11^{ACB}	9^{AC}	19^{ACBD}	20^{ACE}	7^{AF}
	0					
	0					

➔ Step4:



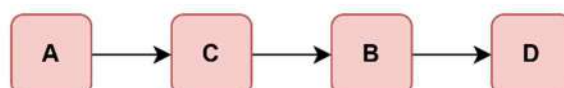
	A	B	C	D	E	F
Initialize	0	∞	∞	∞	∞	∞
A	0	14^{AB}	9^{AC}	∞	∞	7^{AF}
F	0	14^{AB}	9^{AC}	∞	22^{AFE}	7^{AF}
C	0	11^{ACB}	9^{AC}	∞	20^{ACE}	7^{AF}
B	0	11^{ACB}	9^{AC}	19^{ACBD}	20^{ACE}	7^{AF}
D	0	11^{ACB}	9^{AC}	19^{ACBD}	20^{ACE}	7^{AF}
	0					

➔ Step5:



	A	B	C	D	E	F
Initialize	0	∞	∞	∞	∞	∞
A	0	14^{AB}	9^{AC}	∞	∞	7^{AF}
F	0	14^{AB}	9^{AC}	∞	22^{AFE}	7^{AF}
C	0	11^{ACB}	9^{AC}	∞	20^{ACE}	7^{AF}
B	0	11^{ACB}	9^{AC}	19^{ACBD}	20^{ACE}	7^{AF}
D	0	11^{ACB}	9^{AC}	19^{ACBD}	20^{ACE}	7^{AF}
E	0	11^{ACB}	9^{AC}	19^{ACBD}	20^{ACE}	7^{AF}

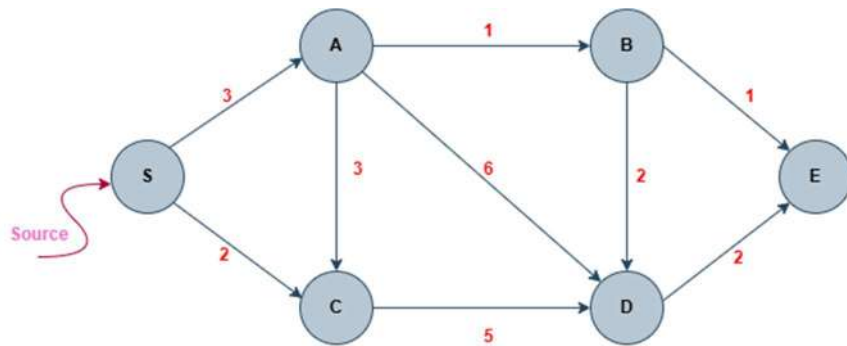
Shortest path A to D:



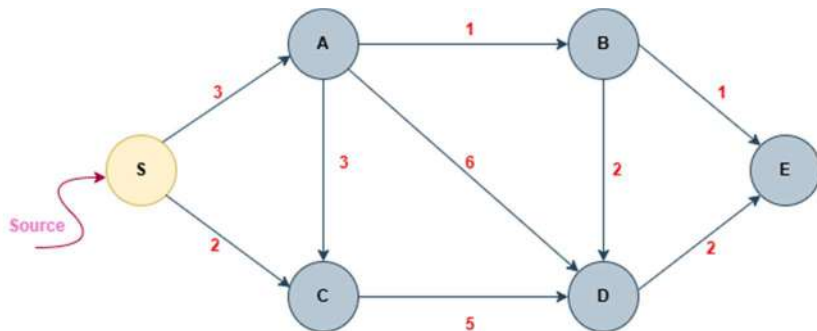
→ Simulation of Dijkstra's Algorithm for Directed Graph:

✂ Example 2:

✂ >>Apply **Dijkstra's Algorithm** in the following graph and find the single source shortest path on the following graph using "S" as the starting/Source node and **find the path S to E.**

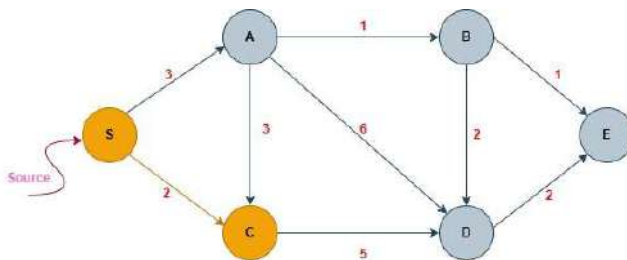


→ Step1:



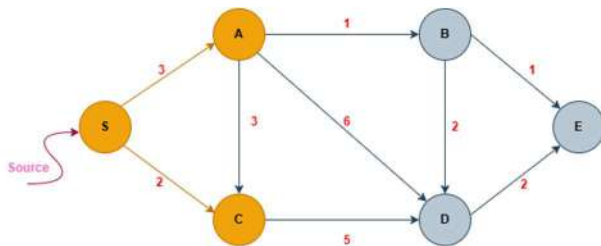
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	3^{SA}	∞	2^{SC}	∞	∞
	0					
	0					
	0					
	0					
	0					

Step2:



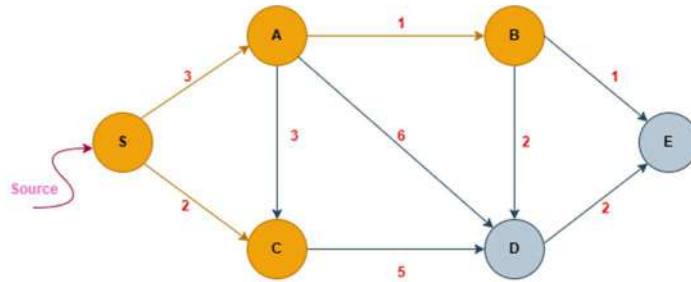
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	3^{SA}	∞	2^{SC}	∞	∞
C	0	3^{SA}	∞	2^{SC}	7^{SCD}	∞
	0					
	0					
	0					
	0					

Step3:



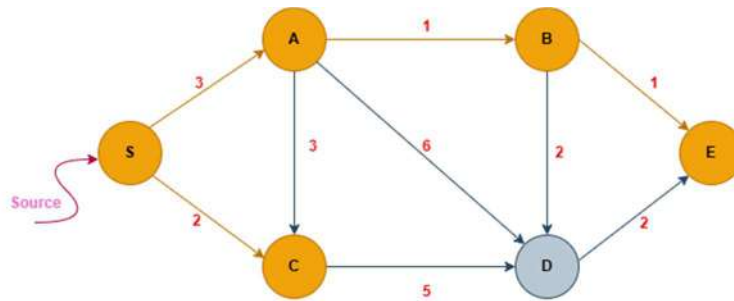
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	3^{SA}	∞	2^{SC}	∞	∞
C	0	3^{SA}	∞	2^{SC}	7^{SCD}	∞
A	0	3^{SA}	4^{SAB}	2^{SC}	7^{SCD}	∞
	0					
	0					
	0					

Step4:



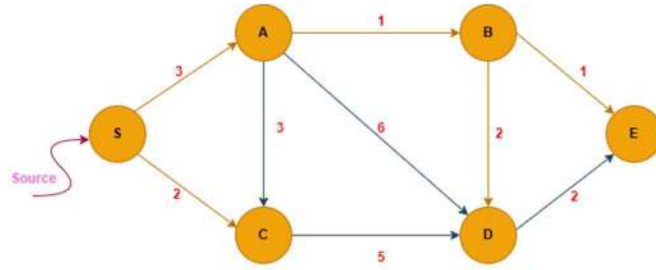
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	3^{SA}	∞	2^{SC}	∞	∞
C	0	3^{SA}	∞	2^{SC}	7^{SCD}	∞
A	0	3^{SA}	4^{SAB}	2^{SC}	7^{SCD}	∞
B	0	3^{SA}	4^{SAB}	2^{SC}	6^{SABD}	5^{SABE}
	0					
	0					

Step5:

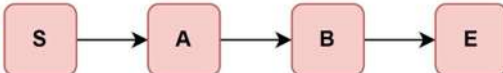


	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	3^{SA}	∞	2^{SC}	∞	∞
C	0	3^{SA}	∞	2^{SC}	7^{SCD}	∞
A	0	3^{SA}	4^{SAB}	2^{SC}	7^{SCD}	∞
B	0	3^{SA}	4^{SAB}	2^{SC}	6^{SABD}	5^{SABE}
E	0	3^{SA}	4^{SAB}	2^{SC}	6^{SABD}	5^{SABE}
	0					

Step6:



	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	3^{SA}	∞	2^{SC}	∞	∞
C	0	3^{SA}	∞	2^{SC}	7^{SCD}	∞
A	0	3^{SA}	4^{SAB}	2^{SC}	7^{SCD}	∞
B	0	3^{SA}	4^{SAB}	2^{SC}	6^{SABD}	5^{SABE}
E	0	3^{SA}	4^{SAB}	2^{SC}	6^{SABD}	5^{SABE}
D	0	3^{SA}	4^{SAB}	2^{SC}	6^{SABD}	5^{SABE}

Shortest path S to E: 

Dijkstra's Algorithm Pseudo Code

```
function dijkstra(G, S)
  for each vertex V in G
    distance[V] <- infinite
    previous[V] <- NULL
    If V != S, add V to Priority Queue Q
  distance[S] <- 0

  while Q IS NOT EMPTY
    U <- Extract MIN from Q
    for each unvisited neighbour V of U
      tempDistance <- distance[U] + edge_weight(U, V)
      if tempDistance < distance[V]
        distance[V] <- tempDistance
        previous[V] <- U
  return distance[], previous[]
```

BELLMAN-FORD'S ALGORITHM

Bellman-Ford's Algorithm

→ Bellman-Ford's Algorithm Conditions:

1. Works on directed/undirected graphs.
2. Handles positive or negative weights.
3. No negative weight cycles.
4. Single source shortest path.

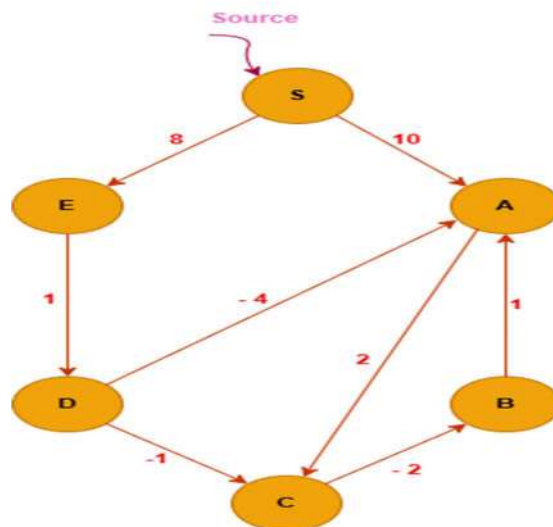
→ Bellman-Ford's Algorithm Steps:

1. Initialize distances: source = 0, others = ∞ .
2. Relax all edges for $(V-1)$ iterations.
3. Check for negative weight cycles.
4. Output shortest distances or detect a negative cycle.

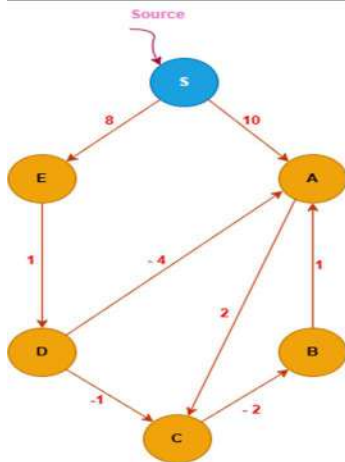
→ Simulation of Bellman-Ford's Algorithm:

✂ Example 1:

>>Apply **Bellman-fords algorithm** in the following graph and find the single source shortest path on the following graph using "S" as the starting/Source node and **find the path S to B**.

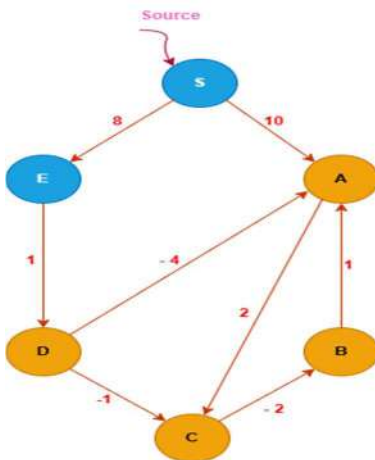


➔Step1:



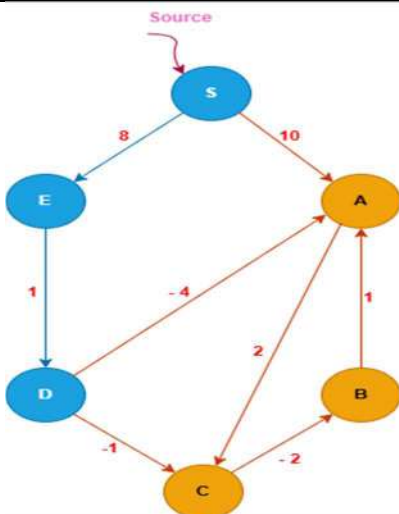
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	10^{SA}	∞	∞	∞	8^{SE}

➔Step2:



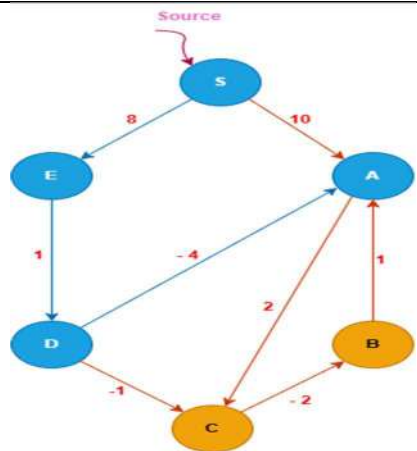
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	10^{SA}	∞	∞	∞	8^{SE}
E	0	10^{SA}	∞	∞	9^{SED}	8^{SE}

➔Step3:



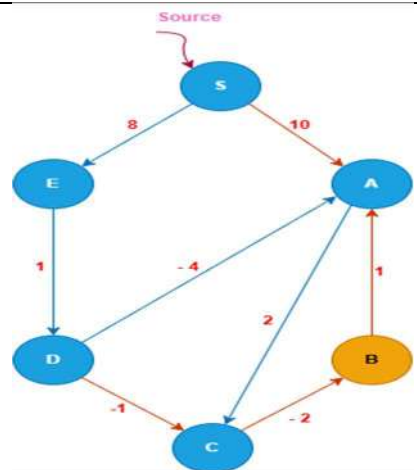
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	10^{SA}	∞	∞	∞	8^{SE}
E	0	10^{SA}	∞	∞	9^{SED}	8^{SE}
D	0	5^{SEDA}	∞	8^{SED}	9^{SED}	8^{SE}

➔ Step4:



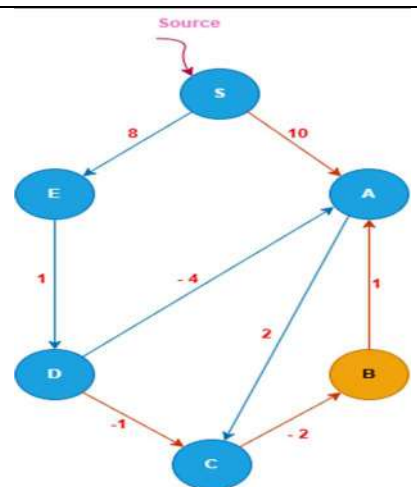
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	10^{SA}	∞	∞	∞	8^{SE}
E	0	10^{SA}	∞	∞	9^{SED}	8^{SE}
D	0	5^{SEDA}	∞	8^{SED}	9^{SED}	8^{SE}
A	0	5^{SEDA}	∞	7^{SEDAC}	9^{SED}	8^{SE}

➔ Step5:



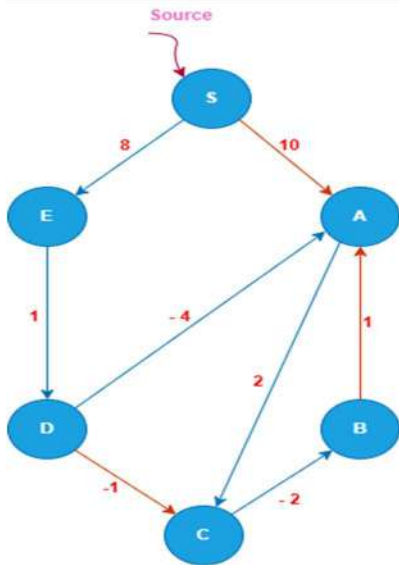
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	10^{SA}	∞	∞	∞	8^{SE}
E	0	10^{SA}	∞	∞	9^{SED}	8^{SE}
D	0	5^{SEDA}	∞	8^{SED}	9^{SED}	8^{SE}
A	0	5^{SEDA}	∞	7^{SEDAC}	9^{SED}	8^{SE}
C	0	5^{SEDA}	5^{SEDACB}	7^{SEDAC}	9^{SED}	8^{SE}

➔ Step6:



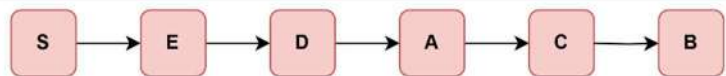
	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	10^{SA}	∞	∞	∞	8^{SE}
E	0	10^{SA}	∞	∞	9^{SED}	8^{SE}
D	0	5^{SEDA}	∞	8^{SED}	9^{SED}	8^{SE}
A	0	5^{SEDA}	∞	7^{SEDAC}	9^{SED}	8^{SE}
C	0	5^{SEDA}	5^{SEDACB}	7^{SEDAC}	9^{SED}	8^{SE}
B	0	5^{SEDA}	5^{SEDACB}	7^{SEDAC}	9^{SED}	8^{SE}

→ Step7:



	S	A	B	C	D	E
Initialize	0	∞	∞	∞	∞	∞
S	0	10^{SA}	∞	∞	∞	8^{SE}
E	0	10^{SA}	∞	∞	9^{SED}	8^{SE}
D	0	5^{SEDA}	∞	8^{SED}	9^{SED}	8^{SE}
A	0	5^{SEDA}	∞	7^{SEDAC}	9^{SED}	8^{SE}
C	0	5^{SEDA}	5^{SEDACB}	7^{SEDAC}	9^{SED}	8^{SE}
B	0	5^{SEDA}	5^{SEDACB}	7^{SEDAC}	9^{SED}	8^{SE}
Last Update	0	5^{SEDA}	5^{SEDACB}	7^{SEDAC}	9^{SED}	8^{SE}

Shortest path S to B:



Bellman-Ford's Algorithm Pseudo Code

```

function BellmanFord(Graph, source):
    // Step 1: Initialization
    for each vertex v in Graph:
        distance[v] := infinity
        predecessor[v] := null
    distance[source] := 0

    // Step 2: Relaxation
    for i from 1 to |V|-1:
        for each edge (u, v) in Graph:
            if distance[u] + weight(u, v) < distance[v]:
                distance[v] := distance[u] + weight(u, v)
                predecessor[v] := u

    // Step 3: Check for negative cycles
    for each edge (u, v) in Graph:
        if distance[u] + weight(u, v) < distance[v]:
            return "Graph contains a negative cycle"

    // Step 4: Output the shortest paths
    return distance[], predecessor[]
  
```

**ALL PAIR
SHORTEST PATH**

All Pair Shortest Path

All Pair Shortest Path Conditions:

1. Works on directed or undirected graphs.
2. Handles positive weights (negative weights depend on the algorithm).
3. Graph can be connected or disconnected.
4. For negative weights, **Floyd-Warshall** algorithm is required; **Dijkstra** is used for non-negative weights.

All Pair Shortest Path Algorithms:

Floyd-Warshall:

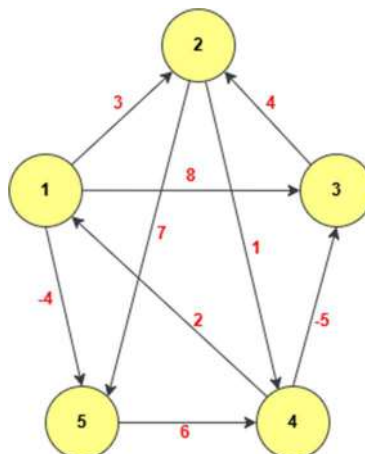
1. Initialize $\text{dist}[i][j]$ with edge weights, 0 for $i = j$, and ∞ for no edge.
2. Update distances using: $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$ for all k .
3. Output $\text{dist}[i][j]$ for shortest paths between all pairs.

Dijkstra:

1. Run Dijkstra's algorithm for each vertex as the source.
2. Store results in an $N \times N$ distance matrix.

✂ Example 1:

➔ Find the All Pair Shortest Path using **Floyd-Warshall's** Algorithms for the given graph.



➔ Step1:

➔ Determine the adjacency matrix for the given graph:

$D^0 =$

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

➔ Step2:

➔ Calculate the D^1 Matrix from the D^0 Matrix :

$D^1 =$

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

➔ Step3:

➔ Calculate the D^2 Matrix from the D^1 Matrix :

$D^2 =$

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

➔ Step4:

➔ Calculate the D^3 Matrix from the D^2 Matrix :

$D^3 =$

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	-1	-5	0	-2
5	∞	∞	∞	6	0

→ Step5:

→ Calculate the D^4 Matrix from the D^3 Matrix :

$D^4 =$

	1	2	3	4	5
1	0	3	-1	4	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

→ Step6:

→ Calculate the D^5 Matrix from the D^4 Matrix :

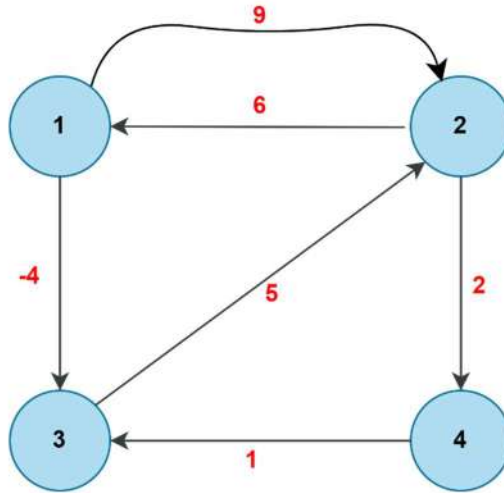
$D^5 =$

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

→ So, after applying the **Floyd-Warshall Algorithm**, D^5 is the shortest path distance for the given matrix.

☞ Example 2:

→ Find the All Pair Shortest Path using **Floyd-Warshalls** Algorithms for the given graph.



→ **Step1:**

→ Determine the adjacency matrix for the given graph:

$D^0 =$

	1	2	3	4
1	0	9	-4	∞
2	6	0	∞	2
3	∞	5	0	∞
4	∞	∞	1	0

➔ Step2:

➔ Calculate the D^1 Matrix from the D^0 Matrix :

$D^1 =$

	1	2	3	4
1	0	9	-4	∞
2	6	0	2	2
3	∞	5	0	∞
4	∞	∞	1	0

➔ Step3:

➔ Calculate the D^2 Matrix from the D^1 Matrix :

$D^2 =$

	1	2	3	4
1	0	9	-4	11
2	6	0	2	2
3	11	5	0	7
4	∞	∞	1	0

➔ Step4:

➔ Calculate the D^3 Matrix from the D^2 Matrix :

$D^3 =$

	1	2	3	4
1	0	1	-4	3
2	6	0	2	2
3	11	5	0	7
4	12	6	1	0

➔ Step5:

➔ Calculate the D^4 Matrix from the D^3 Matrix :

$D^4 =$

	1	2	3	4
1	0	1	-4	3
2	6	0	2	2
3	11	5	0	7
4	12	6	1	0

➔ So, after applying the **Floyd-Warshall Algorithm**, D^4 is the shortest path distance for the given matrix.